

Содержание

1. Механика многопоточности Java

Глава 1.1. Модель потоков JVM

JVM как процесс, поток — единица исполнения внутри неё

Запуск `java -jar app.jar` создаёт **процесс ОС** — экземпляр JVM, со своим адресным пространством. Большинство реализаций JVM работают как единственный процесс ОС, даже если внутри крутится сотня потоков.

Platform thread (обычный `Thread`) отображается на поток ОС 1:1 и планируется штатным планировщиком ОС — как и в любом другом языке с нативными потоками. Минимум два вида потоков в любом JVM-процессе:

- **main** — поток, с которого стартует `public static void main`;
- служебные потоки самой JVM (GC, обработчик сигналов) — не видны в коде, но занимают часть ресурсов процесса.

С Java 21 есть ещё **virtual threads** — их планирует не ОС, а сама JVM, мультиплексируя на пул «несущих» (carrier) platform-потоков; создаются почти бесплатно (по цене объекта, не стека ОС). Концептуально то же, что зелёные потоки/корутины/горутины в других языках, просто на уровне JVM. Дальше в курсе, скорее всего, речь про классические platform-потоки.

Общее и приватное у потока

Все потоки процесса делят **heap** — один и тот же объект доступен из разных потоков одновременно. У каждого потока свои:

- стек вызовов — локальные переменные и параметры методов не видны другим потокам;
- program counter — указатель на текущую инструкцию.

Отсюда следствие: доступ к общему состоянию (полям объектов, `static`-переменным) из нескольких потоков без координации — источник гонок и проблем видимости (Java Memory Model, тема отдельных уроков: `synchronized`, `volatile`, `java.util.concurrent`). Сам факт «потоки живут в одном процессе» уже создаёт эту проблему — не создание потоков как таковое.

Создание потока: три способа

```
// 1 — наследование от Thread
class Worker extends Thread {
    @Override
    public void run() {
        System.out.println("работаю в " + getName());
    }
}
new Worker().start();
```

```
// 2 — Runnable через отдельный класс
class Task implements Runnable {
    @Override
    public void run() {
        System.out.println("работаю в " + Thread.currentThread().getName());
    }
}
new Thread(new Task(), "worker-2").start();

// 3 — Runnable как лямбда, самый лаконичный вариант
Runnable task = () -> System.out.println("работаю в " +
Thread.currentThread().getName());
new Thread(task, "worker-1").start();
```

Официальная позиция JDK (javadoc Thread): наследование от Thread оправдано только для кастомизации самого потока — «большинству приложений это не нужно».

Предпочтительна композиция через Runnable: задача отделена от механизма её выполнения, что удобнее переиспользовать и тестировать.

start() vs run()

- `start()` — планирует выполнение **в новом** потоке (JVM просит у ОС новый поток, и уже в нём вызывается `run()`). Повторный `start()` на том же объекте `Thread` бросает `IllegalThreadStateException`.
- `run()`, вызванный напрямую — обычный вызов метода **в текущем потоке**, новый поток не появляется. Результат может выглядеть похоже, но `Thread.currentThread().getName()` покажет имя вызывающего потока (например, `main`), а не заданное новому `Thread`. Частая ошибка — перепутать эти два вызова.

Жизненный цикл и планирование — кратко

У потока есть состояния: `NEW` → `RUNNABLE` → (`BLOCKED` / `WAITING` / `TIMED_WAITING`) → `TERMINATED`. Подробно разберём триггеры каждого состояния и то, как ими распоряжается планировщик ОС — дальше в курсе. Пока важно одно практическое

следствие: порядок выполнения потоков не детерминирован и не гарантирован языком, а «больше потоков» не значит «быстрее» — у переключения между ними и конкуренции за ресурсы есть своя цена.

`join()` — дождаться завершения

```
public class JoinDemo {

    public static void main(String[] args) throws InterruptedException {
        Thread[] workers = new Thread[10];

        for (int i = 0; i < workers.length; i++) {
            workers[i] = new Thread(JoinDemo::doWork, "worker-" + i);
            workers[i].start();
        }

        for (Thread w : workers) {
            w.join(); // блокирует main, пока поток w не завершится
        }

        System.out.println("все потоки завершены");
    }

    private static void doWork() {
        for (int i = 0; i < 10; i++) {
            System.out.println("working " +
Thread.currentThread().getName());
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }
        }
    }
}
```

Без `join()` `main` может завершиться раньше рабочих потоков — JVM живёт, пока жив хотя бы один не-даемон поток, но сам `main` их не ждёт по умолчанию.

Итого

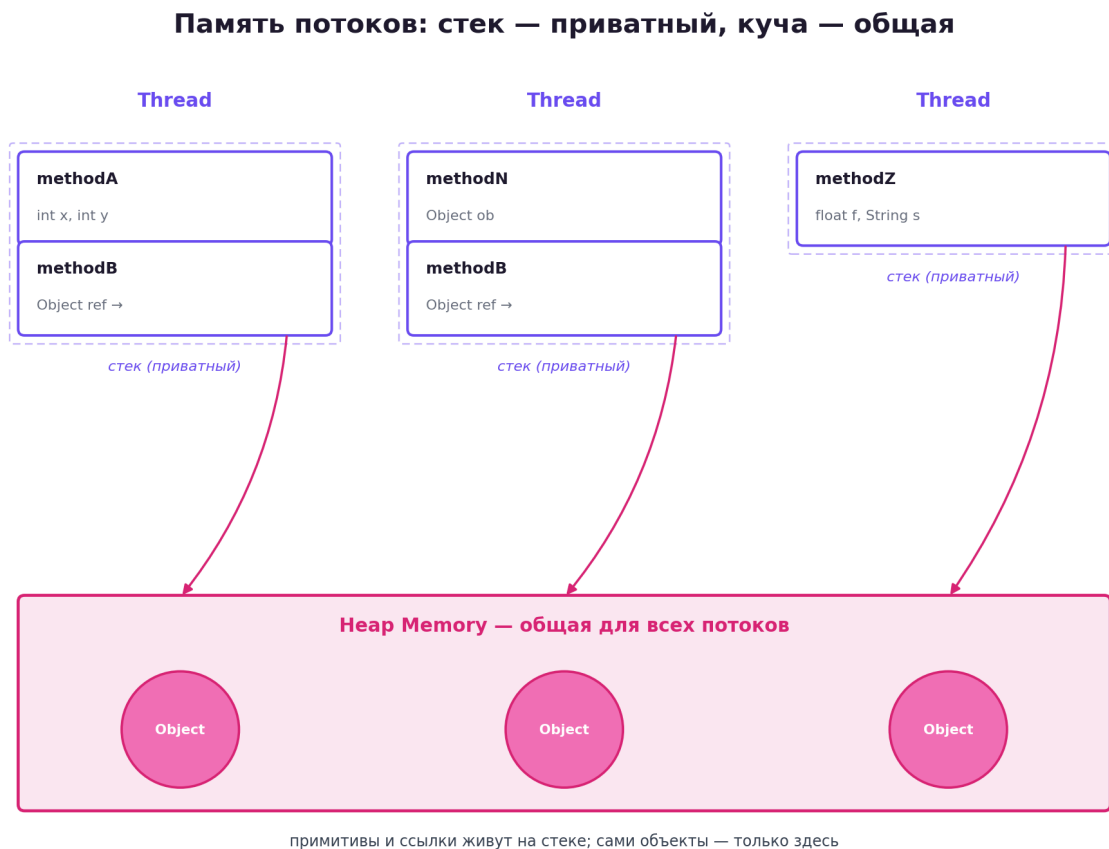
`Thread` / `Runnable` — низкоуровневый примитив; в реальном коде поверх него работают через `ExecutorService` и `java.util.concurrent` — это темы следующих уроков.

Глава 1.2. Железо и память: откуда на самом деле берутся гонки данных

Почему поток дешевле процесса

Новый **процесс** — новое адресное пространство: ОС выделяет и настраивает таблицы страниц с нуля. Новый **поток** внутри существующего процесса переиспользует то же адресное пространство и требует от ОС только стек, регистры и запись в таблице планировщика. Отсюда разница в стоимости на порядки — вот почему масштабироваться потоками проще, а не потому что «так исторически сложилось».

Стек и куча



Стек каждого потока приватен и состоит из фреймов — по одному на активный вызов метода. В нём живут локальные переменные и параметры: примитивы — как значения, объекты — как ссылки. Сами объекты живут в куче — единственной области памяти, физически общей для всех потоков процесса.

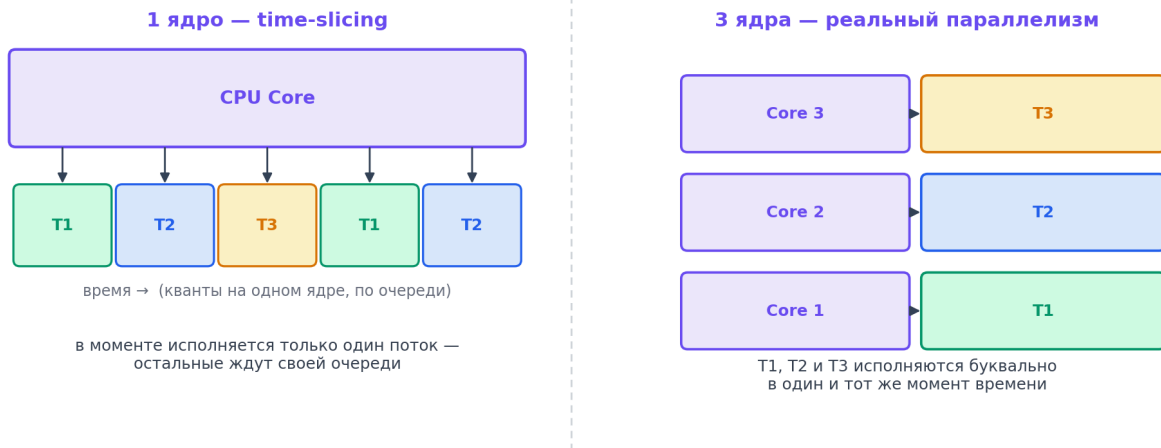
Практический вывод: локальная переменная сама по себе никогда не источник гонки — она не видна из другого потока. Гонка начинается в момент, когда ссылка на объект

уходит в общее место (поле экземпляра, `static`, элемент общей коллекции) и становится доступна больше чем одному потоку.

(Размер стека — не «1–8 МБ» как спецификация, а платформенно-зависимый default порядка 512 КБ–1 МБ, настраиваемый флагом `-Xss`.)

Ядра CPU: конкурентность ≠ параллелизм

Конкурентность vs параллелизм



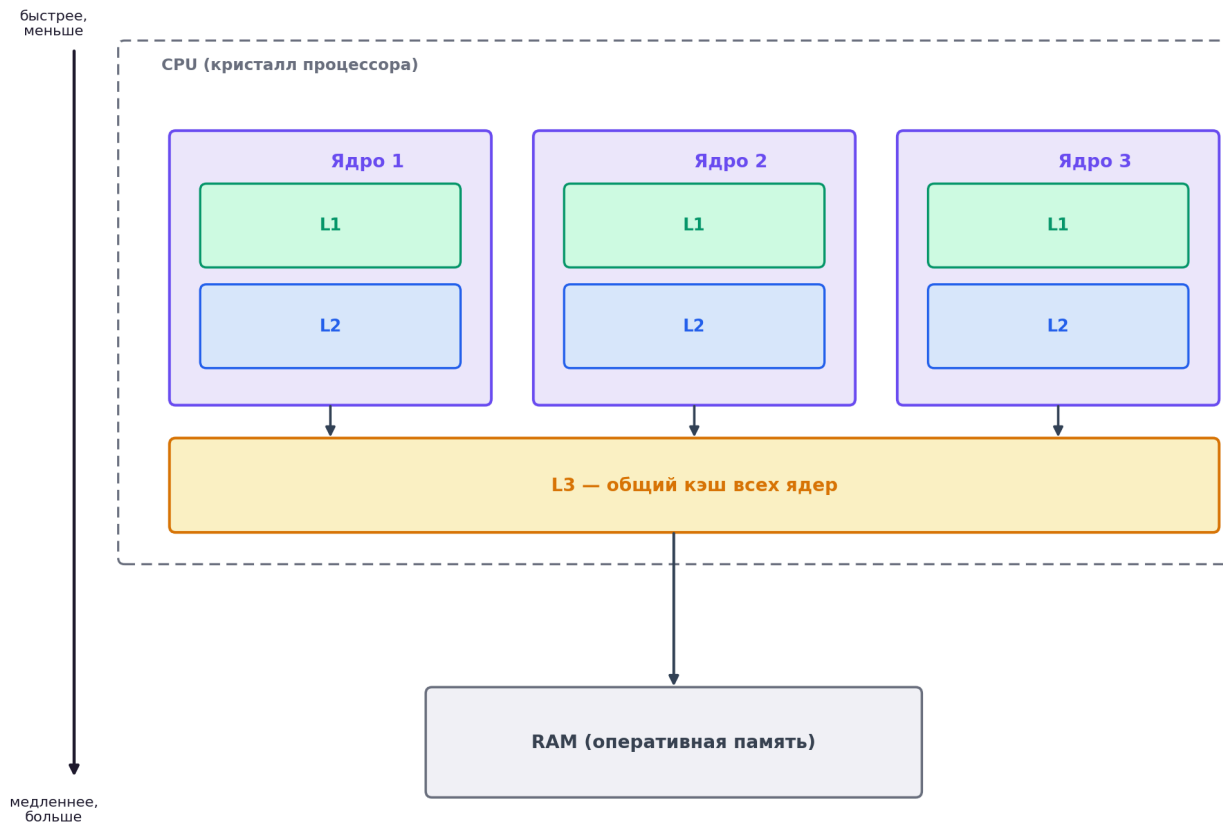
Одно ядро исполняет только одну инструкцию в момент времени. Иллюзия «несколько потоков одновременно» на одном ядре — работа планировщика ОС: он режет время на кванты и переключает потоки (*context switch, time-slicing*) — конкурентность без параллелизма. Настоящий параллелизм требует нескольких ядер.

Отсюда напрашивается «ускорение = число ядер», но это верхняя граница, которая почти не достигается. Предел формально описывает **закон Амдала**: $\text{speedup} = 1 / ((1 - p) + p/N)$, где p — доля параллелизуемого кода, N — число ядер. Если 30% работы обязаны идти последовательно, ядра сверх какого-то момента почти не помогают — а накладные расходы на синхронизацию откусывают часть выигрыша дополнительно.

Кэши CPU: где интуиция обманывает

Иерархия памяти CPU

чем ближе к ядру — тем быстрее и меньше объём



У каждого ядра свои приватные L1/L2; L3 — общий на процессор, больше по объёму, но медленнее (и всё равно на порядки быстрее RAM):

Уровень	Типичная задержка	Медленнее L1
L1	~1 нс (3–5 тактов ядра)	1×
L2	~3–7 нс	~5–10×
L3	~10–20 нс	~15–30×
RAM	~50–100 нс	~100–200×

Цифры гуляют от поколения к поколению, но суть не в точном числе, а в форме: L1→L2→L3 — плавное замедление, а **L3 → RAM — обрыв**, скачок на порядок, а не постепенное ухудшение.

Механика — эскалация наружу, не «сначала RAM»: ядро смотрит в свой L1, не находит — идёт в L2, затем в общий L3, и только если нет нигде — в RAM (*cache miss*, тем дороже, чем дальше пришлось идти). Два нюанса:

- Подтягивается целая **cache line** (обычно 64 байта), не одна переменная — поэтому линейный проход по массиву быстрее случайного доступа к тем же данным.
- Первое обращение почти всегда идёт до RAM (*cold miss*), и только потом данные оседают в L3 → L2 → L1. Запись — по той же логике в обратную сторону: сначала в L1 пишущего ядра, потом «стекает» вниз.

Здесь и расходится с бытовым «рассинхронизация кэшей → data race»: согласованность кэшей между ядрами процессоры держат аппаратно, протоколами когерентности (MESI) — все ядра гарантированно увидят актуальное значение, просто протокол не гарантирует, *когда именно* это случится относительно остальных операций программы. Разница между «когда-нибудь увидят» и «увидят к нужному моменту» — и есть щель, где рождаются баги.

(Важная оговорка: «когда-нибудь увидят» — это про аппаратную часть, кэши. Но без `volatile / synchronized` есть и более жёсткий случай той же проблемы — JIT формально вправе вообще перестать перечитывать переменную из памяти, если внутри текущего потока её никто не меняет, и тогда правильный ответ не «увидят позже», а «может не увидеть никогда». Это не про кэши и не про MESI — это решение компилятора. Подробный разбор на примере `busy-loop` — в главе 5.)

Настоящая причина гонок в Java — в другом:

1. **Неатомарность составных операций.** `value++` — это три шага (прочитать → прибавить → записать), а не одна инструкция. Два потока могут перемежать эти шаги и прочитать одно и то же старое значение до того, как кто-то из них его перезапишет.
2. **Отсутствие гарантии видимости.** Без `volatile / synchronized` JIT вправе держать значение в регистре или переставлять инструкции (*reordering*) — запись одного потока может не стать видимой другому произвольно долго. Причина не в кэшах, а в отсутствии контракта, обязывающего JVM сделать запись видимой именно сейчас.

Оба эффекта формализует **Java Memory Model** через *happens-before* — она определяет, при каких условиях запись одного потока гарантированно видна другому (тема следующих уроков).

Гонка на практике

```
class Counter {
    private int value = 0;

    public void increment() {
        value++; // read -> add -> write, не атомарно
    }
}
```

```

    public int get() {
        return value;
    }
}

public class RaceDemo {
    public static void main(String[] args) throws InterruptedException {
        Counter counter = new Counter();
        Runnable task = () -> {
            for (int i = 0; i < 100_000; i++) {
                counter.increment();
            }
        };

        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);
        t1.start();
        t2.start();
        t1.join();
        t2.join();

        System.out.println(counter.get()); // ожидаем 200000, получаем
        меньше — и по-разному от запуска к запуску
    }
}

```

Результат почти никогда не будет ровно 200000 и меняется от запуска к запуску — часть инкрементов теряется из-за неатомарного `value++`, а не из-за «поломки» кэша.

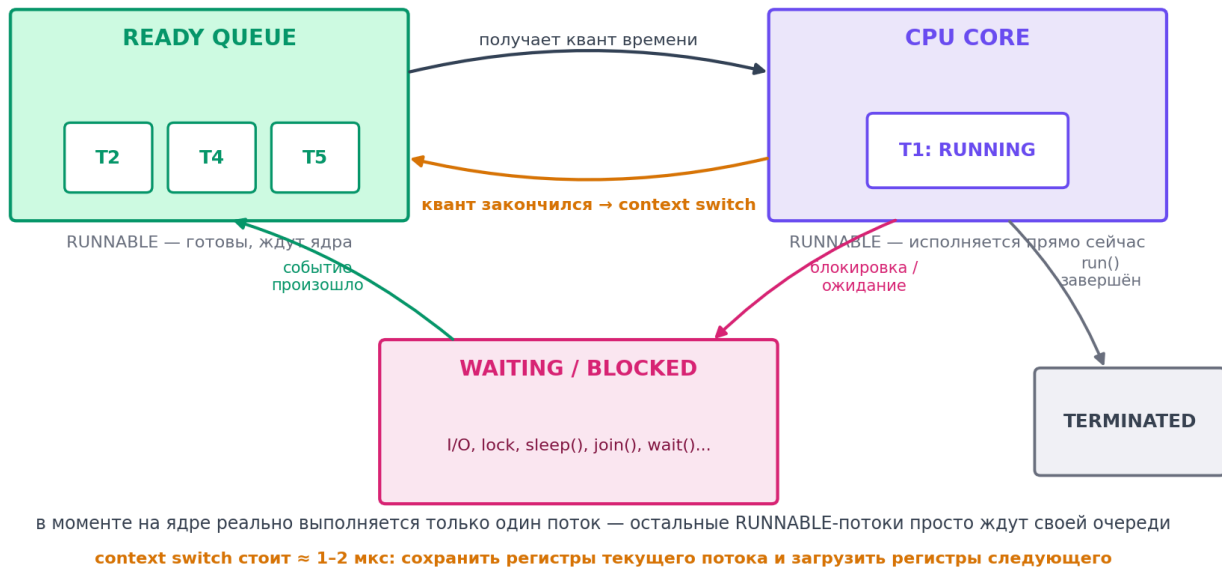
Итого

Поток дешевле процесса, потому что делит адресное пространство и заводит только свой стек. На стеке — только примитивы и ссылки, поэтому гонки живут в куче, а не на стеке. Параллелизм даёт лишь несколько ядер и ограничен законом Амдала; гонки — следствие неатомарных операций и отсутствия happens-before, а не «плохих» кэшей (их когерентность держит MESI).

Глава 1.3. Планировщик: очереди, переключение контекста и почему поток — просьба, а не команда

RUNNABLE — это очередь, а не гарантия исполнения

Планировщик ОС: очереди и переключение контекста



`Thread.State.RUNNABLE` в терминах JVM означает «поток либо выполняется, либо ждёт очереди на ядро» — JVM сознательно не различает эти два случая, это работа планировщика ОС. У него по сути две очереди: **Ready** (готовы, ждут ядра) и **CPU Core** (реально выполняется прямо сейчас). Поток может простоять `RUNNABLE` и не оказаться реально `RUNNING`, если ядер не хватает более приоритетным соседям.

Когда поток блокируется (`I/O`, монитор, `sleep()` / `join()` / `wait()`) — планировщик переводит его в отдельную очередь **Waiting/Blocked** и отдаёт время следующему кандидату из `Ready`. Когда ожидаемое событие случилось — поток возвращается в `Ready`, а не сразу на ядро: снова просто кандидат.

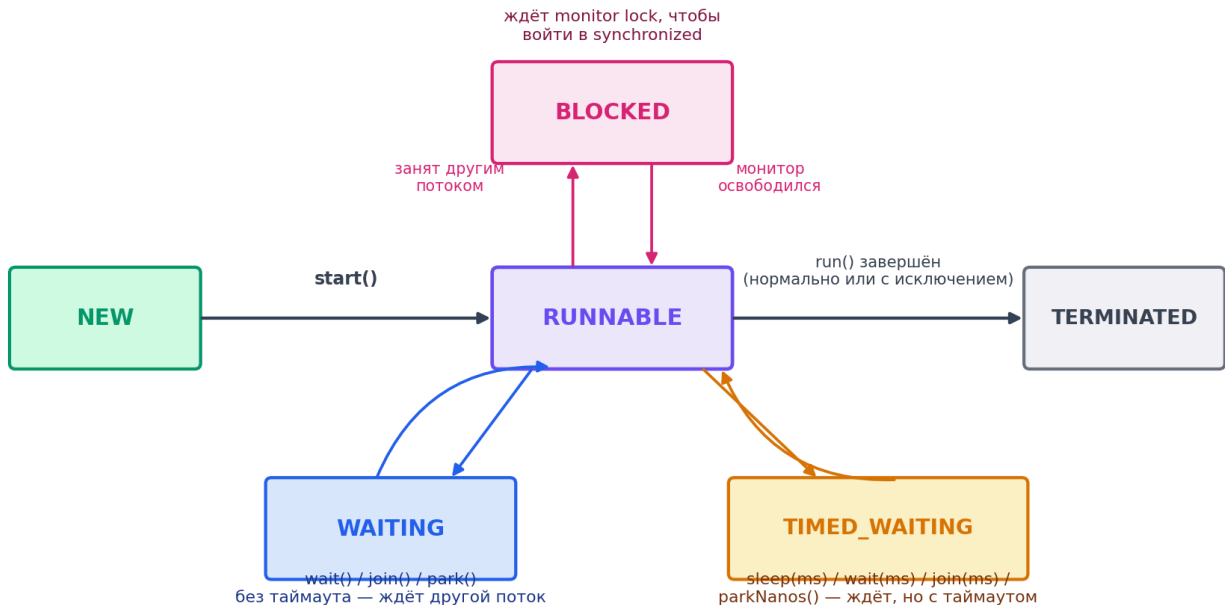
Переключение контекста: конкретные микросекунды

Каждый раз, когда планировщик снимает поток с ядра и ставит другой, происходит **context switch**: сохранить регистры (и `program counter`) снятого потока, загрузить состояние следующего. Дополнительная цена: данные предыдущего потока в `L1/L2` (глава 2) больше не актуальны для нового — начинаются лишние `cache miss`'ы, пока кэш «разогревается» заново.

Замеры на Linux дают **~1.2–1.5 мкс на переключение** с пиновкой к ядру, без пиновки — **~2.2 мкс**. Для сравнения: `L1` стоит ~1 нс — один `context switch` эквивалентен **1000–2000 обращениям к `L1`**. Поэтому если поток просыпается на пару быстрых операций и снова блокируется, накладные расходы на само переключение легко перевешивают полезную работу.

Полный жизненный цикл: Thread.State

Жизненный цикл потока: Thread.State



Шесть значений с конкретным, задокументированным триггером — не просто стрелочки на схеме:

- **NEW** — объект `Thread` создан, `start()` ещё не вызывался.
- **RUNNABLE** — выполняется в JVM (на ядре или в очереди на него — JVM не различает).
- **BLOCKED** — ждёт **monitor lock**: вход в `synchronized`, занятый другим потоком (или повторный вход после `Object.wait()`).
- **WAITING** — `Object.wait()`, `Thread.join()` или `LockSupport.park()` **без таймаута**: ждёт явного пробуждения (`notify()` / `notifyAll()` / `unpark()`) или завершения.
- **TIMED_WAITING** — то же самое, но с таймаутом: `sleep(ms)`, `wait(ms)`, `join(ms)`, `parkNanos()` / `parkUntil()` — проснётся сам.
- **TERMINATED** — `run()` закончился (нормально или с исключением); назад пути нет.

BLOCKED — это конкретно борьба за `monitor lock`; **WAITING/TIMED_WAITING** — добровольный отказ от CPU в ожидании условия, к монитору может быть вообще не привязан (`join()`, например).

Приоритет и `yield()` — подсказки, не приказы

`Thread.MIN_PRIORITY` (1), `NORM_PRIORITY` (5, дефолт), `MAX_PRIORITY` (10) задают шкалу; `setPriority()` может только *понижить* относительно максимума группы потоков, новый

поток наследует приоритет создающего. Но JDK не гарантирует, что более высокий приоритет означает более быстрое или частое исполнение — куда именно приоритет ляжет на шкалу конкретной ОС, решает уже она. (У виртуальных потоков приоритет всегда `NORM_PRIORITY`, `setPriority()` — no-op.)

`Thread.yield()` честнее прямым текстом: *«a hint to the scheduler that the current thread is willing to yield its current use of a processor. The scheduler is free to ignore this hint»*. Отсюда порядок вывода в демках — перемешан и не воспроизводим между запусками с `yield()` или без него: ни приоритет, ни `yield()` не заменяют синхронизацию, если порядок важен.

Антипаттерн «1000 потоков» — в цифрах

1000 потоков на 8 ядрах:

- **Память:** Linux резервирует под поток до 8 МиБ виртуальной памяти по умолчанию — 1000 потоков это уже гигабайты виртуального адресного пространства, даже если реально используется малая часть.
- **Переключения:** одновременно исполняется только 8 потоков, остальные 992 стоят в Ready. Чем больше RUNNABLE-потоков толпится вокруг горстки ядер, тем чаще их приходится менять местами — а каждая замена это ~1–2 мкс (плюс остывший кэш) чистых накладных расходов без пользы.

Поэтому в проде почти никогда не создают поток вручную под каждую задачу — держат пул размером порядка числа ядер (для CPU-bound работы) и переиспользуют через `ExecutorService`.

Итого

RUNNABLE — это «в очереди или на ядре», реально исполняется ровно столько потоков, сколько ядер. Context switch стоит конкретных микросекунд — на три порядка дороже L1 — и не бесплатен. Приоритет и `yield()` — подсказки, которые планировщик вправе игнорировать. «Больше потоков» упирается в память и цену переключений — отсюда пулы потоков вместо потока на задачу.

Глава 1.4. `join()`, `interrupt()` и совместная отмена

`join()` — тоже блокировка

`join()` ставит вызывающий поток в `WAITING / TIMED_WAITING` (глава 3) до завершения целевого потока — тот же механизм, что `sleep()` / `wait()`, просто другое условие

пробуждения. Правило: звать `join()` только там, где результат нужен прямо сейчас, и держать ожидание коротким. Если нужно дождаться десяти потоков — сначала запустить все десять, потом пройтись `join()` по каждому, а не стартовать и `join`'иться по одному: во втором случае потоки теряют параллельность, каждый следующий стартует только после того, как предыдущий уже полностью отработал.

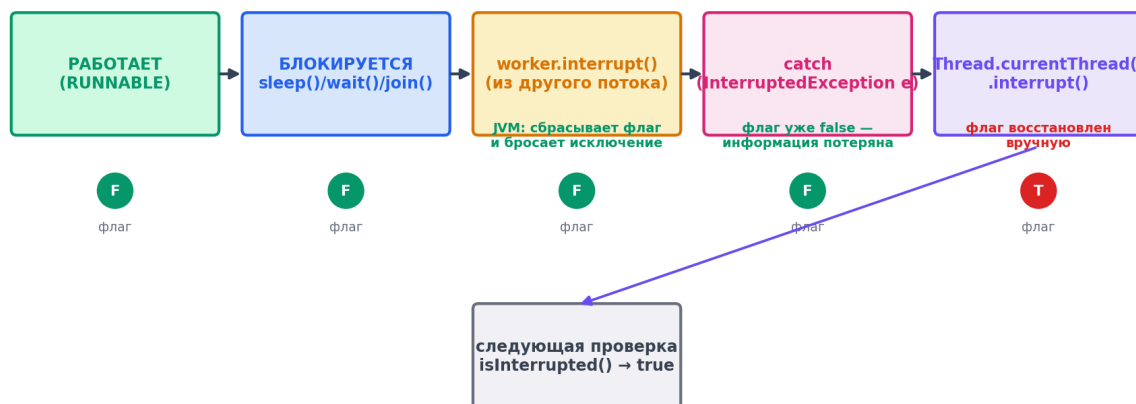
Java не может «убить» поток — только попросить

В Java нет безопасного способа принудительно остановить чужой поток (`Thread.stop()` формально существует, но небезопасен и не используется — мог оборвать поток в середине изменения общего состояния). Вместо этого — только **кооперативный** механизм: `interrupt()` не команда «остановись», а просьба «посмотри на себя и реши сам». Поток может её полностью проигнорировать — это не баг, а дизайн: только сам поток знает, когда безопасно остановиться, не оставив данные в несогласованном состоянии.

Отсюда природа `InterruptedException` — не «что-то сломалось», а доставленный сигнал отмены. Бросают её блокирующие методы (`sleep()`, `wait()`, `join()`) в момент, когда поток получил запрос на прерывание пока был заблокирован — JVM будит его раньше срока, чтобы дать шанс среагировать.

Флаг прерывания: что и когда снимает JVM

Флаг прерывания: что видно в каждый момент



без восстановления флага вызывающий код никогда не узнает, что поток вообще прерывали

У каждого потока — внутренний boolean, *interrupt status*. `interrupt()` обычно просто ставит его в `true`. Но если поток уже заблокирован в `wait()` / `join()` / `sleep()`, по документации JDK: **«its interrupt status will be cleared and it will receive an `InterruptedException`»** — JVM сама сбрасывает флаг в `false` и бросает исключение. К

моменту `catch (InterruptedException e)` флаг уже снова `false`, как будто прерывания и не было.

Отсюда практика — `Thread.currentThread().interrupt()` внутри `catch`: не защитная приписка, а восстановление информации, которую JVM только что стёрла. Без этого факт прерывания теряется безвозвратно — ни этот метод, ни код выше по стеку, проверяющий `isInterrupted()`, не узнают, что поток вообще пытались остановить.

Два метода, которые путают чаще всего:

- `thread.isInterrupted()` — читает флаг, ничего не трогает;
- `Thread.interrupted()` (статический) — читает флаг и сразу сбрасывает в `false`.

```
Thread worker = new Thread(() -> {
    while (!Thread.currentThread().isInterrupted()) {
        try {
            doChunkOfWork();
            Thread.sleep(50);
        } catch (InterruptedException e) {
            // вернуть флагу значение, которое JVM только что стёрла
            Thread.currentThread().interrupt();
            // и реально закончить работу, а не "поймал и живём дальше"
            break;
        }
    }
    Logger.log("worker: graceful stop");
});
```

Антипаттерн: поймать `InterruptedException`, ничего не сделать (ни `break`, ни восстановления флага) и продолжить как ни в чём не бывало — компилируется и «работает», но поток получил явную просьбу остановиться и сознательно её проигнорировал, а весь остальной код об этом никогда не узнает.

Busy loop — та же проблема, вид сбоку

`while (true) { /* ничего не ждём */ }` — зеркальная ошибка. Поток остаётся `RUNNABLE` и удерживает ядро постоянно, не давая планировщику (глава 3) отдать это время другим потокам в очереди. Блокирующее ожидание (`sleep()` / `wait()`) — это буквально то, что переводит поток из `Ready` в `Waiting` и освобождает ядро; busy loop эту возможность у планировщика отбирает без причины.

I/O-bound vs CPU-bound

- **CPU-bound** — время определяют вычисления (хеши, парсинг, сортировка). Поток всё это время реально занимает ядро (`RUNNING`), и потоков сверх числа ядер — только лишний `context switch` без пользы (глава 3).
- **I/O-bound** — время определяет ожидание внешнего устройства (диск, сеть, БД). Пока идёт обмен данными, CPU почти не при делах — поток сидит в `WAITING / TIMED_WAITING` , а реальная работа происходит в контроллере диска или сетевой карте.

Отсюда разные правила размера пула: для CPU-bound больше потоков, чем ядер, почти всегда вредно; для I/O-bound потоков может быть в разы больше числа ядер — большинство из них просто ждут диск или сеть, а не борются за CPU.

Итого

`join()` — такая же блокировка, как `sleep()` / `wait()` , и её стоит минимизировать.
`interrupt()` — кооперативная просьба, а не `kill` ; JVM сама снимает флаг прерывания при выбросе `InterruptedException` , поэтому восстанавливать его в `catch` нужно вручную — иначе сигнал теряется для всего кода выше по стеку. `Busy loop` отбирает ядро у планировщика без необходимости. CPU-bound работа ограничена числом ядер, I/O-bound спокойно переживает намного больше потоков, чем ядер.

Глава 1.5. `synchronized` и `volatile` : взаимное исключение и видимость

Синхронизация — это про взаимное исключение, а не про порядок

В главе 2 гонка возникала из-за двух вещей: `value++` — это три шага, а не одна инструкция, и JIT ничего не обязан делать запись видимой другому потоку сразу. Оба инструмента этой главы закрывают ровно эти две проблемы — но по отдельности, не одним и тем же способом: `synchronized` даёт **mutual exclusion** (взаимное исключение) — гарантирует, что «прочитать → прибавить → записать» выполнится как одна неразрывная операция, конкурентов в этот момент рядом просто не будет. `volatile` ничего не исключает — он только гарантирует **видимость**: если один поток записал значение, следующее чтение другого потока увидит именно его, а не устаревшую копию.

Monitor lock: как `synchronized` устроен изнутри

Monitor lock: как `synchronized` даёт `mutual exclusion`

в один момент времени внутри `synchronized`-блока на этом объекте — ровно один поток



У каждого объекта в Java есть встроенный **monitor** (intrinsic lock) — он есть всегда, даже если ты никогда не писал `synchronized`. Вход в блок `synchronized (obj) { ... }` — это попытка захватить `monitor` именно объекта `obj`; выход из блока (обычным путём или через исключение — не важно, JVM снимает `lock` в любом случае) — его освобождение. `public synchronized void increment()` для нестатического метода — то же самое, что `synchronized (this) { ... }` на всё тело метода, просто короче.

Если `monitor` уже занят другим потоком, новый претендент не крутится и не пытается повторно — он переходит в состояние `BLOCKED` (глава 3) и ждёт, пока текущий владелец не выйдет из блока. Как только это происходит, ровно один из ждущих потоков получает `lock` и становится `RUNNABLE`. `Lock` **реентерабелен**: поток, уже владеющий `monitor`'ом, может свободно войти в другой `synchronized`-блок на том же объекте (например, вызвать из синхронизированного метода другой синхронизированный метод того же объекта) — не заблокирует сам себя.

Помимо взаимного исключения, `synchronized` попутно даёт и видимость: JLS гарантирует `happens-before` между освобождением `monitor`'а одним потоком и его захватом следующим — то есть всё, что поток записал внутри блока, гарантированно увидит следующий поток, вошедший в блок на том же объекте. Это то, чего `volatile` сам по себе не делает для составных операций.

RaceDemo с `synchronized` — что на самом деле изменилось

Если обернуть `increment()` из демки в главе 2 в `synchronized (this) { value++; }` (или просто пометить метод как `synchronized`), результат `RaceDemo` станет стабильно

200000 при любом количестве запусков — гонка исчезла, потому что теперь `read → add → write` внутри lock'a действительно неразрывны.

Но стоит сразу заметить, чего это стоило: раз `increment()` синхронизирован на одном и том же объекте `counter`, оба потока больше не могут выполнять его одновременно в принципе — они буквально встают в очередь на один и тот же `monitor` и по факту работают по очереди, а не параллельно. Для этого конкретного примера (100 000 инкрементов на поток) это означает, что суммарное время у двух потоков будет не быстрее, а часто и медленнее, чем если бы всю работу с самого начала сделал один поток — на серилизацию накладывается ещё и цена самого захвата/освобождения lock'a при конкуренции. Это нормально и ожидаемо: цель демки — показать корректность, а не ускорение. У неё просто неудачная форма для демонстрации пользы многопоточности как таковой: `int`-счётчик, который целиком помещается под один lock, — это ровно тот случай, где `synchronized` съедает всю выгоду от количества потоков. На практике для такой задачи (простой атомарный счётчик) есть инструменты дешевле lock'a — `AtomicInteger / LongAdder` из `java.util.concurrent.atomic`, — которые дают ту же корректность через аппаратные CAS-инструкции, без блокировки потока целиком. Это уже тема отдельного разговора, но держать в голове стоит: `synchronized` — универсальный, но не самый дешёвый инструмент, и его цена (глава 3: захват lock'a конкурентным потоком — это ещё и потенциальный `BLOCKED → RUNNABLE` переход, то есть `context switch`) стоит платить только там, где реально нужна атомарность составной операции.

volatile : гарантия видимости, не атомарности

```
private volatile boolean running = true;
```

`volatile`-поле работает иначе, чем `monitor`: он не исключает конкурентный доступ, а меняет правила чтения и записи именно этой переменной — запись **сразу** становится видна всем последующим чтениям из любого потока, без задержки на «когда-нибудь JIT решит её сбросить». Это ровно то, чего не хватало полю `running` в `busy-loop` потоке из главы 4: без `volatile` компилятор формально вправе один раз прочитать `running` в регистр и больше никогда не перечитывать его из памяти, раз внутри цикла его никто явно не меняет с точки зрения этого потока — тогда `while (running)` не заметит внешнего `running = false` вообще никогда (не потому что «не успеет», а потому что теоретически может не посмотреть снова).

Важно, чего `volatile` **не** делает: он не даёт атомарности составным операциям.

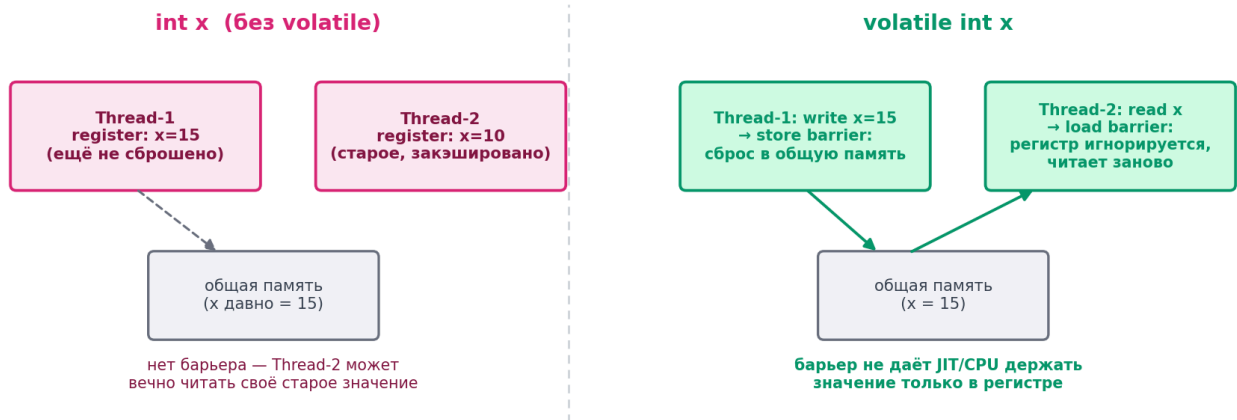
`volatile int counter; counter++;` — всё ещё три шага (прочитать → прибавить → записать), и гонка между двумя потоками на `counter++` никуда не денется, даже если поле `volatile`. Отсюда и таблица сравнения:

Критерий	synchronized	volatile
Защищает от гонок в составных операциях	✓	✗
Гарантирует видимость записи	✓	✓
Цена	высокая (lock, возможен BLOCKED)	низкая (барьер, без блокировки потока)

Отсюда практическое правило: `volatile` — для одиночного флага или ссылки, которую один поток пишет, а другие только читают (`running` , ссылка на актуальную конфигурацию и т.п.). Как только над значением нужно сделать больше одной операции атомарно — инкремент, «прочитать и обновить», — `volatile` уже недостаточен, нужен `synchronized` или атомарный класс.

Механика: барьер памяти, а не «релоад кэша»

volatile: барьер памяти, а не отдельный канал в обход кэша



механика опирается на тот же протокол когерентности кэшей (MESI, глава 2) — `volatile` не создаёт новую память, а ставит барьер вокруг него

По слайдам курса видимость без `volatile` объясняется как «несогласованность кэшей» — и это та же неточность, что уже разбиралась в главе 2: кэши между ядрами и так согласованы аппаратно, протоколом MESI, рано или поздно любое ядро увидит актуальное значение. Проблема без `volatile` в другом — в отсутствии гарантии, когда именно это произойдёт, и в праве JIT держать значение только в регистре, вообще не заглядывая в память повторно.

`volatile` решает это не заменой кэша на что-то другое, а через **барьеры памяти** (memory barriers): запись во `volatile`-поле сопровождается *store barrier* — он не даёт

значению застрять только в регистре или store buffer ядра, а принудительно проталкивает его туда, откуда его увидит протокол когерентности. Чтение `volatile`-поля сопровождается *load barrier* — он запрещает использовать значение, которое, возможно, уже лежит в регистре с прошлого раза, и заставляет прочесть заново. Барьеры также запрещают JIT переставлять эту запись/чтение местами с соседними инструкциями (*reordering* из главы 2). Иными словами: `volatile` не строит отдельный канал памяти в обход кэшей — он просто не даёт компилятору и процессору срезать угол на той самой инфраструктуре когерентности, которая и так есть в железе.

Тонкость: `volatile` защищает ссылку, не содержимое объекта

Один из самых частых источников путаницы: `volatile` на поле-ссылке гарантирует видимость **самой ссылки** — того, на какой объект она указывает, — а не полей объекта, на который она указывает.

```
class Config {
    int retries;
    String host;
}

private volatile Config config = new Config();

// Видимость гарантирована: другие потоки сразу увидят,
// что config теперь указывает на новый объект.
config = new Config();

// НЕ гарантировано: это запись в поле уже существующего объекта,
// а не переприсваивание самой volatile-переменной.
config.retries = 3;
```

Практический вывод отсюда — паттерн **safe publication**: если нужно безопасно «опубликовать» составной объект между потоками через `volatile`-ссылку, объект должен быть либо неизменяемым (создан целиком, со всеми полями выставленными, и только потом присвоен `volatile`-переменной), либо изменения его полей должны защищаться отдельно (`synchronized`, атомарные поля и т.п.). Просто пометить ссылку как `volatile` и продолжать мутировать объект через неё из нескольких потоков — гонка никуда не делась, просто переехала с самой ссылки на поля объекта.

Итого

`synchronized` даёт mutual exclusion через встроенный monitor каждого объекта (JLS гарантирует и happens-before при передаче lock'a) и решает гонки в составных операциях — ценой того, что конкурирующие потоки не работают параллельно, а ждут своей

очереди (BLOCKED , глава 3). `volatile` дешевле, но даёт только видимость записи через барьеры памяти — не атомарность, и для составных операций (`x++`) бесполезен. `Volatile`-ссылка защищает саму ссылку, а не поля объекта, на который она указывает. Оба механизма — не два конкурента, а инструменты для разных задач: `mutex` для составных изменений общего состояния, `volatile` для одиночного флага или безопасной публикации неизменяемого объекта.

2. Инструменты и ловушки синхронизации: от сигналов до своей очереди

Глава 2.1. Java Memory Model: happens-before

Формальная основа гарантий `synchronized` / `volatile` из главы 5: JLS (Java Language Specification) §17.4, отношение **happens-before**.

Зачем нужна JMM

Три независимых источника переупорядочивания — каждый корректен сам по себе, язык не может его запретить:

Источник	Что делает
компилятор / JIT	переставляет/выкидывает инструкции внутри потока, если результат самого потока не меняется (глава 2 — JIT вправе вообще перестать перечитывать переменную)
CPU	out-of-order execution, store-буферы — запись не сразу видна «наружу»
кэши	согласуются протоколом вроде MESI (глава 2) — но согласованность кэшей ≠ порядок операций

JMM (JLS, Java Language Specification, §17.4) — контракт поверх всего этого: при каких условиях эффект операции одного потока гарантированно виден другому и в каком порядке. Вне контракта — не «повезёт/не повезёт», а честно неопределённое поведение (data race).

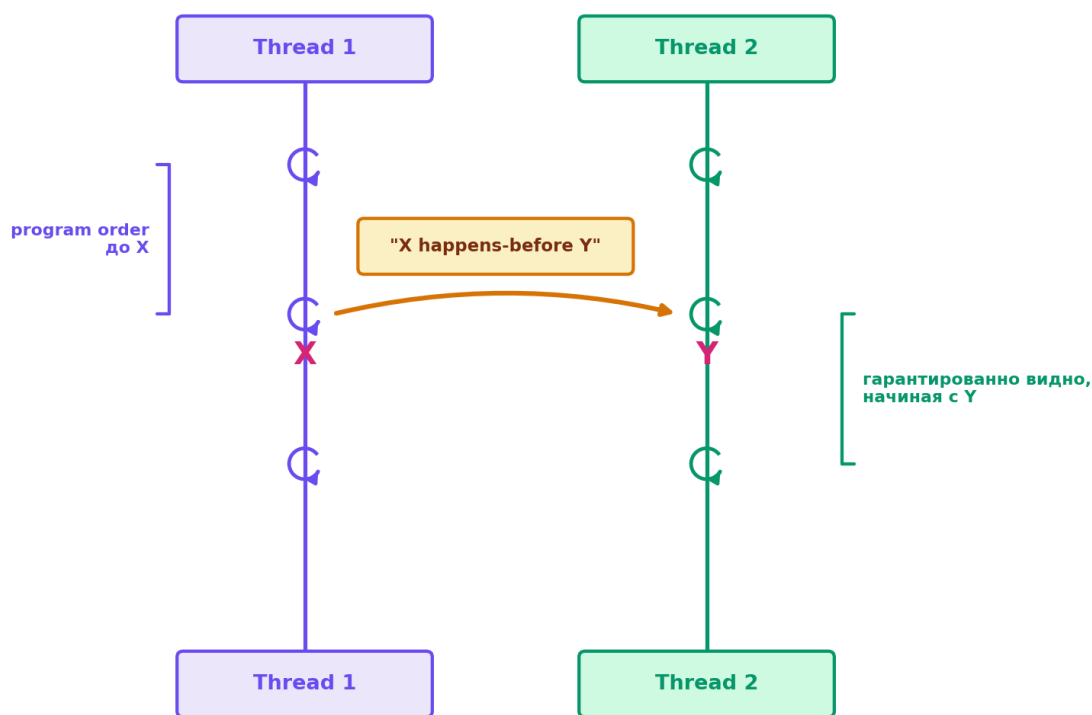
Семантика happens-before

A happens-before B означает: результат A гарантированно виден B, и JVM обязана вести себя так, будто A случилось раньше B — реальный порядок инструкций при этом может быть любым, это лишь наблюдаемый контракт.

Логически *happens-before* — это **гарантия видимости результатов работы операции А для операции Б**. Технически это достигается не «записью в ОЗУ», а **запретом на перестановку инструкций** компилятором (где предшествующие операции нельзя сдвигать ниже границы, а последующие — поднимать выше) + **принудительным опустошением внутренних буферов процессора** в кэш с помощью барьеров памяти.

- **не про физическое время** — про гарантию видимости и порядка, а не про то, что раньше случилось по часам;
- **партиальный порядок** — рёбра есть не между любыми двумя действиями; если между действиями двух потоков нет ребра (напрямую или по цепочке), JMM не даёт вообще никаких гарантий — это и есть определение *data race*;
- **транзитивность** — если $A \text{ hb } B$ и $B \text{ hb } C$, то $A \text{ hb } C$, даже когда А и С в разных потоках и напрямую никак не связаны.

Семантика happens-before



транзитивность: всё, что было ДО X в потоке 1, становится видно всему, что идёт ПОСЛЕ Y в потоке 2

Основные рёбра happens-before

JMM задаёт несколько базовых правил, из которых строятся практически все гарантии видимости между потоками.

Список этих правил конечный. Если нужного ребра нет и его нельзя получить транзитивно через другие ребра, JMM **не гарантирует** видимость одного действия другому — сколько бы код ни «работал на глаз».

Ребро	Условие
program order	действие A в одном потоке happens-before последующему действию B этого же потока
monitor	unlock() монитора happens-before последующему lock() того же монитора
volatile	запись в volatile-поле happens-before последующему чтению того же поля
thread start	вызов Thread.start() happens-before первым действиям запущенного потока
thread termination	последнее действие потока happens-before успешному возврату из t.join() , дождавшегося этого потока

Главная идея: **не нужно запоминать отдельные магические свойства** volatile , synchronized и join() . Нужно уметь построить цепочку happens-before между действиями

Program order

Первое и самое простое правило: действия внутри одного потока упорядочены относительно друг друга.

```
int data = 0;

data = 42;           // A
System.out.println(data); // B
```

Здесь:

A hb B

Поток не может наблюдать собственные действия так, будто B произошло раньше A .

Но это правило действует **только внутри одного потока**. Сам по себе program order не создаёт связи с другим потоком.

Если между действиями разных потоков нет happens-before-связи, нельзя переносить program order одного потока на другой поток. Сам факт, что в потоке А сначала была запись, а потом какая-то другая операция, ещё не означает, что поток В обязан увидеть эти действия в таком порядке.

Например: если в потоке А выполняются три операции — `a = 1`, затем `b = a + 1` (зависит от `a`), и следом `c = 3` (независимая переменная), то JMM гарантирует правильный порядок вычислений только *внутри* потока А. Но для внешнего потока В, из-за отсутствия барьеров памяти, независимая запись `c = 3` вполне может выполняться и стать видимой **раньше**, чем запишутся `a` и `b`. Без happens-before чужие потоки видят независимые операции как бы «параллельно» и в случайном порядке.

Это происходит потому, что процессор видит внутренние зависимости Потока А и ради оптимизации безопасно переставляет `c` вперёд, не ломая локальный результат. Поток В же не знает контекста чужого кода и считывает физическую память напрямую, фиксируя эту перестановку.

Thread start

Вызов `Thread.start()` happens-before первому действию запущенного потока.

```
int[] box = {0};

box[0] = 42; // запись до start()

Thread child = new Thread(() -> {
    System.out.println(box[0]); // гарантированно 42
});

child.start();
```

Вызов `Thread.start()` служит полноценным барьером памяти, который принудительно сбрасывает все накопленные родительским потоком данные в общую память. Дочерний поток при запуске обязан обновить свой кэш, поэтому благодаря транзитивности *happens-before* он гарантированно видит `42` без всякого `volatile`.

Monitor: synchronized

Выход из `synchronized`-блока создаёт happens-before с последующим входом в `synchronized`-блок на **том же мониторе**.

```

static final Object LOCK = new Object();
static int shared = 0;

Thread writer = new Thread(() -> {
    synchronized (LOCK) { // lock(LOCK)
        shared = 9000;
    } // unlock(LOCK)
});

Thread reader = new Thread(() -> {
    synchronized (LOCK) { // lock(LOCK)
        System.out.println(shared); // гарантированно 9000
    } // unlock(LOCK)
});

writer.start();
// Небольшая пауза, чтобы writer гарантированно захватил LOCK первым.
Thread.sleep(100);
reader.start();

```

Поэтому `shared` может быть обычным `int`: гарантию видимости создаёт не сам тип поля, а happens-before через монитор.

Важно: речь именно о **том же мониторе**. `unlock(LOCK)` не создаёт happens-before с `lock(LOCK2)`.

Volatile

Запись в `volatile`-поле happens-before последующему чтению **того же volatile-поля**.

```

static int data = 0;
static volatile boolean ready = false;

Thread writer = new Thread(() -> {
    data = 777; // обычная запись
    ready = true; // volatile write
});

Thread reader = new Thread(() -> {
    while (!ready) { // volatile read
        // ждём
    }
}

```

```
        System.out.println(data); // гарантированно 777
    });

    writer.start();
    reader.start();
```

На первый взгляд может показаться, что `volatile` защищает только `ready`. Но здесь работает транзитивность happens-before.

`volatile`-флаг готовности может опубликовать результаты предыдущих обычных записей потока.

То есть `volatile` здесь используется не столько как «защита флага», сколько как **точка публикации** состояния, подготовленного до `volatile`-записи.

Thread termination и join()

Последнее действие завершившегося потока happens-before успешному возврату из `join()`, который дождался завершения этого потока.

```
int[] box = {0};

Thread worker = new Thread(() -> {
    box[0] = 42;
});

worker.start();
worker.join();

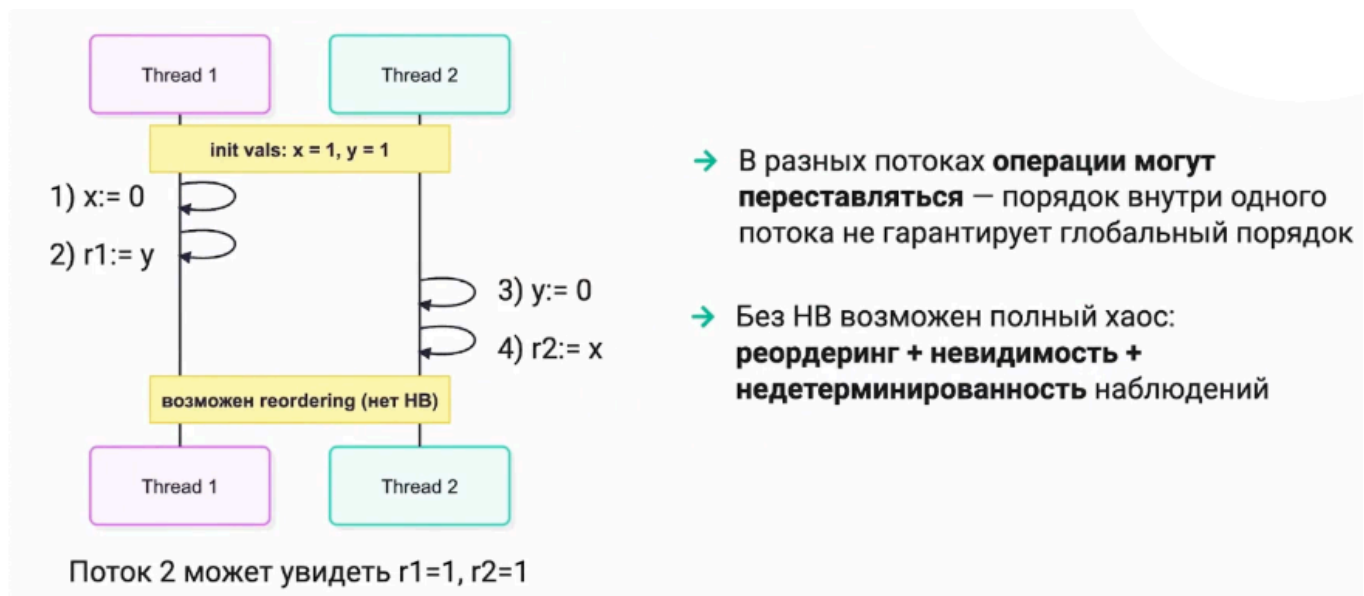
System.out.println(box[0]); // гарантированно 42
```

Поэтому `box[0]` здесь также не обязан быть `volatile`.

Это объясняет, почему `join()` — не просто механизм ожидания завершения потока. Он одновременно создаёт **гарантию видимости результатов работы завершившегося потока**.

Reordering операций: где и почему

Без happens-before «очевидный» порядок операций не гарантирован — ни компилятором, ни процессором:



Между потоками нет ни одного ребра $\Rightarrow$ JVM вправе переставить операции, независимые по данным внутри каждого потока ($x=0$ / $r1=y$ не зависят друг от друга, так же как $y=0$ / $r2=x$). Результат недостижим ни при каком последовательном чередовании этих 4 строк — но реально наблюдается на компиляторах и процессорах, которые честно следуют JMM. В коде это выглядит так:

Publication: ссылка видна $\neq$ объект виден

Тот же принцип reordering'a бьёт по публикации объектов: увидеть новую ссылку — не значит увидеть корректно инициализированные поля объекта, на который она указывает.

без volatile	c volatile
ребро конструктор $\rightarrow$ публикация ссылки	нет
ребро публикация $\rightarrow$ чтение ссылки	нет
что видит читатель	ссылку — да; поле — не гарантировано (broken publication)

Этап синхронизации	Без volatile (Broken Publication)	C volatile (Safe Publication)
1. Сборка объекта (Конструктор → Публикация ссылки)	✗ Нет связи. Процессор может сначала опубликовать ссылку на holder, а только потом заполнять внутренние поля.	Есть связь (program order). Инициализация полей a, b, c... = 1 гарантированно произойдет до публикации ссылки на holder.
2. Передача между потоками (Запись → Чтение ссылки)	✗ Нет связи. Адрес ссылки или состояние полей могут застрять во внутренних буферах процессора.	Есть связь (volatile rule). Запись ссылки holder в потоке Writer happens-before чтения этой ссылки в потоке Reader.
3. Итоговая цепочка (Транзитивность)	✗ Связи разорваны. Потоки видят независимые операции выделения памяти и записи полей в случайном порядке.	Цепочка замкнута. Поля = 1 $\xrightarrow{hb}$ Запись holder $\xrightarrow{hb}$ Чтение holder $\Rightarrow$ Поля видны Читателю.
Что видит Поток-Читатель	Только ссылку. Объект holder уже не null, но его поля некое количество a, b, c... считываются как дефолтные нули (0).	И ссылку, и корректные поля. Читатель гарантированно увидит единицы (1) во всех полях от a до h.

```

public class BrokenPublicationReorderingDemo {
    static Holder holder; // ⚠ Переменная НЕ volatile! Границы (шлагбаума) нет.
    static volatile boolean stop;
    static int anomalies;

    static class Holder {
        int a, b, c, d, e, f, g, h;
        Holder() {
            a = b = c = d = e = f = g = h = 1; // Запись в поля
        }
    }

    public static void main(String[] args) throws InterruptedException {
        // ... (код инициализации и запуска 8 потоков-писателей и 8 читателей)
    }
}

```

```

// ПОТОК-ПИСАТЕЛЬ делает:
holder = new Holder();
// ✨ Из-за отсутствия барьера памяти процессор публикует ссылку
// в общую память ДО того, как буферы с полями a,b,c... успели
опустошиться.

// ПОТОК-ЧИТАТЕЛЬ делает:
Holder h = holder;
if (h != null && (h.a != 1 || h.b != 1 ...)) {
    anomalies++; // 🔥 Срабатывает аномалия broken publication!
}
}
}

```

Это не противоречит главе 5 («volatile защищает ссылку, не объект») — там речь про мутацию полей **через уже опубликованную** ссылку другим потоком, здесь — про сам момент публикации. Разные фазы жизни объекта, разные гарантии одного и того же механизма.

Где volatile не поможет

- **составные операции** — `counter++` не становится атомарным сам по себе (нужен `AtomicInteger` / `synchronized`, не `volatile`);
- **несколько volatile-полей, обновляемых вместе** — у каждого своё ребро, «атомарность пары полей» не гарантирована;
- **мутация объекта через уже опубликованную ссылку другим потоком** — ограничение из главы 5, выше.

Итого

`happens-before` — язык описания того, что уже гарантируют `synchronized`, `volatile`, `start()` / `join()` и `program order`, а не отдельный механизм.

Глава 2.2. Межпоточная сигнализация

Монитор объекта

Монитор — внутренний механизм синхронизации в Java. Логически у него две очереди:

```

class ObjectMonitor {
    Thread owner;           // кто держит монитор
    Queue<Thread> entryList; // ждут входа в synchronized(obj)
}

```

```
Queue<Thread> waitList;    // ждут notify/notifyAll
}
```

- `wait()` → освобождает монитор → поток уходит в `waitList` (WAITING)
- `notify()` → один поток из `waitList` → в `entryList`
- `notifyAll()` → все потоки из `waitList` → в `entryList`

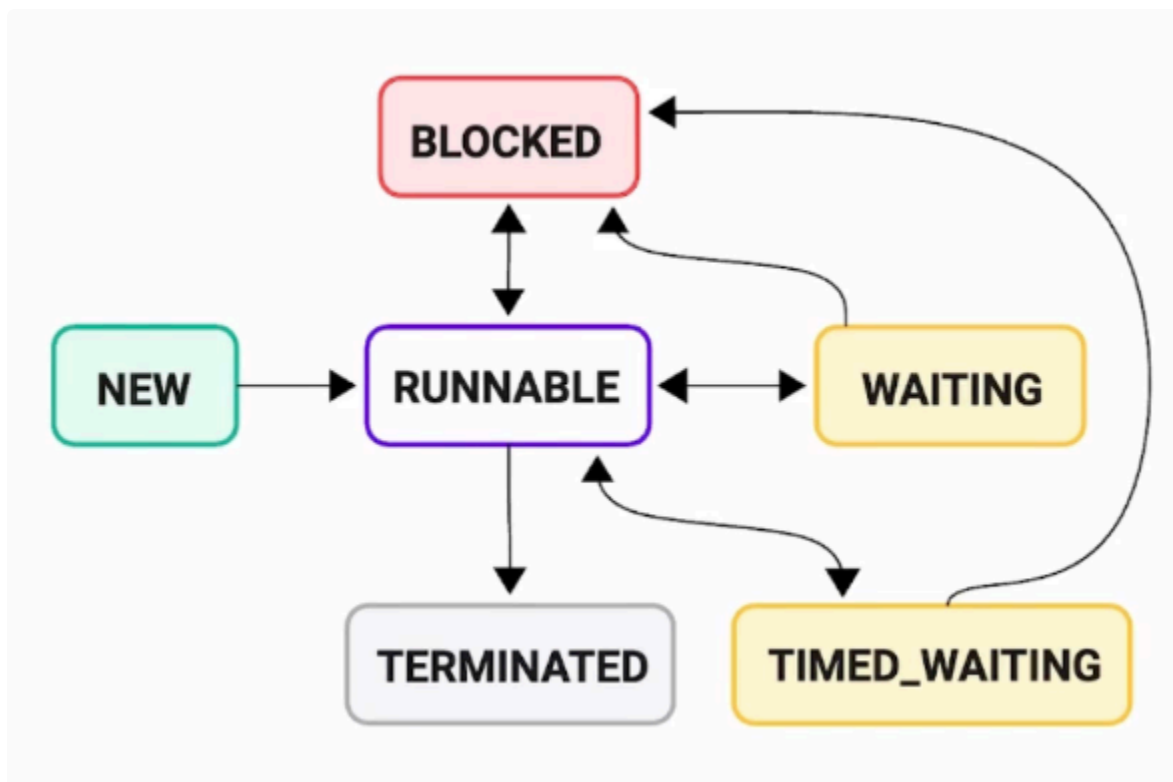
⚠ **Пробуждение ≠ немедленный вход.** Поток из `waitList` снова становится BLOCKED и должен повторно захватить монитор, прежде чем продолжить выполнение.

Методы `wait()` / `notify()` / `notifyAll()`

Метод	Что делает
<code>wait()</code>	Освобождает монитор и паркует поток до сигнала (WAITING / TIMED_WAITING с таймаутом)
<code>notify()</code>	Помечает один ожидающий поток к пробуждению
<code>notifyAll()</code>	Помечает все ожидающие потоки к пробуждению

Вызовы возможны только внутри `synchronized`-блока на том же объекте, иначе — `IllegalMonitorStateException`.

Состояния потока (Thread States)



- Поток, пытающийся войти в занятую критическую секцию → **BLOCKED**
- `wait()` → **WAITING** / **TIMED_WAITING** и **освобождение монитора**
- `notify()` / `notifyAll()` → ожидающие потоки переводятся обратно в **BLOCKED** (ждут повторного захвата)

Классический пример: кофемашина (producer/consumer)

```
public class CoffeeMachine {
    private boolean coffeeReady = false;

    public synchronized void makeCoffee() {
        while (coffeeReady) { // пока чашка не забрана — ждём
            try {
                wait();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
        }
        System.out.println("Машина варит кофе...");
        try { Thread.sleep(1000); }
        catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }
}
```

```

        coffeeReady = true;
        System.out.println("Кофе готов! Можно забирать.");
        notify(); // разбудим потребителя
    }

    public synchronized void takeCoffee() {
        while (!coffeeReady) { // ждём, пока кофе приготовится
            try {
                wait();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
        }
        System.out.println("Забрал кофе и пошёл пить!");
        coffeeReady = false;
        notify(); // разбудим производителя
    }
}

```

Почему именно `while`, а не `if`?

Если будет 2+ потребителя: производитель приготовил один кофе и послал `notify()`, но система разбудила «не того» потребителя. С `if` второй потребитель, пробудившись, забрал бы несуществующий кофе. Цикл заставляет **перепроверить предикат** после каждого пробуждения — это защита от ложных пробуждений (spurious wakeups) и «чужих» `notify`.

Универсальный шаблон (защита от ложных пробуждений)

```

synchronized (lock) {
    while (!condition) { // проверяем ПРЕДИКАТ в цикле, не if!
        lock.wait();
    }
    // ... работаем, меняем состояние
    lock.notifyAll(); // notifyAll безопаснее при множественных
    // ожидаателях
}

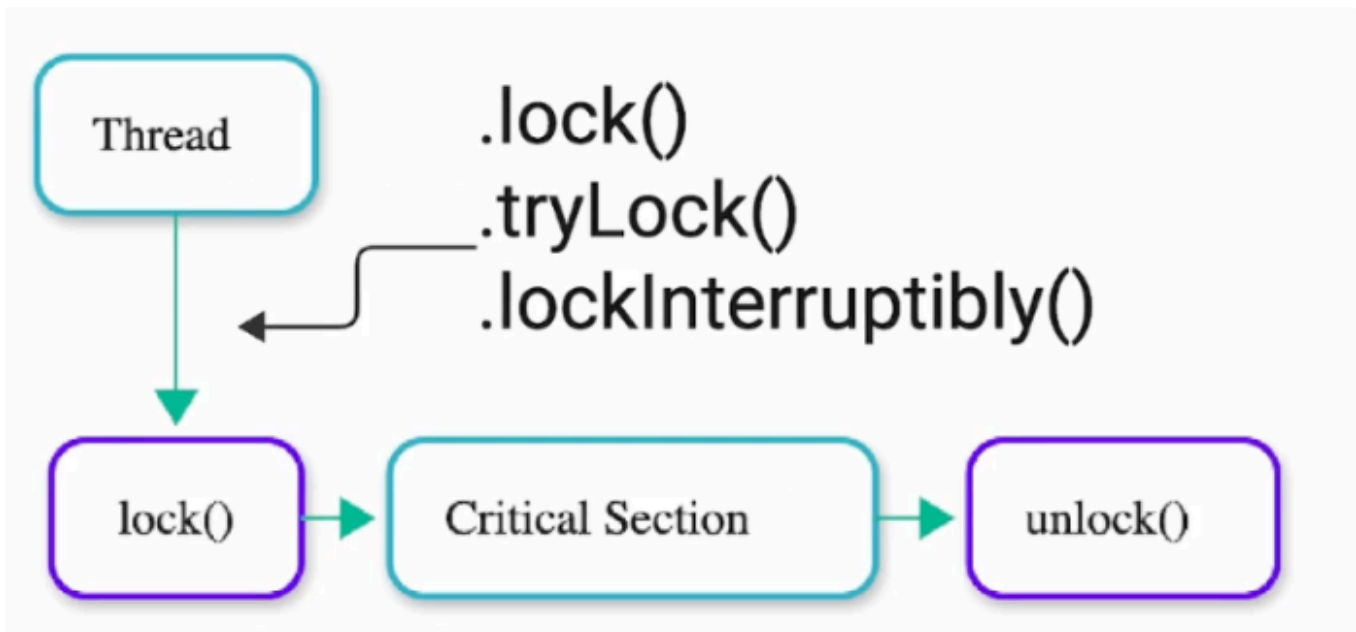
```

Золотые правила: `wait()` — только в `while`; ловим `InterruptedException` → восстанавливаем флаг (`Thread.currentThread().interrupt()`).

Типичные ошибки с `synchronized` + `wait/notify`

Ошибка	Чем опасна
Синхронизация «не на том объекте» (лок — изменяемая ссылка)	Если ссылку переприсвоили, потоки синхронизируются на разных объектах → синхронизация сломана полностью
Разные локи на чтение/запись одного ресурса	Писатель блокируется на объекте А, читатель — на В → не видят блокировок друг друга → Race Condition
Смешивание <code>this</code> и статических локов	Нестатический <code>synchronized</code> — лок на экземпляре (<code>this</code>), статический — на <code>Class.class</code> . Это разные мониторы, они не защищают общие данные
Долгая работа / I/O внутри монитора	Все остальные потоки копятся в <code>entryList</code> → приложение «замерзает»

ReentrantLock и Condition



Три способа захвата

Метод	Поведение
<code>lock()</code>	Обычный захват; поток засыпает, если занято
<code>tryLock(timeout, unit)</code>	Попытка с таймаутом → <code>true / false</code> , не зависнешь навсегда

Метод	Поведение
<code>lockInterruptibly()</code>	Захват, который можно прервать через <code>interrupt()</code> → бросает <code>InterruptedException</code>

Condition — типобезопасный аналог `wait/notify`

- Создаётся через `lock.newCondition()`
- `await()` вместо `wait()`, `signal()` / `signalAll()` вместо `notify()` / `notifyAll()`
- Плюс: **несколько условий на одном локе** (например, отдельные очереди для «документов» и «денег»)

```
class Office {
    private final Lock lock = new ReentrantLock();
    private final Condition docsCondition = lock.newCondition();
    private final Condition moneyCondition = lock.newCondition();
    private boolean hasDocs = false, hasMoney = false;

    public void bring(String type) {
        lock.lock();
        try {
            if (type.equals("docs")) {
                hasDocs = true;
                docsCondition.signalAll();
            } else if (type.equals("money")) {
                hasMoney = true;
                moneyCondition.signalAll();
            }
        } finally {
            lock.unlock();
        }
    }

    public void take(String type) {
        lock.lock();
        try {
            if (type.equals("docs")) {
                while (!hasDocs) docsCondition.await();
                hasDocs = false;
            } else if (type.equals("money")) {
                while (!hasMoney) moneyCondition.await();
                hasMoney = false;
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

```
        } finally {
            lock.unlock();           // обязательно в finally!
        }
    }
}
```

Универсальный шаблон для Condition

```
lock.lock();
try {
    while (!(condition)) {           // while, а не if!
        condition.await();
    }
    // do work
    condition.signalAll();
} finally {
    lock.unlock();                  // обязательно в finally – иначе deadlock
}
```

Зачем нужны прерываемые и таймаутные захваты

- `lockInterruptibly()` — обычный `synchronized` нельзя прервать: поток в очереди будет ждать до победного. Этот метод позволяет гибко отменять зависшие задачи.
- `tryLock(timeout)` — избавляет от вечного ожидания и deadlock: если за 50 мс лок не взят — получаем `false` и выполняем запасной сценарий.

Советы по работе с ReentrantLock

- Выход из вечного ожидания → `tryLock(timeout)` или `lockInterruptibly()`
- Короткие критические секции: всё, что можно — выносить из-под лока
- Никаких I/O, сетевых запросов и логирования в файл под локом
- `unlock()` — **только** в `finally`

Lock vs synchronized

Критерий	<code>synchronized</code>	<code>ReentrantLock</code>
Простота	Очень просто	Гибче, сложнее
Прерываемость	✗	<code>lockInterruptibly()</code>
Тайм-аут	✗	<code>tryLock(timeout)</code>
Условия	<code>wait/notify</code>	<code>Condition</code>

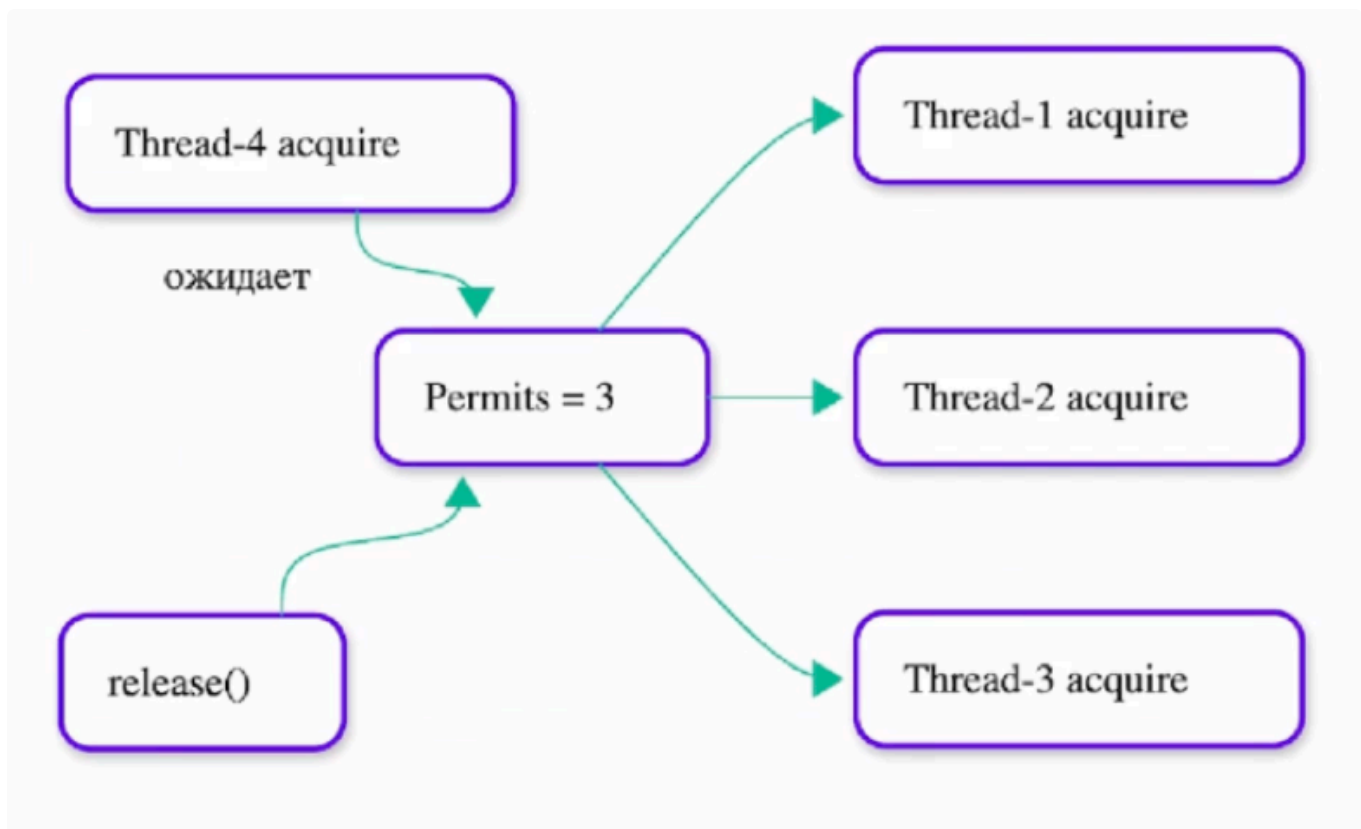
Итоги

- Сигнализация на мониторе → только через цикл `while` (проверка предиката)
- `ReentrantLock` / `Condition` — гибкая замена `synchronized` (несколько условий, таймауты, прерывание)
- Таймауты и прерываемые захваты спасают от зависаний и deadlock'ов
- Если есть возможность — вообще не блокируетесь
- Всегда уменьшайте размер критической секции

Глава 2.3. Примитивы синхронизации

Темы: `Semaphore`, `CountDownLatch`, `ReadWriteLock` + обзор остальных примитивов `java.util.concurrent`

1. Semaphore — семафор (счётчик разрешений)



Идея

Семафор хранит счётчик разрешений (**permits**):

- `acquire()` — взять разрешение (счётчик -1 ; если разрешений нет — поток блокируется)
- `release()` — вернуть разрешение (счётчик $+1$, будит ожидающих)

Пример: `permits = 3`. Thread-1, Thread-2, Thread-3 успешно вызвали `acquire()`. Thread-4 вызывает `acquire()` и **ждёт**, пока кто-то не вызовет `release()`.

Зачем нужен

- Ограничение параллелизма к общему ресурсу (не более k потоков одновременно)
- Пулы соединений, лимиты на I/O-операции, «места» у ресурса

Базовый шаблон

```
final Semaphore semaphore = new Semaphore(3);

void doWork() throws InterruptedException {
    semaphore.acquire();           // ждём разрешение
    try {
        useLimitedResource();      // крит. секция только для 3 потоков
    } finally {
```

```
semaphore.release(); // освобождаем ВСЕГДА в finally!
}
}
```

⚠ `release()` — обязательно в `finally`, иначе при исключении «утечёт» разрешение, и остальные потоки зависнут навсегда.

Fairness (честность)

```
Semaphore seats = new Semaphore(5, true); // fair = true
```

- По умолчанию семафор **нечестный**: пробуждение ожидающих не гарантирует порядок, один поток может «по кд» оставаться в хвосте очереди (starvation возможен).
- `new Semaphore(n, true)` включает FIFO-очередь — разрешения выдаются в порядке ожидания.
- **Цена честности**: усложняется внутренний алгоритм → растёт среднее время ожидания. Берём `fair` только если порядок реально важен.

Демо: фуд-корт (15 посетителей, 5 мест)

```
public class SemaphoreFoodCourtDemo {

    static class Visitor implements Runnable {
        private final int id;
        private final Semaphore seats;

        Visitor(int id, Semaphore seats) { this.id = id; this.seats = seats;
    }

    @Override
    public void run() {
        try {
            log("Пришёл в фудкорт, пытаюсь занять место... (ID: %d)",
id);

            seats.acquire(); // нет мест — ждём
            log("Сел за стол. Свободных мест: %d",
seats.availablePermits());
            sleep(1000 + ThreadLocalRandom.current().nextInt(2000)); //
ест 1–3 сек

            log("Поел и уходит (ID: %d)", id);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        } finally {

```

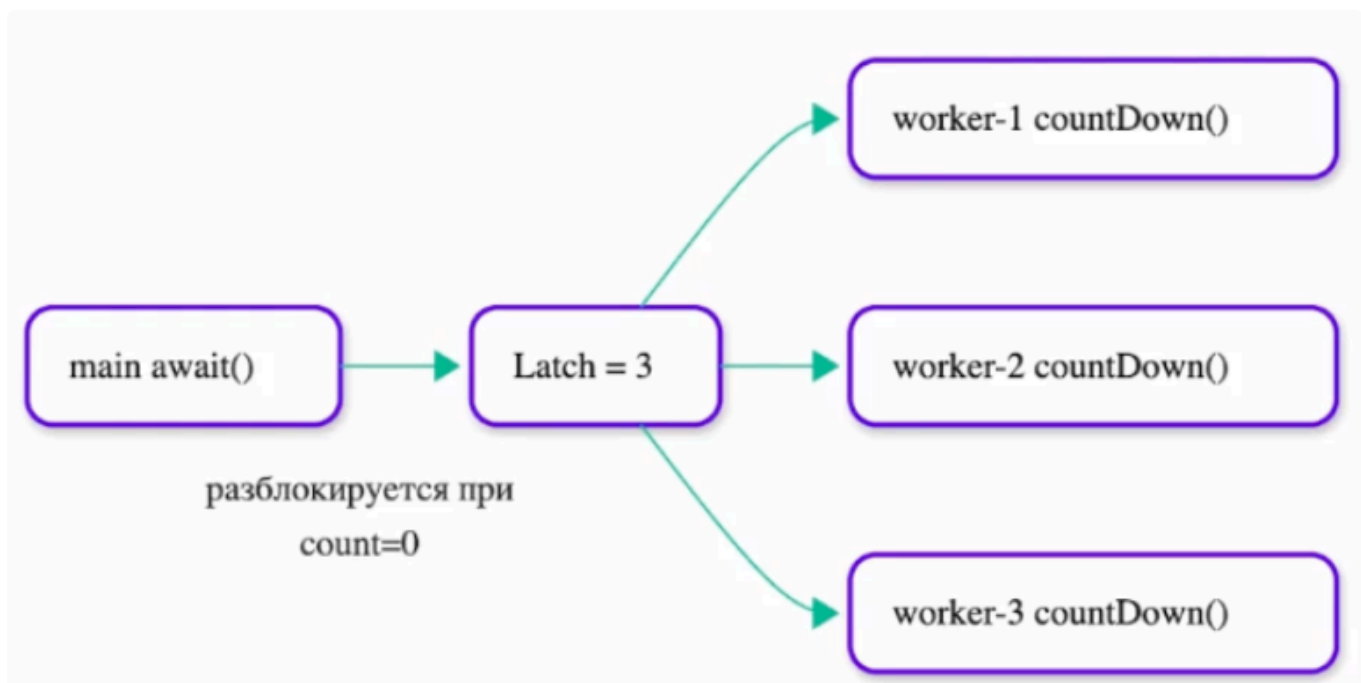
```

        seats.release(); // место освобождаем
    }
    всегда
    log("Освободил место. Свободных мест теперь: %d",
seats.availablePermits());
}
}

public static void main(String[] args) {
    Semaphore seats = new Semaphore(5); // 5 мест
    for (int i = 1; i <= 15; i++)
        new Thread(new Visitor(i, seats), "Посетитель-" + i).start();
}
}

```

2. CountdownLatch — защёлка-счётчик



Идея

- Потоки вызывают `await()` — блокируются, пока счётчик **не станет 0**
- Другие потоки вызывают `countDown()` — уменьшают счётчик на 1
- Счётчик **не сбрасывается** — latch одноразовый

Зачем нужен

- **Стартовый барьер:** «все готовы — поехали»

- **Ожидание завершения N задач:** main ждёт, пока все воркеры не отработают

Демо: ракета ждёт расчёты инженеров

```
public class CountdownLatchRocketDemo {

    static class Engineer implements Runnable {
        private final int id;
        private final CountdownLatch latch;
        private final int[] flightParams;

        @Override
        public void run() {
            log("Начал расчёты...");
            sleep(500 + ThreadLocalRandom.current().nextInt(1000));
            int result = ThreadLocalRandom.current().nextInt(1000);
            flightParams[id] = result;          // 1) пишем результат
            log("Расчёт завершён: %d", result);
            latch.countDown();                  // 2) отчитываемся
            log("Отчитался, latch=%d", latch.getCount());
        }
    }

    static class Rocket implements Runnable {
        private final CountdownLatch latch;
        private final int[] flightParams;

        @Override
        public void run() {
            try {
                log("Ожидаю готовности инженеров...");
                latch.await();                  // ждём count == 0
                log("Все инженеры отчитались, стартуем!");
                int checksum = Arrays.stream(flightParams).sum();
                log("Полёт запущен. Контрольная сумма: %d", checksum);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }

    public static void main(String[] args) {
        int engineers = 10;
        CountdownLatch latch = new CountdownLatch(engineers);
        int[] flightParams = new int[engineers];
```

```
        new Thread(new Rocket(latch, flightParams), "Ракета").start();
        for (int i = 0; i < engineers; i++)
            new Thread(new Engineer(i, latch, flightParams), "engineer-" +
i).start();
    }
}
```

Почему данные видны без `volatile` ? (Happens-Before)

В `CountDownLatch` есть официальное правило видимости памяти:

Вывод из `await()` «видит» всё, что произошло до вызова `countDown()`.

Цепочка в примере:

1. Инженер пишет `flightParams[id] = result`
2. Инженер вызывает `countDown()`
3. Ракета выходит из `await()`
4. Ракета читает массив → гарантированно видит все записи

Поэтому **никакой** `volatile` / `synchronized` **не нужен** — `latch` сам обеспечивает видимость.

3. `ReadWriteLock` — блокировка «читатели-писатели»

Идея

Доступ делится на два вида:

- **Read lock** — разделяемый: несколько читателей могут держать его одновременно
- **Write lock** — эксклюзивный: только один писатель, и **никаких читателей** в этот момент

Логика работы очереди

- Читатели друг другу не мешают → их работу распараллеливаем
- Когда приходит **писатель**:
 - новые читатели видят писателя и встают в очередь, не обгоняя его
 - писатель ждёт, пока текущие читатели дочитают
 - как только читателей не осталось — писатель пишет
- Несколько писателей выстраиваются в очередь между собой

Зачем нужен

Эффективен при **R-heavy** нагрузке (чтений значительно больше, чем записей): чтения не блокируют друг друга.

Шаблон

```
final ReadWriteLock rw = new ReentrantReadWriteLock(true); // fair
final Map<String, String> cache = new HashMap<>();

String get(String k) {
    rw.readLock().lock();
    try { return cache.get(k); }
    finally { rw.readLock().unlock(); }
}

void put(String k, String v) {
    rw.writeLock().lock();
    try { cache.put(k, v); }
    finally { rw.writeLock().unlock(); } // в finally!
}
```

⚠ Классическая опечатка из кода слайда: в `put()` освободить надо через `lock.writeLock().unlock()`, а не повторный `lock()`.

Ограничения

- **Нельзя апгрейд:** держа read lock, нельзя захватить write lock (риск дедлока — писатель ждёт сам себя)
- **Starvation писателей:** при непрерывном потоке читателей писатель может долго не получить лок (лечится fair-режимом)
- Два лока — двойная забота про `finally`: оба `unlock()` должны быть в `finally`

Демо: 3 читателя + 1 «босс»-писатель

```
public class ReadWriteLockDemo {

    static class ReaderWorker {
        private final ReadWriteLock lock;
        void doWorkInLoop() {
            while (!Thread.currentThread().isInterrupted()) {
                sleep(500);
                try {
                    lock.readLock().lock();
                }
            }
        }
    }
}
```

```

        log("Reader working...");
        spinOnCpu(500);
    } finally {
        lock.readLock().unlock();    // а не .lock()!
    }
}

static class WriterWorker {
    private final ReadWriteLock lock;
    void doWorkInLoop() {
        while (!Thread.currentThread().isInterrupted()) {
            sleep(5000);
            try {
                log("--- BIG BOSS HERE ---");
                lock.writeLock().lock();
                log("Writer working...");
                spinOnCpu(5000);
            } finally {
                lock.writeLock().unlock();    // а не .lock()!
            }
        }
    }
}

public static void main(String[] args) {
    ReadWriteLock lock = new ReentrantReadWriteLock();
    List<Thread> threads = List.of(
        new Thread(() -> new ReaderWorker(lock).doWorkInLoop(), "reader-
1"),
        new Thread(() -> new ReaderWorker(lock).doWorkInLoop(), "reader-
2"),
        new Thread(() -> new ReaderWorker(lock).doWorkInLoop(), "reader-
3"),
        new Thread(() -> new WriterWorker(lock).doWorkInLoop(), "writer-
1")
    );
    threads.forEach(Thread::start);
    sleep(30_000);
    threads.forEach(Thread::interrupt);    // останавливаем через
interrupt
}
}

```

4. Сводная таблица примитивов

Инструмент	Ключевая идея	Когда брать	Плюсы	Осторожно
ReentrantLock	Явный эксклюзивный лок	Таймауты, прерывание, несколько условий	Гибкость, Condition	Сложнее протокол; легко забыть <code>unlock()</code>
Semaphore	Счётчик разрешений	Ограничить параллелизм (I/O, пулы)	Просто, fair-режим	Не защищает инварианты данных
ReadWriteLock	R — совместно, W — эксклюзивно	Много чтений, мало записей	Высокая пропускная способность при R-heavy	Starvation писателей; нельзя upgrade
CountDownLatch	Счётчик $N \rightarrow 0$, <code>await()</code>	Старт/инициализация, ожидание завершения N задач	Простая координация + happens-before	Одноразовый барьер

5. Другие примитивы `java.util.concurrent` (обзорно)

Примитив	Что это	Зачем
CyclicBarrier	Барьер для группы потоков: все ждут, пока не соберутся все участники	Синхронизация фаз работы; в отличие от latch — многоразовый (циклический)
Exchanger	Точка двунаправленного обмена данными между парой потоков	Обмен буферами/пакетами «один-к-одному» без общей памяти
Phaser	Расширяемый многофазный барьер с динамическим числом участников	Сложные многошаговые протоколы синхронизации
StampedLock	Блокировка с оптимистичным чтением и штампами (stamp)	Очень высокая доля чтений: читатель читает без лока и лишь проверяет валидность штампа после

6. Итоги

- **Semaphore** — ограничиваем, сколько потоков одновременно работают с ресурсом (`acquire / release` , `fair` по желанию)
 - **CountDownLatch** — ждём событие «N задач завершено»; даёт `happens-before` бесплатно; одноразовый
 - **ReadWriteLock** — параллелим читателей, изолируем писателя; выигрыш только при преобладании чтений
 - Все примитивы: освобождение (`release / unlock`) — **строго в `finally`**
 - Для выбора инструмента → сводная таблица в разделе 4
-

Что дальше?

1. Мини-тест: «Semaphore vs CountDownLatch — что выбрать для X?»
2. Следующая тема: атомики (`AtomicInteger` , `CAS`) и `volatile`
3. Разобрать `StampedLock` и оптимистичное чтение на живом примере

Глава 2.4. Atomic Types and CAS-loop

Atomic-типы: введение

Atomic-типы — классы из пакета `java.util.concurrent.atomic` , обеспечивающие **атомарные (неделимые)** операции чтения и записи **без блокировок**, обычно через CAS (compare-and-swap).

Класс	Назначение
<code>AtomicInteger</code> , <code>AtomicLong</code>	Атомарные счётчики
<code>AtomicBoolean</code>	Флаг состояния
<code>AtomicReference<T></code>	Атомарная ссылка на объект

Атомарность без `synchronized` = нет блокировок, нет переключения контекста, нет deadlock'ов.

Демо: `AtomicInteger`

```

import java.util.concurrent.atomic.AtomicInteger;

public class AtomicIntegerDemo {
    static AtomicInteger counter = new AtomicInteger(0);

    public static void main(String[] args) throws InterruptedException {
        int threadsCount = 4;
        int perThread = 250_000;
        Thread[] threads = new Thread[threadsCount];

        Runnable r = () -> {
            for (int k = 0; k < perThread; k++) {
                counter.getAndIncrement(); // аналог counter++
            }
        };

        for (int i = 0; i < threadsCount; i++) {
            threads[i] = new Thread(r);
            threads[i].start();
        }
        safeJoinAll(threads);

        System.out.println("counter = " + counter.get());
        // ожидаем: 4 * 250_000 = 1_000_000 — всегда точно
    }
}

```

Почему обычный `int` тут не сработает: операция `counter++` — это три шага (прочитать → увеличить → записать). Два потока могут прочитать одно значение, и одно обновление затрёт другое (потерянное обновление).

Классическая альтернатива на `synchronized` (для сравнения):

```

static class Counter {
    private int counter = 0;
    public synchronized void increment() { counter++; }
    public synchronized int getCounter() { return counter; }
}

```

Работает, но тяжелее: монитор, блокировки, конкуренция за вход. `AtomicInteger` решает ту же задачу дешевле.

Полезные методы:

Метод	Что делает	Аналог в Java (обычные операции)
<code>getAndIncrement()</code>	Увеличивает значение на 1 и возвращает старое значение	<code>i++</code> (атомарно)
<code>incrementAndGet()</code>	Увеличивает значение на 1 и возвращает новое значение	<code>++i</code> (атомарно)
<code>getAndAdd(delta)</code>	Добавляет <code>delta</code> и возвращает старое значение	<code>long old = i; i += delta; return old;</code>
<code>addAndGet(delta)</code>	Добавляет <code>delta</code> и возвращает новое значение	<code>i += delta; return i;</code>
<code>compareAndSet(expect, update)</code>	Меняет значение на <code>update</code> , только если текущее равно <code>expect</code>	Ядро CAS (Compare-And-Swap)
<code>get()</code>	Возвращает текущее значение	Атомарное чтение (<code>volatile</code>)

Демо: AtomicReference

```
import java.util.concurrent.atomic.AtomicReference;

public class AtomicReferenceDemo {
    static AtomicReference<String> name = new AtomicReference<>("Alice");

    public static void main(String[] args) throws InterruptedException {
        Runnable r = () -> name.updateAndGet(
            n -> n.equals("Alice") ? "Bob" : "Alice"    // атомарный swap
        );

        Thread[] threads = new Thread[100];
        for (int i = 0; i < threads.length; i++) {
            threads[i] = new Thread(r);
            threads[i].start();
        }
        safeJoinAll(threads);

        System.out.println(name.get());
        // Всегда будет "Alice": 100 потоков = чётное число атомарных
        swap'ов
    }
}
```

```
}  
}
```

Можно подумать почему бы не использовать `volatile`, но тут атомарно сравнивать можем а с `volatile` не можем.

`updateAndGet` применяет функцию атомарно: читает значение, вычисляет новое и записывает через CAS-цикл. Все 100 переключений — неделимы, поэтому результат детерминирован.

CAS (Compare-And-Swap) — как это работает

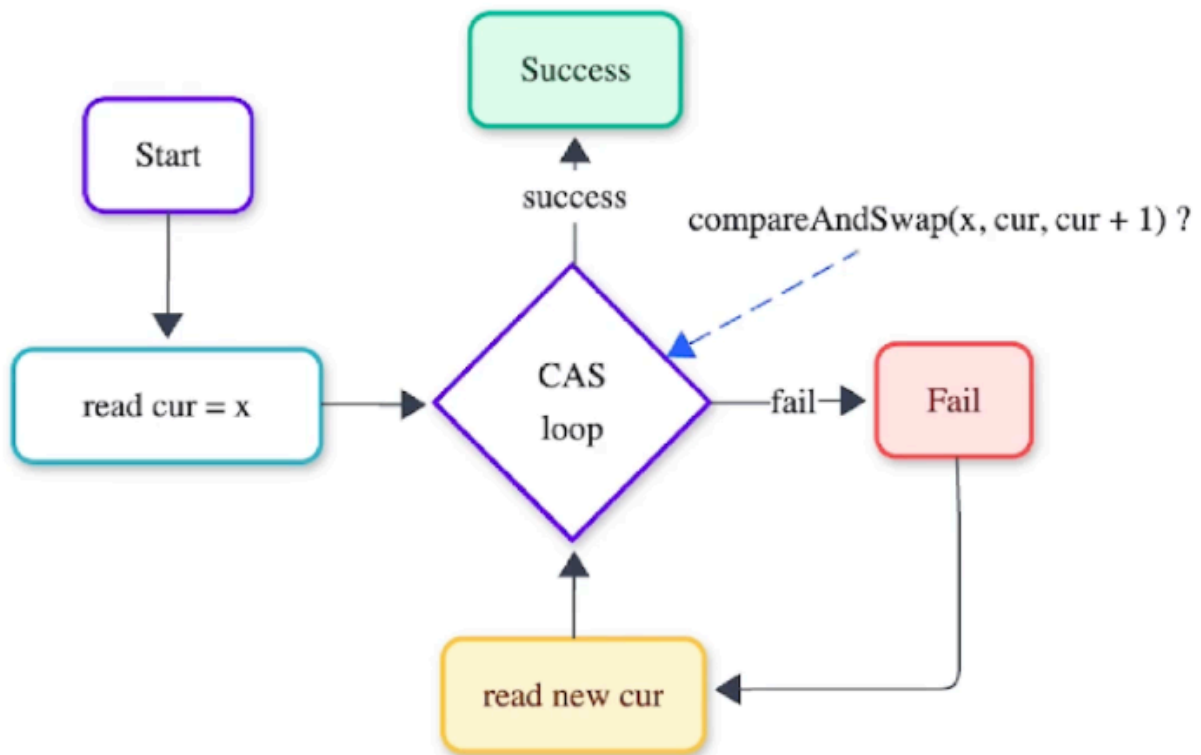
CAS — аппаратная атомарная инструкция процессора. Логика:

```
compareAndSwap(адрес, expectedValue, newValue):  
    если *адрес == expectedValue:  
        *адрес = newValue    ← одной неделимой инструкцией  
        return true  
    иначе:  
        return false        ← ничего не меняем
```

CAS-loop (спин-цикл)

Атомарные операции строятся как цикл повторных попыток:

```
value := 0  
function inc():  
    do {  
        old := load(value)    // 1. читаем текущее  
        new := old + 1        // 2. вычисляем новое  
    } while (!CAS(addr(value), old, new)) // 3. пытаемся записать
```



- **Сравнивает** текущее значение с ожидаемым (`expectedValue`)
- **Если совпало** — меняет на новое одной атомарной инструкцией CPU
- **Если не совпало** (другой поток успел раньше) — операция не выполняется, поток **повторяет попытку** (спин-цикл)

CAS **АБСОЛЮТНЫЙ ЛИДЕР** по времени в низкой конкуренции и **АБСОЛЮТНЫЙ** неудачник при высокой, т.е. в некоем роде все или ничего.

- **При низкой конкуренции:** Он выдает **космическую скорость**. Потoki не тратят ни одной наносекунды на создание замков, проверку блокировок и укладывание в спячку. База данных выдает свой исторический максимум пропускной способности.
- **При высокой конкуренции:** Он превращается в **ядерный взрыв для процессора** (Retry Storm). Потoki начинают жрать 100% ресурсов CPU, бесконечно перечитывая и пересчитывая одни и те же данные по кругу. Система не просто замедляется — она может полностью задедлотить саму себя на аппаратном уровне и упасть.

CAS нужен везде, где логика звучит так: **«Я хочу обновить данные X на значение Y, но только если за время моих размышлений никто другой эти данные X не трогал».**

Свойства CAS

- **✓** Все операции — **без `synchronized`**, но с гарантией видимости по JMM
- **⚠** При **высокой конкуренции** CAS может «вертеться» в цикле — это не бесплатно (процессорное время на повторы)

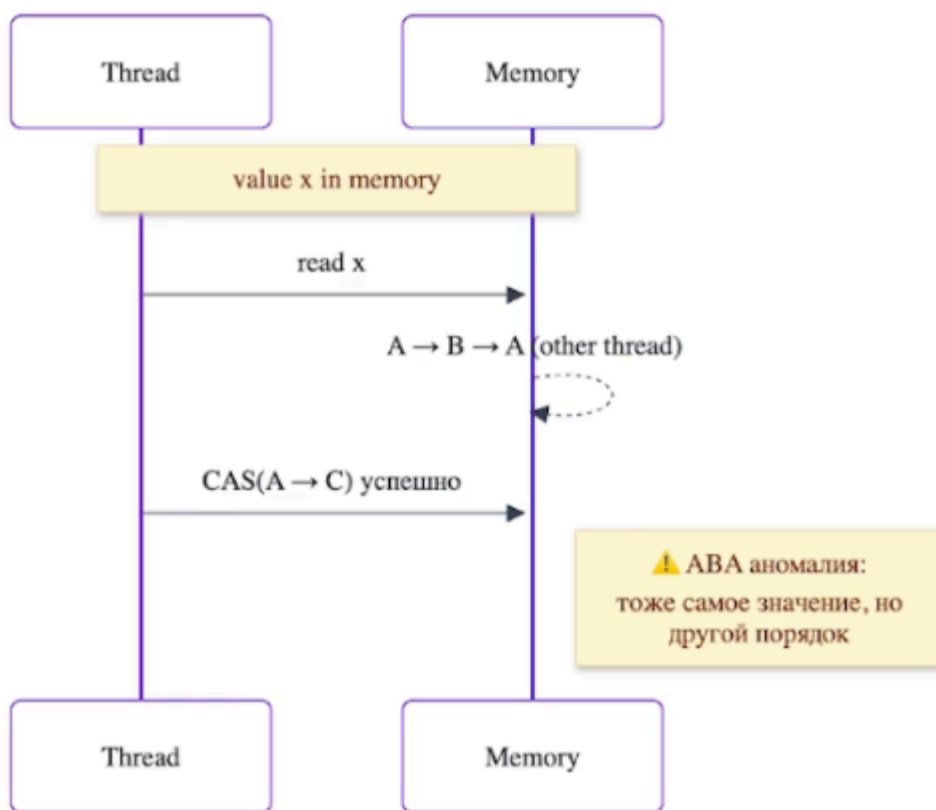
- 💡 Когда конфликтов много → много спинов. Лекарство: LongAdder / LongAccumulator (разбивают счётчик на ячейки по потокам и суммируют при чтении)

Проблема АВА

Суть

Поток читает значение **A**, собирается сделать CAS(**A** → **C**). Между тем другой поток успевает сделать **A** → **B** → **A** — значение снова стало "A".

- CAS «думает», что ничего не изменилось, и успешно обновляет данные поверх новых значений
- Аномалия: то же значение, но **другой порядок событий** — промежуточное состояние потеряно



Демо АВА

```
public class ABAProblemDemo {  
    public static void main(String[] args) {  
        AtomicReference<String> reference = new AtomicReference<>("A");  
    }  
}
```

```

// Thread-0: читает A, ждёт, пытается записать C
Thread thread1 = new Thread(() -> {
    String oldValue = reference.get();
    log("Прочитал значение: " + oldValue);

    safeSleep(300); // даём время другому потоку всё поменять

    boolean ok = reference.compareAndSet(oldValue, "C");
    log("CAS(A→C) результат: " + ok + ", текущее значение: " +
reference.get());
});

// Thread-1: A → B → A
Thread thread2 = new Thread(() -> {
    safeSleep(100);
    log("Меняю A → B");
    reference.compareAndSet("A", "B");

    safeSleep(100);
    log("Возвращаю B → A (ABA)");
    reference.compareAndSet("B", "A");
});

thread1.start();
thread2.start();
}
}

```

Вывод:

```

Прочитал значение: A
Меняю A → B
Возвращаю B → A (ABA)
CAS(A→C) результат: true, текущее значение: C ← CAS прошёл, хотя данные
менялись!
Финальное значение: C

```

Решение: AtomicStampedReference (версионирование)

Вместе со значением хранится **штамп (stamp)** — счётчик версий, который меняется при каждом обновлении. CAS теперь проверяет пару (**значение + штамп**):

```

AtomicStampedReference<String> reference =
    new AtomicStampedReference<>("A", 0); // ref + stamp

```

```
// get() отдаёт значение и кладёт штамп в int[] holder:
int[] holder = new int[1];
String oldValue = reference.get(holder);

// CAS: expectedRef, newRef, expectedStamp, newStamp
boolean ok = reference.compareAndSet(
    oldValue, "C",
    holder[0], holder[0] + 1    // штамп тоже должен совпасть и
    инкрементиться
);
```

Даже если значение вернулось к "A", штамп уже другой → CAS **вернёт false**, и поток перечитает актуальное состояние:

```
writer-1 Прочитал значение: A (stamp=0)
writer-2 Меняю A → B          (stamp: 0→1) OK: true
writer-2 Возвращаю B → A (ABA) (stamp: 1→2) OK: true
writer-1 CAS(A→C) → OK: false ← штамп не совпал! ABA отловлена
Финальное значение: A
```

💡 Это и есть **оптимистичная блокировка** (в отличие от пессимистической — `synchronized / Lock`): не блокируем ресурс заранее, а пытаемся обновить и проверяем версию; при конфликте — перечитываем и повторяем. Тот же принцип лежит в основе `StampedLock` и оптимистичного версионирования в БД.

CAS в бд

В таблицу добавляется колонка `version`. Обновление происходит одной неделимой инструкцией с проверкой старой версии:

```
UPDATE accounts
SET balance = 80, version = 2
WHERE id = 777 AND version = 1; -- <---- Ядро CAS

while (true) {
    Account acc = db.find(777);                // 1. Читаем (balance=100,
    version=1)
    int newBalance = acc.balance - 20;          // 2. Вычисляем (80)

    int updated = db.update(newBalance, acc.version); // 3. CAS-запрос
    if (updated > 0) break;                      // Успех → выходим
    // Если updated == 0 → уходим на новый круг (retry)
}
```

- **Успех (true):** Если версия в БД всё еще 1, база меняет данные и возвращает `Rows affected: 1`.
- **Промаш (false):** Если другой поток уже изменил строку и версия стала 2, условие `WHERE version = 1` не сработает. База ничего не изменит и вернет `Rows affected: 0`.

CAS **АБСОЛЮТНЫЙ ЛИДЕР** по времени в низкой конкуренции и **АБСОЛЮТНЫЙ** неудачник при высокой, те в некоем роде все или ничего.

- **При низкой конкуренции:** Он выдает **космическую скорость**. Потoki не тратят ни одной наносекунды на создание замков, проверку блокировок и укладывание в спячку. База данных выдает свой исторический максимум пропускной способности.
- **При высокой конкуренции:** Он превращается в **ядерный взрыв для процессора** (Retry Storm). Потoki начинают жрать 100% ресурсов CPU, бесконечно перечитывая и пересчитывая одни и те же данные по кругу. Система не просто замедляется — она может полностью задедлочить саму себя на аппаратном уровне и упасть.

Пессимистичные локи (`FOR UPDATE`) обеспечивают стабильность при высокой конкуренции за счет защиты CPU базы данных, однако при низкой нагрузке они избыточны и неэффективны из-за затрат на создание замков. В условиях высокой конкуренции они предотвращают перегрузку процессора, но создают риск исчерпания пула соединений (Connection Pool Exhaustion) в приложении, так как ожидающие потоки удерживают сетевые коннекты. При запредельной конкуренции эффективным решением становится перенос горячих данных в оперативную память (например, Redis) или организация очереди через брокеры сообщений (Kafka/RabbitMQ).

Таблица атомарных классов

Тип	Когда брать	Заметка
<code>AtomicInteger / AtomicLong</code>	Счётчики, индексы	Быстрые инкременты
<code>AtomicBoolean</code>	Флаги	Простые переключатели
<code>AtomicReference</code>	Обмен ссылкой	Нестандартные структуры
<code>AtomicStampedReference</code>	Против ABA	Хранит «штамп» версии

Итоги

- Atomic-типы дают атомарность **без блокировок** через CAS-инструкцию CPU
- CAS-loop = «прочитал → вычислил → попробовал записать → при конфликте повторил»
- CAS АБСОЛЮТНЫЙ ЛИДЕР по времени в низкой конкуренции и АБСОЛЮТНЫЙ неудачник при высокой, те в некоем роде все или ничего.
- Дёшево при низкой конкуренции; при высокой — спины → LongAdder
- AtomicReference не защищает от ABA → AtomicStampedReference (штамп = версия)
- Принцип CAS = **оптимистичная блокировка**: не запираем ресурс, а проверяем версию при записи

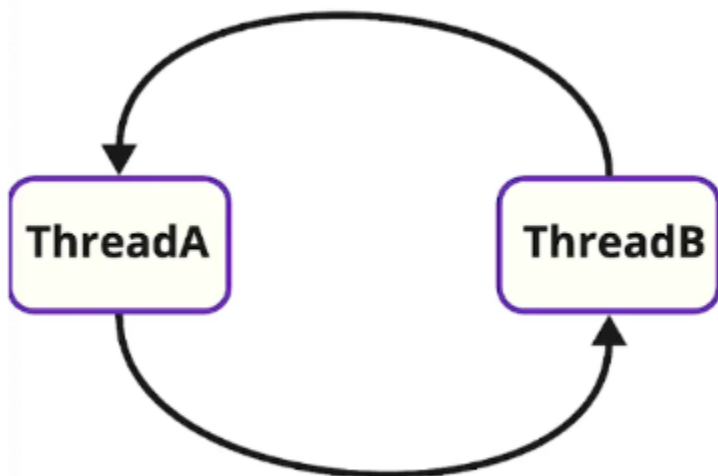
Глава 2.5. Deadlock and Livelock

Темы: взаимные блокировки, условия Кофмана, порядок локов, tryLock(timeout), обедающие философы, лайвлок

1. Deadlock — взаимная блокировка

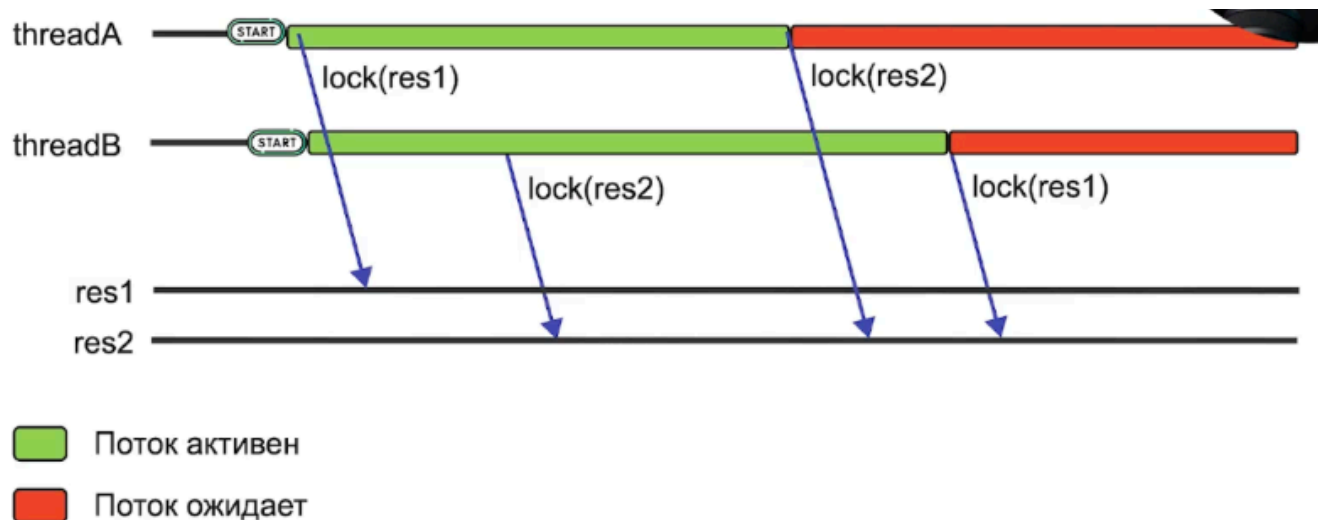
Определение

- Два или более потоков **блокируют друг друга**
- Каждый ожидает ресурс, который уже **захвачен другим**
- **Ни один поток не может продолжить выполнение** — навсегда



⚠ Дедлок **не всегда возникает сразу** — может понадобиться много итераций, прежде чем он «выстрелит». Отсутствие дедлока в тесте ≠ его отсутствие в проде.

Таймлайн возникновения



Классическая причина: **блокировки берутся в разном порядке в разных потоках.**

2. Демо: воспроизводим дедлок (перевод денег)

```
public class DeadlockAccountTransferExample {

    public static class Account {
        private final int id;
        private int balance;
        private final Object lock = new Object();

        public Account(int id, int balance) {
            this.id = id;
            this.balance = balance;
        }
    }

    // ПЕРЕВОД: держим лок ОТКУДА и КУДА одновременно
    public static void makeTransfer(Account from, Account to, int amount) {
        synchronized (from.lock) {
            synchronized (to.lock) {
                Logger.log("Transfer from acc %s to acc %s", from.id,
to.id);

                from.balance -= amount;
                to.balance += amount;
            }
        }
    }

    public static void main(String[] args) {
```

```

var account1 = new Account(1, 100);
var account2 = new Account(2, 100);

for (int i = 0; i < 1000; ++i) {
    System.out.println("Iteration " + i + " started");
    Thread t1 = new Thread(() -> makeTransfer(account1, account2,
100));

    Thread t2 = new Thread(() -> makeTransfer(account2, account1,
100)); // обратный порядок!

    t1.start();
    t2.start();
    safeJoin(t1, t2);
    System.out.println("Iteration " + i + " ended");
}
System.out.println("All iterations ended");
}
}

```

Проблема: t1 берёт локи 1 → 2, t2 — 2 → 1. В какой-то итерации каждый захватит «свой» первый лок → дедлок, программа зависает на середине.

3. Условия Коффмана

Дедлок возможен, только если выполняются **ВСЕ 4 условия**:

#	Условие	Смысл
1	Взаимное исключение	Ресурс неделим, занят только одним потоком
2	Удержание и ожидание	Поток держит ≥1 ресурс и запрашивает ещё
3	Отсутствие принудительного освобождения	Ресурс нельзя отобрать у держателя
4	Циклическое ожидание	Есть цикл потоков, где каждый ждёт ресурс следующего

💡 Ключевая стратегия: **нарушить хотя бы одно условие** → дедлок невозможен.

4. Решение 1: фиксированный порядок локов

Нарушаем условие «циклического ожидания» — все потоки берут локи в **одинаковом порядке**:

```
public static void makeTransfer(Account from, Account to, int amount) {
    // Определяем порядок одинаково для ВСЕХ потоков
    Account first = from.lock.hashCode() <= to.lock.hashCode() ? from : to;
    Account second = (first == from) ? to : from;

    synchronized (first.lock) {
        synchronized (second.lock) {
            Logger.log("Transfer from acc %s to acc %s", from.id, to.id);
            from.balance -= amount;
            to.balance += amount;
        }
    }
}
```

Теперь оба потока берут локи в одном порядке → цикл ожидания невозможен → дедлока нет.

5. Решение 2: tryLock(timeout) + откат

Нарушаем «ожидание бесконечное» — не получилось взять лок за время → **отпускаем уже взятый и пробуем снова**:

```
public static void makeTransfer(Account from, Account to, int amount) {
    while (true) {
        try {
            if (!from.lock.tryLock(10, TimeUnit.MILLISECONDS)) {
                Logger.log("Не удалось взять лок с acc %s. Retry...",
from.id);

                safeSleepRandom(5, 15); // случайная пауза против livelock
                continue;
            }
            if (!to.lock.tryLock(10, TimeUnit.MILLISECONDS)) {
                Logger.log("Не удалось взять лок с acc %s. Откатываем и
retry...", to.id);
                from.lock.unlock(); // ← ОТКАТ: отпускаем первый лок!
                safeSleepRandom(5, 15);
                continue;
            }
        }
    }
}
```

```

    }

    // оба лока захвачены — работаем
    Logger.log("Transfer from acc %s to acc %s", from.id, to.id);
    from.balance -= amount;
    to.balance   += amount;
    return;

} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    return;
} finally {
    // освобождаем ТОЛЬКО то, что реально держим
    if (to.lock.isHeldByCurrentThread()) to.lock.unlock();
    if (from.lock.isHeldByCurrentThread()) from.lock.unlock();
}
}
}

```

Ключевые моменты:

- Не взял второй лок → **отпускаешь первый** и ретраишься
- `finally` с проверкой `isHeldByCurrentThread()` — чтобы не кинуть `IllegalMonitorStateException`
- **Рандомная пауза** перед ретраем: без неё два потока могут «сцепиться» (см. livelock)

6. Livelock — «вежливый» дедлок

Определение

- Потоки **активно работают**, постоянно уступая друг другу, но **не продвигаются к цели**
- Состояние **меняется**, прогресса нет
- Частая причина: **симметричные протоколы «вежливого» отступления** (оба отпускают локи одновременно → оба ретраятся одновременно → повтор)

T1: держит A? отпустил B → пытается взять B → конфликт → отступает
 T2: держит B? отпустил A → пытается взять A → конфликт → отступает



Лечение

Рандомный backoff: разнести время повторных попыток (`safeSleepRandom(5, 15)`) — потоки перестают быть синхронными.

```
public static void safeSleepRandom(long lower, long upper) {  
    try {  
        Thread.sleep(ThreadLocalRandom.current().nextLong(lower, upper));  
    } catch (InterruptedException e) {  
        Thread.currentThread().interrupt();  
    }  
}
```

	Deadlock	Livelock
Состояние потоков	Ждут, ничего не делают	Активно работают/отступают
Прогресс	Нет	Нет

Демо: муж и жена за одной ложкой

Постановка: у супругов **одна ложка**. Оба голодны. Каждый «вежливо» отдаёт ложку другому, если тот голоден. Никто не ест — **лайвлок**.

```
public class LivelockExample {

    static class Spoon {
        private volatile Diner owner;    // volatile – чтобы видеть смену
        владельца

        public Spoon(Diner owner) { this.owner = owner; }

        public Diner getOwner() { return owner; }

        public synchronized void setOwner(Diner diner) { this.owner = diner;
    }

    public synchronized void use() {
        System.out.println(owner.name + " ест!");
    }
}

static class Diner {
    private final String name;
    private boolean isHungry;

    public Diner(String name, boolean hungry) {
        this.name = name;
        this.isHungry = hungry;
    }

    public void eatWith(Spoon spoon, Diner spouse) {
        while (isHungry) {
            // Ложка не у меня – жду
            if (spoon.getOwner() != this) {
                safeSleep(10);
                continue;
            }

            // ⚠ Если другой голоден – уступаю (ИМЕННО ТУТ ЛАЙВЛОК)
            if (spouse.isHungry) {
                System.out.println(name + ": мой(я) " + spouse.name + "
голоден, отдаю ложку...");
                spoon.setOwner(spouse);
                continue;
            }
        }
    }
}
```

```

    }

    // Ем (сюда никогда не дойдём, пока оба вежливы)
    spoon.use();
    isHungry = false;
    System.out.println(name + ": я поел!");
    spoon.setOwner(spouse);
}
}

public static void main(String[] args) {
    Diner husband = new Diner("Муж", true);
    Diner wife     = new Diner("Жена", true);
    Spoon spoon    = new Spoon(husband);

    Thread t1 = new Thread(() -> husband.eatWith(spoon, wife));
    Thread t2 = new Thread(() -> wife.eatWith(spoon, husband));

    t1.start();
    t2.start();
}
}

```

Вывод (лайвллок в действии):

```

Муж: мой(я) Жена голоден, отдаю ложку...
Жена: мой(я) Муж голоден, отдаю ложку...
Муж: мой(я) Жена голоден, отдаю ложку...
... // бесконечно, CPU горит, прогресса нет

```

Почему это именно livelock, а не deadlock

- Никто **не блокирован** — потоки крутятся, меняют владельца, печатают
- Но логика «вежливого уступания» симметрична → **состояние меняется, цель не достигается**

Лечение

Закомментировать блок «вежливости» — и хоть кто-то поест:

```

// if (spouse.isHungry) {
//     spoon.setOwner(spouse);

```

```
// continue;  
// }
```

В реальных системах аналог — **рандомный backoff**: иногда уступаем, иногда едим (вероятностное нарушение симметрии).

Ключевые детали кода

- `volatile Diner owner` — без него поток мог бы не увидеть, что ложку ему отдали
- `synchronized setOwner()/use()` — защита самой ложки (атомарная смена владельца)
- `isHungry` не `volatile` — и это ОК: каждый поток читает **свой** флаг (поле своего объекта `Diner`)

7. Метафора: обедающие философы

Постановка

- 5 философов за круглым столом, между каждыми — по вилке
- Философы чередуют состояния: **думает** → **ест** → **думает**
- Чтобы есть, нужны **обе вилки** — левая и правая



Маппинг на потоки

Философ	Поток
Вилка	Ресурс (лок)
Потянулся за обеими вилками	Поток держит 2 лока

Deadlock-сценарий: все одновременно берут левую вилку → каждый ждёт правую у соседа → кольцо ожидания.

Решения:

1. Взять вилки в фиксированном порядке (например, по номеру) → нет циклического ожидания
2. «Вилка-арбитр» / ограничение числа одновременно сидящих (Semaphore на 4 места)
3. tryLock + откат + рандомный backoff → нет бесконечного ожидания

8. Итоги

- **Deadlock** = кольцо потоков, каждый ждёт ресурс соседа → вечный ступор
- Возможен только при **всех 4 условиях Коффмана** → ломаем хотя бы одно
- Практические приёмы:
 1. **Единый порядок захвата локов** (по id/hashCode)
 2. tryLock(timeout) + откат + рандомный backoff (через ReentrantLock)
- **Livelock** — «вежливый» аналог: потоки не ждут, а бесконечно отступают → лечится рандомной паузой
- Дедлок может не проявиться в тестах → закладывайте порядок локов/таймауты **на этапе проектирования**

Глава 2.6. ThreadLocal устройство, утечки

ThreadLocal — что это

ThreadLocal — это как «карман» внутри каждого потока: можно положить данные, и другие потоки их не увидят.

```
static ThreadLocal<String> user = new ThreadLocal<>();

public static void main(String[] args) {
```

```

Runnable task = () -> {
    user.set(Thread.currentThread().getName()); // кладём в СВОЙ карман
    System.out.println("Handling " + user.get());
    user.remove(); // чистим за собой
};
new Thread(task, "user-1").start();
new Thread(task, "user-2").start();
}

```

Свойства

- **Изоляция данных на уровне потока** — каждый поток видит только своё значение
- Удобно для **локальных данных**, не требующих синхронизации (гонок нет по построению)
- Классический пример — хранить **данные текущего запроса пользователя**

Где используется в реальной жизни

В Spring (например, `SecurityContextHolder`) хранит контекст аутентификации через `ThreadLocal`: каждый HTTP-запрос обрабатывается **своим потоком** (потоки переиспользуются пулом), поэтому каждому запросу можно привязать «свои» данные — текущего юзера, трейс-id, транзакцию — без передачи их через все методы в параметрах.

Демо: базовое использование

```

public class ThreadLocalBasicsDemo {
    private static final ThreadLocal<String> TL = new ThreadLocal<>();

    public static void main(String[] args) throws Exception {
        Thread t1 = new Thread(() -> runWith("A"), "t1");
        Thread t2 = new Thread(() -> runWith("B"), "t2");
        t1.start();
        t2.start();
        t1.join();
        t2.join();
    }

    static void runWith(String val) {
        TL.set(val);
        Logger.log("TL=%s", TL.get()); // t1 видит "A", t2 — "B"
        TL.remove();
    }
}

```

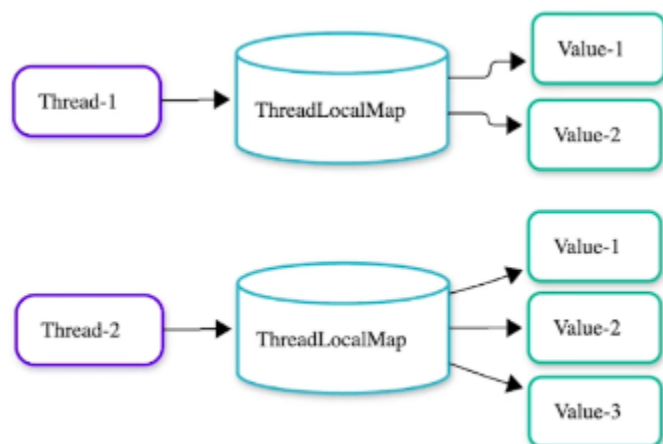
```
}  
}
```

Вывод:

```
[t1] TL=A  
[t2] TL=B
```

Один и тот же статический объект `TL`, но значения не пересекаются — никаких блокировок не нужно.

Как устроен внутри



- Каждый поток хранит свои значения в **локальной** `ThreadLocalMap` (поле внутри самого объекта `Thread`)
- Ключи в этой мапе — сами объекты `ThreadLocal` (слабые ссылки)
- **Нет гонки за общий объект** — каждый поток работает только со своей мапой

Исходник `set()` (упрощённо)

```
public void set(T value) {  
    Thread t = Thread.currentThread();  
    ThreadLocalMap map = getMap(t);           // берём карту ТЕКУЩЕГО потока  
    if (map != null) {  
        map.set(this, value);                 // this = ключ ThreadLocal  
    } else {  
        createMap(t, value);                  // ленивое создание мапы  
    }  
}
```

```
}  
}
```

Кажется: «почему бы просто не писать в свою `Map` руками?» — `ThreadLocal` и есть эта обёртка: он сам находит мапу текущего потока, лениво её создаёт и прячет всю механику.

Утечки и практики

`ThreadLocal` сам по себе не «вечен»: значение живёт, пока жив поток. В **пулах потоков** (`Tomcat`, `ExecutorService`) потоки живут долго → значение останется в мапе навсегда, если его не убрать.

Правила

Правило	Почему
Всегда <code>remove()</code> в <code>finally</code> при пулах потоков	Иначе «мусор» останется в живом потоке → утечка памяти + следующая задача может прочитать чужое значение
Не хранить крупные объекты	Поток (и его мапа) живёт долго → память забита надолго
Инициализировать «как можно позже»	Чем позже <code>set()</code> , тем меньше время жизни значения
Не использовать статические <code>ThreadLocal</code> без очистки	Статика живёт вечно → риск накопления мусора в каждом потоке пула

Правильный шаблон

```
try {  
    context.set(buildContext());  
    // ... бизнес-логика  
} finally {  
    context.remove(); // гарантированная очистка даже при исключении  
}
```

Итоги

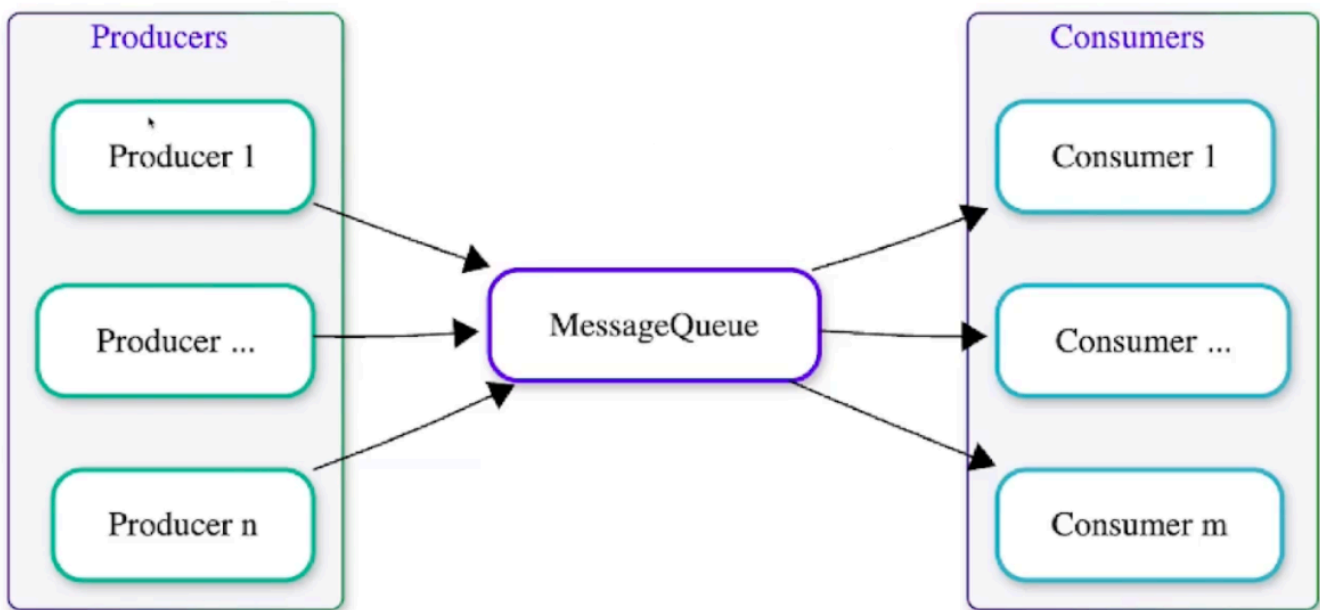
- `ThreadLocal` = «карман» потока: данные видит только тот поток, который их положил
 - Внутри: у каждого потока своя `ThreadLocalMap` → **ни блокировок, ни гонок**
 - Главные применения: контекст запроса/пользователя, транзакции, трейсинг
 - Главная опасность — **пулы потоков**: без `remove()` в `finally` → утечки и «протекание» данных между задачами
-

Глава 2.7. Пишем свою реализацию `ArrayBlockingQueue` (компактная версия)

Producer–Consumer (Производитель–Потребитель)

Темы: суть паттерна, блокирующая очередь на кольцевом буфере, `wait/notifyAll`, таймауты через `deadline`, `Poison Pill`

1. Суть паттерна



- **n производителей** кладут данные в очередь, **m потребителей** забирают
- Очередь **развязывает скорости**: быстрый производитель не «давит» медленного потребителя (и наоборот)
- Стороны **взаимодействуют только через очередь**, не напрямую

Классические кейсы

Где	Что происходит
Kafka / RabbitMQ	Брокер = очередь между сервисами
ExecutorService	Потоки-потребители берут задачи из <code>BlockingQueue</code>
Логирование	Потоки приложения → очередь → поток записи на диск

2. Реализация очереди: два подхода

Подход	Структура	Особенности
Linked list	<code>Node next</code> ссылки	Неограниченный размер, аллокации на каждый элемент
Массив-циклический буфер	<code>Object[]</code> + <code>head / tail</code>	Фикс. <code>capacity</code> , минимум аллокаций, блокирующая

Ниже — вторая: **циклический буфер** (как в `ArrayBlockingQueue`). Фиксированный `Object[]` + указатели `head / tail`, бегущие по кругу. Не linked list: меньше аллокаций, предсказуемая память.

3. `SimpleArrayBlockingQueue`

Поля и инварианты

```
public class SimpleArrayBlockingQueue<T> {
    private final Object[] array;
    private int size, head, tail;

    public SimpleArrayBlockingQueue(int capacity) {
        if (capacity <= 0) throw new IllegalArgumentException("capacity must
be > 0");
        this.array = new Object[capacity];
    }
}
```

`put / take`

```

public synchronized void put(T element) throws InterruptedException {
    Objects.requireNonNull(element);
    while (isFull()) wait();           // предикат в while — от ложных
пробуждений
    enqueue(element);
    notifyAll();                       // будим ждущих потребителей
}

public synchronized T take() throws InterruptedException {
    while (isEmpty()) wait();
    T element = dequeue();
    notifyAll();                       // будим ждущих производителей
    return element;
}

```

Кольцевой буфер

```

private void enqueue(T element) {
    array[tail] = element;
    tail = (tail + 1) % array.length;
    size++;
}

@SuppressWarnings("unchecked")
private T dequeue() {
    T element = (T) array[head];
    array[head] = null;               // помогаем GC
    head = (head + 1) % array.length;
    size--;
    return element;
}

```

Внутренние методы

```

public synchronized int getSize()    { return size; }
public synchronized boolean isEmpty() { return size == 0; }
public synchronized boolean isFull() { return size == array.length; }

```

4. Таймауты: ждём, но не вечно

Главный приём — `deadline`, а не фиксированный `wait(timeout)`

Ошибка новичка: `wait(100)` в цикле — каждый ложный выход из `wait` **продлевает общее ожидание** (спам `notifyAll` → ждём вечно). Правильно — считать **дедлайн один раз** и ждать **оставшееся время**:

```
private void await(BooleanSupplier condition, long timeoutMillis)
    throws InterruptedException, TimeoutException {
    long deadline = System.currentTimeMillis() + timeoutMillis;
    while (condition.getAsBoolean()) {
        long remaining = deadline - System.currentTimeMillis();
        if (remaining <= 0) throw new TimeoutException("timeout exceeded");
        wait(remaining);    // ровно оставшееся время
    }
}
```

Вся логика таймаута — один цикл. Проверка дедлайна ровно одна, корректность видна с первого взгляда.

Таймаутные версии `put/take` — делегируют в `await`

```
public synchronized void put(T element, long timeout, TimeUnit unit)
    throws InterruptedException, TimeoutException {
    Objects.requireNonNull(element);
    await(() -> isFull(), unit.toMillis(timeout));
    enqueue(element);
    notifyAll();
}

public synchronized T take(long timeout, TimeUnit unit)
    throws InterruptedException, TimeoutException {
    await(() -> isEmpty(), unit.toMillis(timeout));
    T element = dequeue();
    notifyAll();
    return element;
}
```

5. Демо: 2 производителя + 1 потребитель + Poison Pill

```
public static void main(String[] args) throws InterruptedException {
    var queue = new SimpleArrayBlockingQueue<Integer>(50);
    int poisonPill = Integer.MIN_VALUE;    // сигнал «завершайся»
```

```

Thread p1 = new Thread(() -> produce(queue, 1000), "producer-1");
Thread p2 = new Thread(() -> produce(queue, 1000), "producer-2");
Thread c1 = new Thread(() -> consume(queue, poisonPill), "consumer-1");

p1.start(); p2.start(); c1.start();

safeJoin(p1, p2);
queue.put(poisonPill); // все произвели → стоп-сигнал
safeJoin(c1);
}

```

```

static void produce(SimpleArrayBlockingQueue<Integer> queue, int count) {
    for (int i = 0; i < count; i++) {
        int element = i * i; // вычисляем ОДИН раз
        try {
            queue.put(element, 100, TimeUnit.MILLISECONDS);
            System.out.printf("[%s] pushed: %d\n",
Thread.currentThread().getName(), element);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return;
        } catch (TimeoutException e) {
            System.err.printf("[%s] timeout, dropped %d\n",
Thread.currentThread().getName(), element);
        }
        sleep(100);
    }
}

static void consume(SimpleArrayBlockingQueue<Integer> queue, int poisonPill)
{
    try {
        while (true) {
            int element = queue.take();
            if (element == poisonPill) break;
            System.out.printf("[%s] consumed: %d\n",
Thread.currentThread().getName(), element);
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}

```

6. Ключевые решения и почему они верны

Решение	Почему
<code>while (isFull()) / while (isEmpty())</code>	Предикат в цикле — защита от ложных пробуждений и «чужого» <code>notify</code>
<code>notifyAll()</code> вместо <code>notify()</code>	Ожидателей двух типов (производители и потребители); <code>notify()</code> может разбудить «не того»
Deadline вместо фикс. <code>wait(timeout)</code>	Спам <code>notifyAll</code> не продлевает ожидание бесконечно
<code>await(condition, millis)</code> — один метод	Таймаутная логика в одном месте, одна проверка дедлайна, нет дублирования
<code>array[head] = null</code> при изъятии	Помогаем GC, не держим ссылки на съеденные элементы
<code>(head + 1) % length</code>	Кольцевой буфер: переиспользуем массив без сдвигов
Poison Pill	Стандартный способ мягко остановить потребителя без <code>interrupt</code>
Нельзя <code>null</code> в <code>put</code>	<code>null</code> — зарезервирован (как в <code>ArrayBlockingQueue</code> : сигнал «пусто»)

7. Итоги

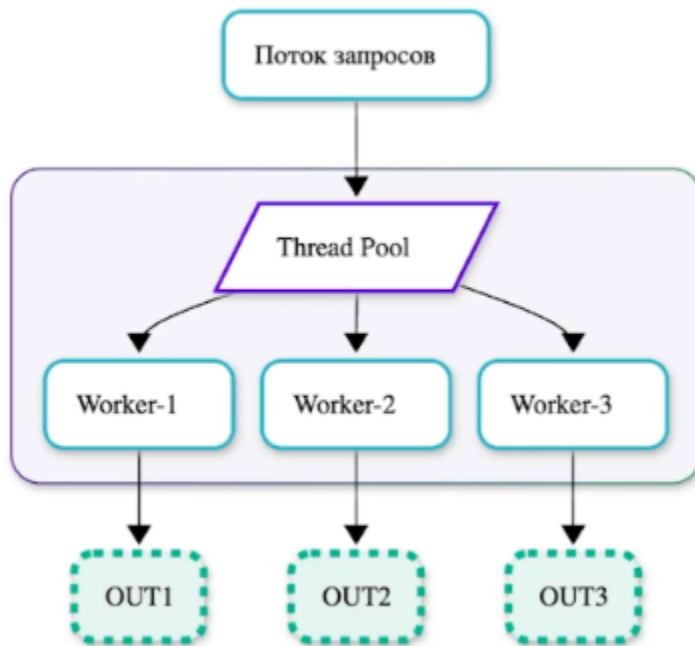
- **Producer–Consumer** — базовый паттерн декомпозиции: стороны общаются только через очередь
- Блокирующая очередь = циклический буфер + `synchronized` + `wait/notifyAll`
- `while` + предикат + `notifyAll` — три кита корректной сигнализации
- Таймауты делаем через **дедлайн**, Poison Pill — для корректной остановки
- В проде берите готовое: `ArrayBlockingQueue` , `LinkedBlockingQueue` , `ConcurrentLinkedQueue`

3. Пулы потоков, конкурентные коллекции и виртуальные потоки

Глава 3.1. Пулы потоков и Fork Join выбор и практика

Темы: зачем нужен пул потоков, IO-bound vs CPU-bound, `ExecutorService` и его жизненный цикл, `ThreadPoolExecutor` изнутри, `ForkJoinPool` и work-stealing, `RecursiveTask` , `parallel streams`

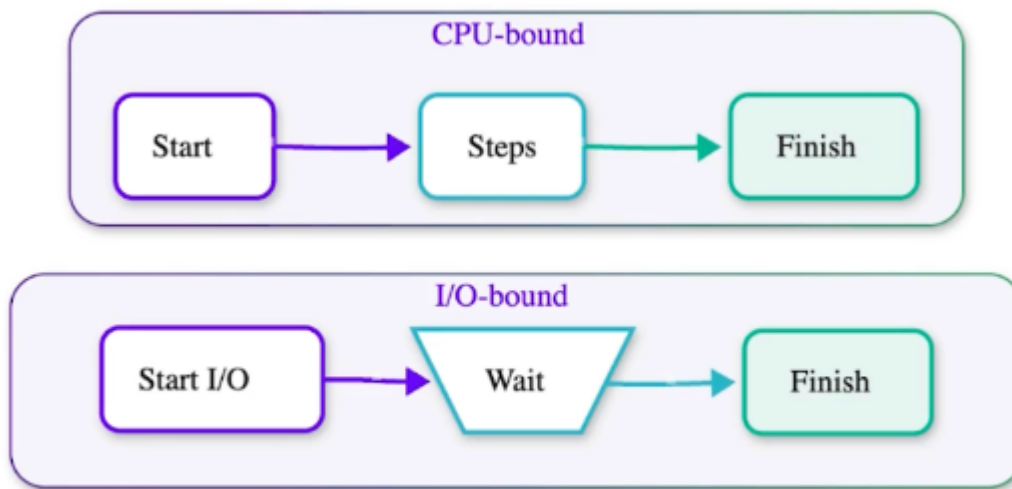
Что такое пул потоков и зачем



- **Пул потоков = «завод рабочих потоков».** Вместо того, чтобы создавать тысячи `new Thread()`, задачи кладутся в очередь и выполняются уже готовыми потоками
- **Снижение накладных расходов:** создание/уничтожение потока — дорого (память на стек, системные вызовы, context-switch). Пул переиспользует потоки
- Дополнительно пул даёт **контроль над количеством параллельных задач** — очередь блокирующая, лишнее ждёт, а не взрывает систему

IO-bound vs CPU-bound — от этого зависит размер пула

Тип	Время уходит на	Пул
IO-bound	Ожидание внешних операций (сеть, диск, БД)	Можно больше потоков, чем ядер
CPU-bound	Вычисления, ограничены ядрами	≈ количеству ядер (больше — только context-switch)



Правило: для CPU-bound нет смысла создавать больше потоков, чем ядер. Для IO-bound — можно сильно больше, т.к. потоки большую часть времени ждут (например, в бэкенде в 99% случаев это IO).

ExecutorService: создание и демо

Типы пулов из Executors

Фабрика	Что внутри	Когда
<code>newFixedThreadPool(n)</code>	Ровно n потоков, бесконечная очередь	Известная нагрузка, back-pressure через размер
<code>newCachedThreadPool()</code>	Потоки создаются по требованию, умирают при простое (60 с)	Много коротких асинхронных задач, но осторожно — может разрастись
<code>newSingleThreadExecutor()</code>	Один поток, FIFO	Строгая упорядоченность задач
<code>newScheduledThreadPool(n)</code>	Пул + расписание (<code>schedule</code> , <code>scheduleAtFixedRate</code> , <code>scheduleWithFixedDelay</code>)	Периодические задачи, таймеры

Демо

`submit()` возвращает `Future` — механизм получения результата из потока пула:

```
ExecutorService fixed = Executors.newFixedThreadPool(4);
try {
    Future<Integer> future = fixed.submit(() -> 42);
    // ...
    log(future.get()); // блокируемся до результата
} finally {
    fixed.shutdown();
    try {
        fixed.awaitTermination(5, TimeUnit.SECONDS);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}
```

fixed vs cached

Один и тот же эксперимент, меняется только пул. 100 IO-задач (`ExternalCallTask`, каждая спит/ждёт ~секунду):

В цикле мы намеренно не сохраняем возвращаемый `submit()` 'ом `Future` — результат задачи нам не нужен, и это нормально: задача выполнится в любом случае, пулу безразлично, держим мы ссылку или нет.

Справка: `Future` нужен, только если хочешь результат (`get()`), отмену (`cancel()`) или статус (`isDone()`). Но учти подводный камень: если задача кинет исключение, а `Future` игнорирован — ошибка проглотится молча, она так и останется спрятанной внутри `Future`. Поэтому либо обрабатывай ошибки внутри самой задачи, либо сохраняй `Future` и вызывай `get()`.

```
// ФИКСИРОВАННЫЙ пул: потоков меньше, чем задач — часть ждёт в очереди
long start = System.currentTimeMillis();
try (ExecutorService executorService = Executors.newFixedThreadPool(10)) {
    for (int i = 0; i < 100; i++) {
        executorService.submit(new ExternalCallTask(i));
    }
} // close() ≈ shutdown() + awaitTermination
log("Processed time: %sms", System.currentTimeMillis() - start);
// ~10 сек: 10 потоков × 10 волн задач
```

```
// CACHED пул: под каждую задачу поднимается свой поток
long start = System.currentTimeMillis();
```

```
try (ExecutorService executorService = Executors.newCachedThreadPool()) {
    for (int i = 0; i < 100; i++) {
        executorService.submit(new ExternalCallTask(i));
    }
}
log("Processed time: %sms", System.currentTimeMillis() - start);
// ~1 сек: все 100 задач ждут одновременно
```

Вывод демо: для IO-bound задач cached выигрывает ~в 10 раз, потому что потоки там ждут, а не считают — их можно много. Но плата за это: создание 100 потоков (накладные расходы) и риск взрыва пула, если задач прилетит слишком много. Fixed даёт предсказуемость и back-pressure.

Как правильно закрывать executor

Главный паттерн — **shutdown** → **awaitTermination** → **shutdownNow**:

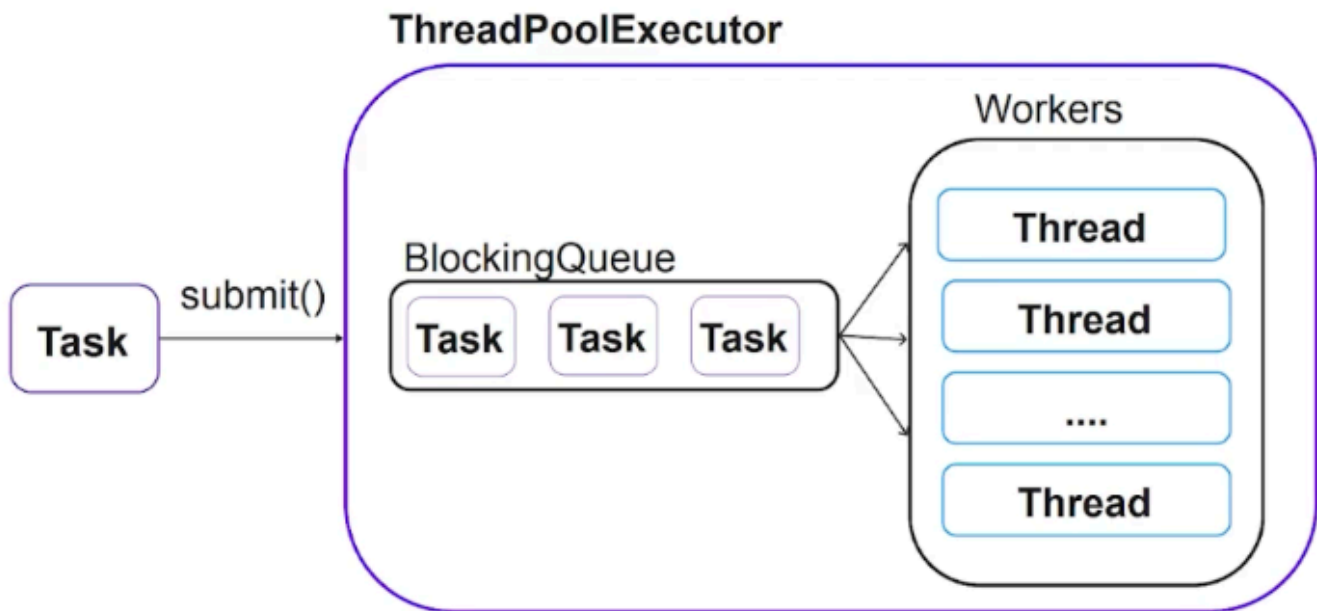
```
ExecutorService pool = Executors.newFixedThreadPool(4);
try {
    // do work ...
} finally {
    pool.shutdown(); // мягко: не принимает новые,
    // ждёт текущие
    try {
        if (!pool.awaitTermination(5, TimeUnit.SECONDS)) {
            pool.shutdownNow(); // жёстко: interrupt всем рабочим
            // потокам
        }
    } catch (InterruptedException e) {
        pool.shutdownNow();
        Thread.currentThread().interrupt(); // восстанавливаем флаг!
    }
}
```

Важно: `shutdownNow()` посылает `interrupt`, поэтому код задач должен **уважать флаг прерывания** (не глотать `InterruptedException` молча, выходить из циклов). Иначе потоки не завершатся вообще.

Разница методов:

Метод	Что делает
<code>shutdown()</code>	Новые задачи не принимает, текущие дорабатывают
<code>awaitTermination(t, u)</code>	Ждёт завершения <code>t</code> времени, вернёт <code>true</code> , если успели
<code>shutdownNow()</code>	<code>interrupt</code> всем потокам, возвращает невыполненные задачи
<code>close()</code> (try-with-resources)	<code>shutdown()</code> + <code>awaitTermination</code> , под капотом

ThreadPoolExecutor: внутри



- Клиент через `submit()` кладёт задачу в **общую** `BlockingQueue`
- **Workers** (`Thread`) в цикле берут задачи из очереди, выполняют и возвращаются в `idle` (`take()` блокирует их, не жгут CPU)
- Это и есть наш паттерн **Producer–Consumer** из главы **G**: клиенты — производители, workers — потребители

Да, этот код стоит вставить — `scheduled`-пул пока у тебя в главе есть только в таблице, без демо. Вот сокращённая версия (убрал повторы схожих задач, оставил суть) + короткое пояснение, чтобы блок читался как конспект

Демо: `ScheduledThreadPool` — задачи в будущем по расписанию

```

public class ScheduledAtFixedRateVsDelayDemo {
    public static void main(String[] args) {
        long start = System.currentTimeMillis();
        try (ScheduledExecutorService executorService =
            Executors.newScheduledThreadPool(8)) {

            var task1 = new ExternalCallTask(1);
            var task2 = new ExternalCallTask(2);
            var task3 = new ExternalCallTask(3);

            // одноразовый запуск с задержкой — выполнятся по порядку delay
            executorService.schedule(task1, 30, TimeUnit.SECONDS);
            executorService.schedule(task2, 10, TimeUnit.SECONDS);
            executorService.schedule(task3, 20, TimeUnit.SECONDS);
            // порядок старта: task2 → task3 → task1, несмотря на порядок
            schedule()

            // периодический запуск: каждые 1 сек ОТ СТАРТА предыдущего
            // executorService.scheduleAtFixedRate(task1, 1, 1,
            TimeUnit.SECONDS);
            // ThreadSleepUtil.safeSleepWithoutThrow(10_000);

        } finally {
            long duration = System.currentTimeMillis() - start;
            Logger.log("Processed time: %sms", duration);
        }
    }
}

```

Три метода — три смысла:

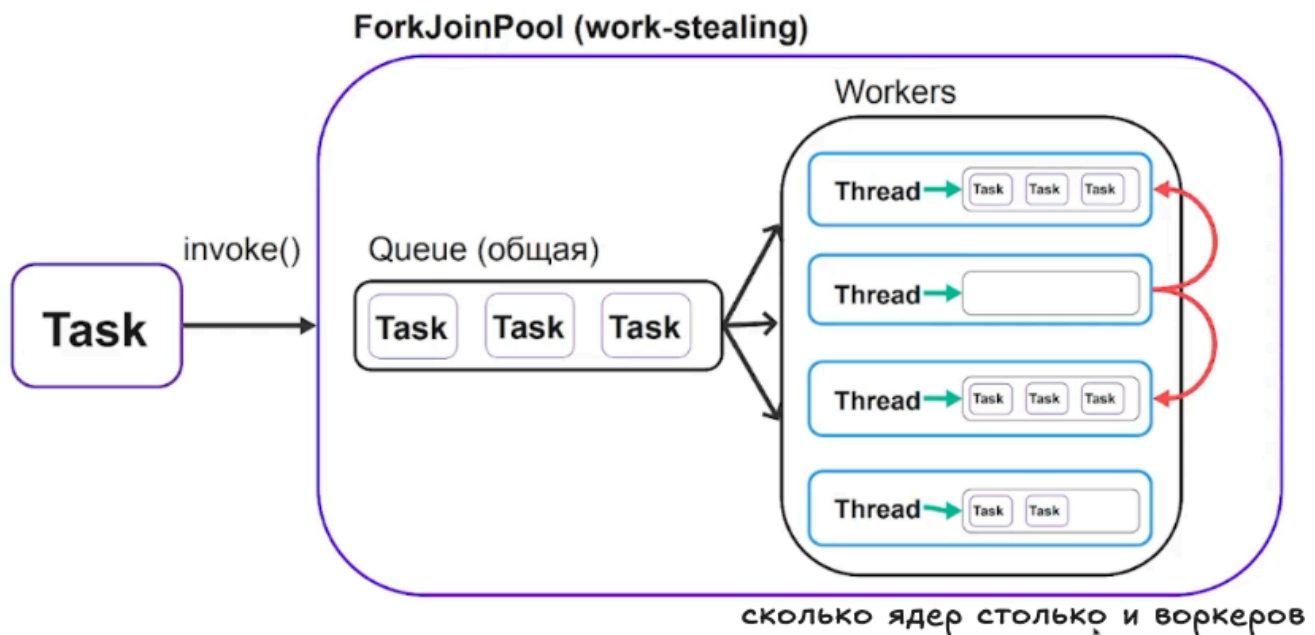
Метод	Когда запускается
<code>schedule(task, delay, unit)</code>	Один раз, через <code>delay</code>
<code>scheduleAtFixedRate(task, init, period, unit)</code>	Каждые <code>period</code> от начала предыдущего запуска . Если задача длиннее <code>period</code> — запуски «съезжаются», вплоть до параллельных
<code>scheduleWithFixedDelay(task, init, delay, unit)</code>	Каждый раз через <code>delay</code> после завершения предыдущего — гарантия паузы между запусками

Справка: у всех трёх методов возвращаемый `ScheduledFuture` можно не ловить — он нужен только для отмены (`cancel()`) или если хочешь узнать, когда задача стартанёт.

Итоги по `ThreadPoolExecutor`

- Клиент подаёт задачи (`Runnable` / `Callable`) в общую очередь
- Workers берут задачи, выполняют и возвращаются в idle
- Жизненный цикл: `shutdown()` → `awaitTermination()`
- **Не блокируйте главный поток на `Future.get()` без нужды**
- Динамическое наращивание потоков «до небес» (cached) — потери на context-switch

ForkJoinPool: work-stealing



- Каждый воркер имеет **свою деку (deque) задач** (LIFO — свои подзадачи берём с головы)
- Свободные воркеры могут **воровать задачи из хвостов** чужих дек (FIFO) — так балансируется нагрузка
- Воркеров ≈ количеству ядер (по умолчанию `availableProcessors - 1`)
- Используется для **рекурсивных и параллельных вычислений** (CPU-bound)
- **Основа для `parallelStream()` и `CompletableFuture.supplyAsync()` по умолчанию** — это `ForkJoinPool.commonPool()`

⚠ **CommonForkJoinPool** — общий пул на всю JVM (размер = ядра - 1), через него по умолчанию работают `parallelStream` и `CompletableFuture.supplyAsync`. Так как он

разделяемый, тяжёлая задача займёт все его воркеры и встанут чужие задачи — латентность всплывёт в другом месте. Поэтому в серверном коде используй свой пул, а `commonPool` — только для коротких CPU-вычислений.

? Зачем FJP, если есть ThreadPool с потоками = ядрам?

Потому что рекурсивная задача **ждёт результаты своих подзадач** (`join()`). В обычном пуле поток в этот момент блокируется и ядро простаивает; добавлять потоков нельзя — `context-switch` убьёт CPU-bound. В FJP воркер вместо ожидания **берёт другую задачу** (из головы своей деки или ворует с хвоста чужой) — так все ядра заняты без лишних потоков. Плюс мелкие подзадачи ложатся в локальную деку без гонки за общей очередью. «Просто пул = ядрам» работает, только если задачи независимы и не ждут друг друга — а `divide-and-conquer` как раз ждёт.

RecursiveTask: сумма массива

Классика ForkJoin: задача, которая **форкается (fork)**, пока не станет **маленькой, потом считается в лоб**.

```
static class SumTask extends RecursiveTask<Long> {
    private static final int THRESHOLD = 10_000; // порог «мелкости»
    private final int[] arr;
    private final int start, end;

    @Override
    protected Long compute() {
        int len = end - start;
        if (len <= THRESHOLD) { // база рекурсии –
            считаем в лоб
                long sum = 0;
                for (int i = start; i < end; i++) sum += arr[i];
                return sum;
        }

        int mid = start + len / 2;
        SumTask left = new SumTask(arr, start, mid);
        SumTask right = new SumTask(arr, mid, end);
        left.fork(); // левую – асинхронно в
        пул
        long r = right.compute(); // правую – считаем сами
        (не зря сидим)
        long l = left.join(); // ожидаем левую
```

```

        return l + r;
    }
}

ForkJoinPool pool = ForkJoinPool.commonPool();
long sum = pool.invoke(new SumTask(arr, 0, arr.length));

```

Метод	Смысл
<code>fork()</code>	Отправить подзадачу в пул, вернуться сразу
<code>compute()</code>	Выполнить подзадачу в текущем потоке
<code>join()</code>	Дождаться результата <code>fork()</code> -нутой задачи
<code>invoke()</code>	Отправить корневую задачу и дождаться результата

Почему `left.fork() + right.compute()`, а не две `fork()`? Если обе форкнуть, текущий воркер пойдёт воровать чужие задачи вместо того, чтобы сразу заняться своей правой половиной. Один `fork` + один `compute` — стандартный оптимальный паттерн деления.

- `RecursiveTask<V>` — возвращает результат. Есть близнец `RecursiveAction` — для задач без результата (например, сортировка/обход)
- `ForkJoinTask` нельзя переиспользовать: один экземпляр = один запуск

Parallel Streams

```

long s = IntStream.rangeClosed(1, 1_000_000)
    .parallel()
    .mapToLong(i -> i)
    .sum();

```

Параллельный стрим сам разбивает данные на сегменты, раздаёт воркерам `common FJP` и склеивает результат — по сути, `ForkJoin` «из коробки».

Стоит, но сократить — вторая цепочка дублирует смысл первой (тот же эффект, просто другие операции). В конспект достаточно одной. Ценность демо: наглядно видно **недетерминированный порядок** выполнения воркерами `common-пула`. Вот сжатая версия:

Демо: parallel stream на common ForkJoinPool

```

/**
 * Параллельные стримы используют общий ForkJoinPool и его параллелизм.
 */
public class ParallelStreamsCommonPoolDemo {
    public static void main(String[] args) {
        // 1..20 → квадрат → сумма, параллельно на Common FJP
        int sum = IntStream.rangeClosed(1, 20)
            .parallel()
            .peek(i -> log("peek %s", i)) //
порядок НЕ гарантирован!
            .map(i -> {
                ThreadSleepUtil.safeSleepWithoutThrow(40); //
имитация CPU-работы
                int v = i * i;
                log("map %s -> %s", i, v);
                return v;
            })
            .sum();
        log("Calculated sum=%d", sum); //
результат тот же = 2870
    }
}

```

Что видно в логах: элементы обрабатываются «вразнобой» разными воркерами (`map 7` может напечататься раньше `map 2`), но итоговая сумма всегда корректна — `parallel stream` сам разбивает диапазон на сегменты, считает в воркерах `common`-пула и склеивает результат. Детерминирован только результат, не порядок.

Что помнить

- Работают поверх **общего ForkJoinPool (common)** — занимают общий ресурс
- Хороши для **чистых CPU-задач без блокировок** и с крупными данными
- **Не использовать для IO-bound** — можно «съесть» общий пул и ухудшить латентность всех остальных
- Верхняя рамка параллелизма — `Runtime.getRuntime().availableProcessors()`
- Порядок в `peek / forEach` становится недетерминированным (демо с логами это показывает наглядно)

Итог: ThreadPool vs ForkJoinPool

	ThreadPoolExecutor	ForkJoinPool
Очередь	одна общая	deque у каждого воркера + воровство
Задачи	независимые	рекурсивные, делимые (fork/join)
Нагрузка	IO-bound + CPU-bound	CPU-bound
Типичное применение	веб-сервер, задачи извне	parallelStream, divide & conquer

Чек-лист:

- Выбор пула начинается с профиля задачи: **IO-bound vs CPU-bound**
- ThreadPoolExecutor — общая очередь; ForkJoinPool — work-stealing
- Parallel Streams используют common FJP — осторожно в серверных сценариях
- Минимум блокировок, чёткое завершение пула (shutdown → awaitTermination)

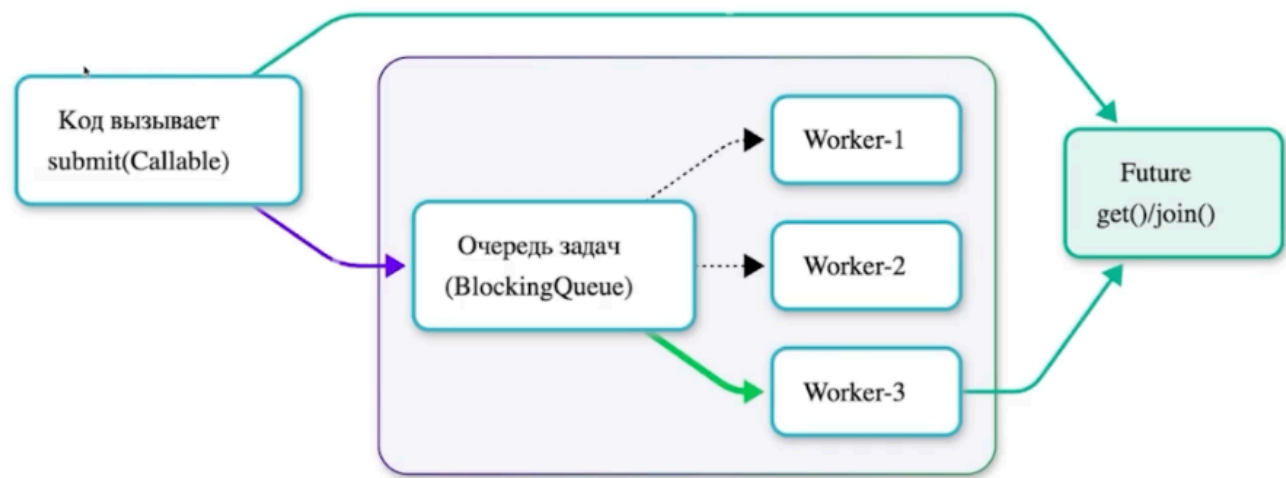
Глава 3.2. Асинхронность от Future к CompletableFuture

Темы: Callable vs Runnable , Future<T> и его жизненный цикл, CompletableFuture — цепочки, композиция, обработка ошибок, allOf/anyOf

1. Callable и Future

- **Callable<T>** — как Runnable , но **возвращает результат** и может кидать checked-исключения
- **Future<T>** — интерфейс, представляющий **результат асинхронной операции**. Это «ручка» на задачу, которая выполняется в фоне
- Future возвращается, когда задача **уже запланирована в пул или запущена** — не ждём результата на этом месте
- Получение результата — через **блокирующий get()**

Поток выполнения:



Демо: fixed pool + Future

```
ExecutorService executor = Executors.newFixedThreadPool(4);
List<Future<String>> futures = new ArrayList<>();

for (int i = 0; i < 10; i++) {
    futures.add(executor.submit(() -> {
        Thread.sleep(200);
        return "done by " + Thread.currentThread().getName();
    }));
}

for (Future<String> f : futures) {
    System.out.println(f.get()); // блокируемся по очереди на каждой
}
executor.shutdown();
```

Справка: цикл `f.get()` по списку ждёт результаты **по порядку** — если первая задача самая долгая, мы простоим, хотя остальные уже готовы. Для «какая готова — ту и берём» есть `ExecutorCompletionService`.

Жизненный цикл Future

Состояние	Как наступает	Что вернёт <code>get()</code>
Создан	<code>ExecutorService.submit(...)</code> вернул <code>Future</code> (задача в очереди)	блокируется
Выполняется	воркер взял задачу <code>Callable/Runnable</code>	блокируется

Состояние	Как наступает	Что вернёт <code>get()</code>
Завершён нормально	<code>isDone() == true</code>	результат <code>T</code>
Завершён с ошибкой	<code>isDone() == true</code>	бросит <code>ExecutionException</code> (cause внутри)
Отменён	<code>cancel(...) → isCancelled() == true</code>	бросит <code>CancellationException</code>

⚠ Бросок исключения из задачи **не убивает пул** — ошибка «прячется» в `Future` и вылетит только в `get()`. Если `Future` выброшен на пол — ошибка проглотится молча.

2. `CompletableFuture`

`CompletableFuture<T>` — реализует `Future` и `CompletionStage`: позволяет строить **неблокирующие цепочки** действий, композицию асинхронных задач и объединение результатов. Главное отличие от обычного `Future`: **мы не ждём результат, а описываем, что с ним сделать**.

```
CompletableFuture<Integer> compFuture = CompletableFuture
    .supplyAsync(() -> {
        // do work
    })
    .thenApply(x -> x * x);

compFuture.get();
```

Демо: базовая цепочка

```
CompletableFuture<Void> cf = CompletableFuture.supplyAsync(
    CompletableFutureBasicsDemo::doSomeWork // асинхронно в
commonPool
)
    .thenApply(x -> x * 2) // преобразуем
результат
    .thenAccept(r -> log("result=%d", r)) // потребляем,
ничего не возвращаем
    .exceptionally(ex -> { // ловим ошибку
        ЛЮБОЙ стадии выше
        log("error: %s", ex);
    });
```

```

        return null;
    });

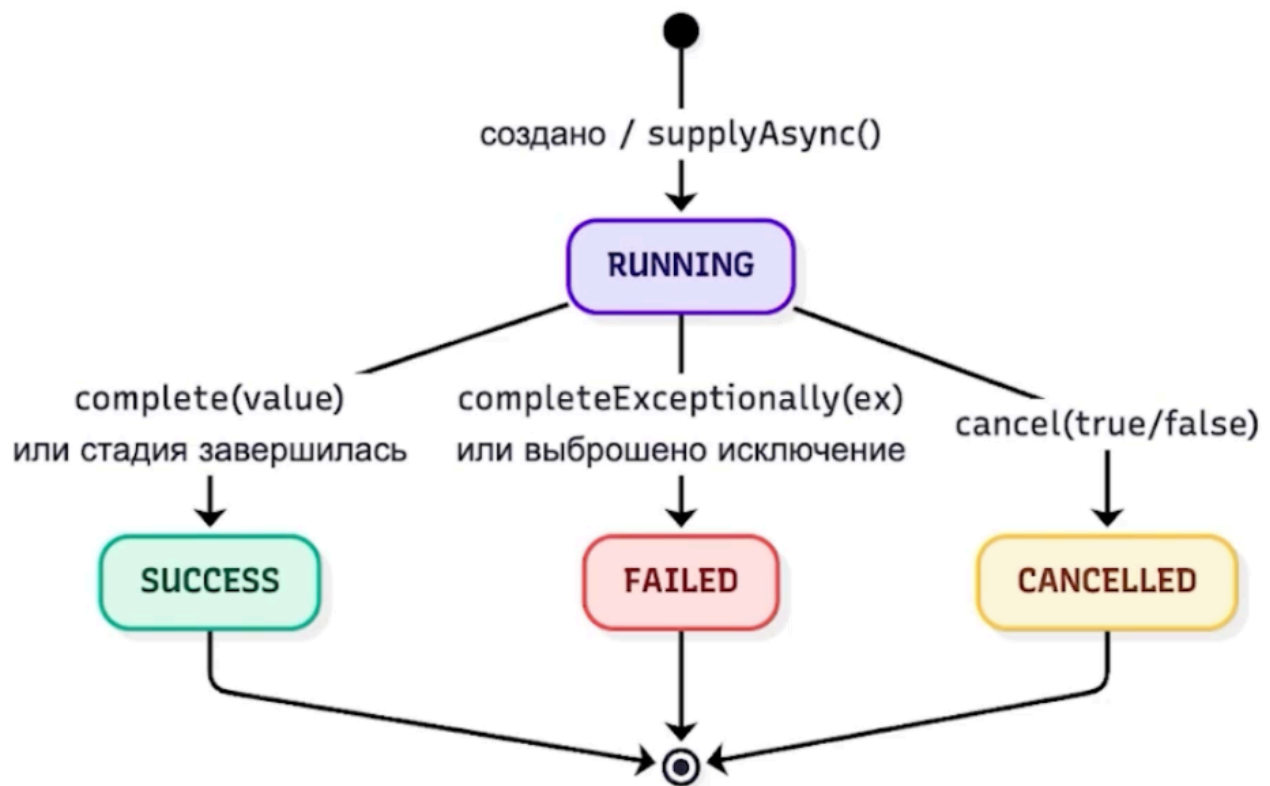
cf.join();
log("done");

private static int doSomeWork() {
    log("started working");
    safeSleepWithoutThrow(50);
    throw new RuntimeException("test exception");    // ← цепочка пойдёт в
exceptionally
}

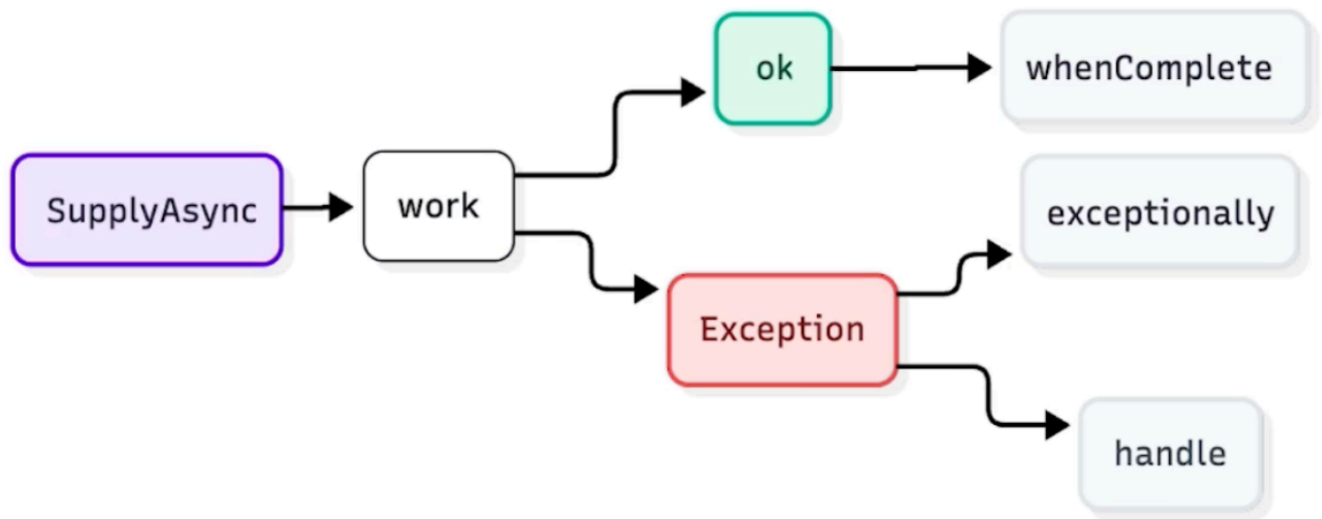
```

Важно: исключение на любой стадии «протекает» вниз по цепочке, пропуская все `then*`, пока не встретит `exceptionally / handle`. Поэтому один `exceptionally` в конце ловит всё.

Жизненный цикл CF



Ветка	Куда идёт
ok	<code>whenComplete</code>
Exception	<code>exceptionally</code> , <code>handle</code>



3. Операторы CompletableFuture

Оператор	Sync/Async	Назначение	Когда применять
<code>thenApply</code>	sync/async	преобразует результат	преобразовать $R \rightarrow T$
<code>thenCompose</code>	sync/async	разворачивает внутренний CF	когда следующая операция тоже асинхронная
<code>thenCombine</code>	sync/async	слияние двух CF	два независимых CF нужно слить
<code>allOf / anyOf</code>	async	ждёт всех / первого	fan-out/fan-in сценарии
<code>exceptionally</code>	sync	обрабатывает ошибку	вернуть дефолт/фоллбек
<code>handle</code>	sync/async	обрабатывает результат + ошибку	логирование/обёртка
<code>whenComplete</code>	sync/async	побочный эффект, ничего не меняет	финализация, логирование

Ключевое различие:

- `thenApply(f)` : `f` возвращает **значение** $\rightarrow$ `CompletableFuture<T>`
- `thenCompose(f)` : `f` возвращает **CompletableFuture** $\rightarrow$ сшивает две цепочки в одну (иначе получим `CF<CF<T>>`)

`thenCompose` vs `thenCombine`

`thenCombine` сливает **два независимых** `CompletableFuture`: ждёт, пока оба завершатся, и скармливает оба результата в би-функцию. Главное — оба `Future` должны быть **уже запущены**, тогда вызовы шли параллельно и мы просто ждём обоих.

`thenCompose` — для **последовательной** зависимости: функция получает результат первой задачи и возвращает **новый** `CompletableFuture`, который сшивается с цепочкой (по сути, `flatMap` для асинхронных цепочек). Если вместо `compose` использовать `apply` и вернуть `Future` — получишь `CF<CF<T>>`, `compose` это разворачивает.

```
// thenCombine: два НЕЗАВИСИМЫХ вызова идут ПАРАЛЛЕЛЬНО, объединяем
результаты
var orderFuture = simulateExternalCallAsync("load-order", "order-234");
var userFuture  = simulateExternalCallAsync("load-user", "Vasya");

String combined = orderFuture
    .thenCombine(userFuture, String::concat) // ждёт оба, склеивает
    .join();

// thenCompose: ПОСЛЕДОВАТЕЛЬНАЯ зависимость — второй вызов зависит от
первого
String composed = simulateExternalCallAsync("load-user", "Vasya")
    .thenCompose(user -> simulateExternalCallAsync("load-order", "order-
" + user))
    .join();
```

Правило выбора: **независимые задачи** → запускаем обе сразу + `thenCombine`; **вторая зависит от первой** → `thenCompose`. Ошибка — делать последовательные вызовы через `thenCombine`: потеряешь параллелизм.

обе готовы → `thenAcceptBoth` / `runAfterBoth`

Это «потребляющие» родственники `thenCombine`: также ждут оба `Future`, но **ничего не возвращают** (результат — `CF<Void>`).

Разница между собой: `thenAcceptBoth` получает **оба результата** как аргументы — нужны значения, но новое не строим; `runAfterBoth` не получает ничего — просто триггер «оба завершились», когда результаты неважны.

```
userFuture.thenAcceptBoth(orderFuture,
    (user, orders) -> log("user=%s, orders=%s", user, orders))
    .join(); // нужны оба результата

cf1.runAfterBoth(cf2, () -> log("both tasks completed")).join(); //
результаты не нужны
```

allOf / anyOf

Оба статические: принимают **несколько** `CompletableFuture` сразу. `allOf` ждёт завершения **всех** и возвращает `CF<Void>` — сам результат не несёт, поэтому значения потом забирают `join()` 'ом у каждого из входных. `anyOf` ждёт **первого** завершившегося и возвращает его результат как `Object` (тип стирается). Классические сценарии: `allOf` — «запросил 5 сервисов, соберу ответ когда все ответят» (fan-out/fan-in), `anyOf` — «запросил два зеркала, беру кто быстрее».

```
var cf1 = simulateExternalCallAsync("task-1", 42);
var cf2 = simulateExternalCallAsync("task-2", -42);
var cf3 = simulateExternalCallAsync("task-3", 100);

CompletableFuture.allOf(cf1, cf2, cf3).join();           // ждём ВСЕ
log("allOf done -> %s,%s,%s", cf1.join(), cf2.join(), cf3.join());
// ⚠ allOf возвращает CF<Void> – результаты забираем join()'ом у каждого

Object winner = CompletableFuture.anyOf(cf1, cf2, cf3).join(); // ждём
ЛЮБОГО
log("anyOf winner=%s", winner);
```

handle — результат и ошибка в одном месте

Все три перехватывают ошибку на стадии, но по-разному. `exceptionally` срабатывает **только при ошибке**: результат не видит, зато может вернуть дефолт и «вылечить» цепочку. `whenComplete` — чистый побочный эффект: видит и результат, и ошибку, но **ничего не может вернуть**, цепочка идёт дальше как есть. `handle` — комбо: видит и то, и другое через `(T, Throwable)` и **может вернуть значение** в обеих ветках — единственный, кто полноценно обрабатывает оба исхода и заменяет `exceptionally+then` в одну стадию.

```
ExecutorService pool = Executors.newFixedThreadPool(100);

var future = CompletableFuture.supplyAsync(() -> {
    log("start risky call");
    if (Math.random() > 0.5) throw new RuntimeException("network
error");
    return 42;
})
    .thenApplyAsync(x -> x * 2)           // async-версия: стадия
тоже уйдёт в пул
    .handle((result, ex) -> {           // (T, Throwable) –
видим И то, И другое
```

```

        if (ex != null) {
            log("fallback: %s", ex.getMessage());
            return 0; // дефолт при ошибке
        }
        return result + 10;
    })
    .thenAcceptAsync(r -> log("result=%d", r), pool); // свой пул для
конкретной стадии

future.join();

```

Обработчик	Видит результат	Видит ошибку	Может вернуть другое значение
exceptionally	нет	да	да (только ветка ошибки)
handle	да	да	да
whenComplete	да	да	нет — побочный эффект

4. Синхронные vs асинхронные версии

У всех операторов есть пара: `thenApply` / `thenApplyAsync`. Разница — **в каком потоке выполнится стадия**:

- `thenApply(f)` — выполнится **в том же потоке**, что завершил предыдущую стадию (или в потоке, который вызвал `complete()`)
- `thenApplyAsync(f)` — уйдёт в **пул** (`commonPool` или переданный)

Когда важно: тяжёлая стадия после лёгкой — бери `async`, чтобы не занимать чужой воркер; лёгкая стадия (лог, преобразование) — `sync`, без переключения потоков.

Советы и итоги

- **Обрабатывай исключения:** `exceptionally` / `handle` — иначе ошибка «протечёт» до конца цепочки и вылетит в `join()` обёрнутой в `CompletionException`
- **Таймауты и fallback — для внешних вызовов:** `completeOnTimeout(default, t, unit)` / `orTimeout(t, unit)` (Java 9+)
- **Потеря исключений при `join()` без `handle`** — частый фейл: цепочка падает, а в логах тишина

- **Блокирующий `get / join` в цепочках убивает асинхронность:** если внутри `thenApply` делаешь `otherFuture.join()` — это тот же `thenCombine`, но с блокировкой воркера
- **Всегда передавай свой пул для IO-задач:** `supplyAsync(task, pool)` — иначе задача уйдёт в общий `commonForkJoinPool` (см. главу H)

Итог одной фразой: `Future` — «ручка» с блокирующим `get()`; `CompletableFuture` — способ описать, что делать с результатом, когда он появится, без блокировок, с ветвлением на ошибки и композицией нескольких асинхронных вызовов.

Что добавил от себя: справка про `ExecutorCompletionService` (порядок `get` по списку), предупреждение про «молчаливую» ошибку в цепочке, `orTimeout / completeOnTimeout`, раздел `sync vs async` версии, таблицу сравнения `exceptionally / handle / whenComplete`, правило выбора `compose vs combine` одной строкой, замечание про `join()` внутри `thenApply`.

Глава 3.3. Concurrent Collections практика и устройство

Темы: почему обычные коллекции опасны, `CopyOnWriteArrayList`, `ConcurrentHashMap`, `BlockingQueue`, как выбирать коллекцию под нагрузку

1. Проблематика

- Обычные коллекции (`HashMap`, `ArrayList`) **не потокобезопасны** → гонки, потеря данных, `ConcurrentModificationException`
- Concurrent-коллекции дают **масштабируемость вместо «глобальной блокировки»:** сокращают время ожидания между потоками и повышают параллелизм
- **Предсказуемость:** операции имеют чёткие гарантии видимости и атомарности

Что ломается без синхронизации

`ArrayList`: конкурентные `add()` пишут в общий массив `elementData` → при расширении массива иногда даже `ArrayIndexOutOfBoundsException`, размер уезжает.

```
private static final List<Integer> list = new ArrayList<>(); // НЕ
потокобезопасно
// ... 2 потока × 100_000 add()
log("Expected size: %d, actual: %d", expected, list.size()); // actual <
expected
```

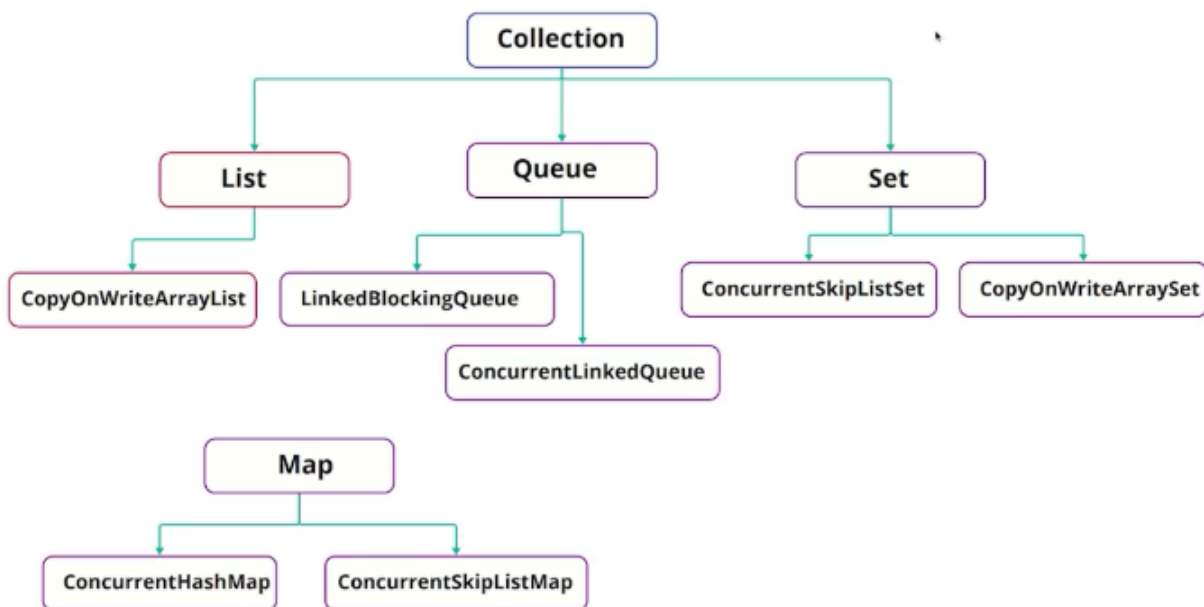
HashMap: при конкурентной записи в одну корзину теряются узлы → `size()` меньше ожидаемого, в худшем случае — бесконечный цикл при `resize` (в старых JDK).

```
private static final Map<Integer, String> map = new HashMap<>(); // НЕ  
потокобезопасно  
// ... 2 потока × 50_000 put()  
log("Expected hashmap size: %d, actual: %d", expected, map.size());
```

**«Костыли» вроде `Collections.synchronizedList/Map` работают, но это глобальная блокировка на ВСЮ коллекцию — параллелизм умирает (см. бенчмарк ниже).

2. Карта concurrent-коллекций

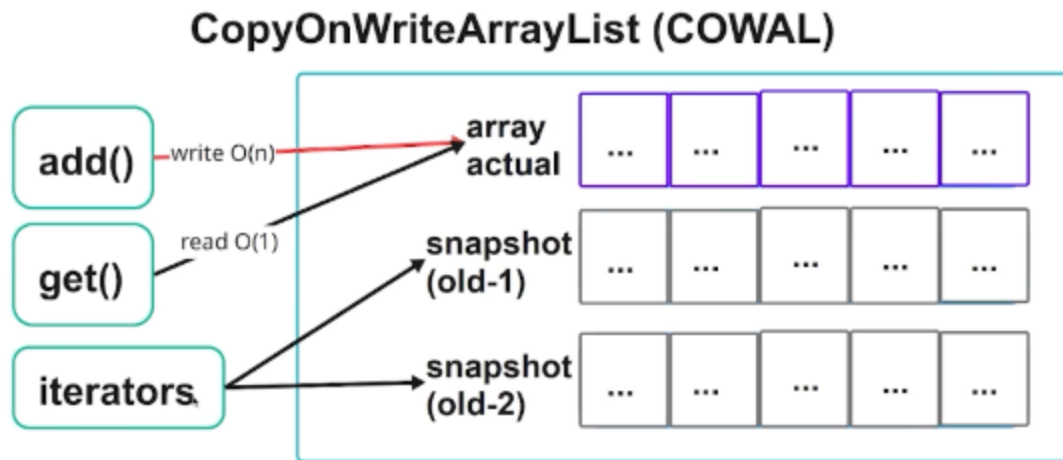
Concurrent collections



3. CopyOnWriteArrayList (COWAL)

Идея: каждая запись — это **полная копия массива**. `add()` копирует массив, вставляет элемент и **атомарно подменяет ссылку** на новый массив. Читатели всегда работают со снимком (snapshot), который уже никогда не изменится.

Цена: запись $O(n)$ + аллокация нового массива при КАЖДОМ изменении. Выгода: чтение $O(1)$ **без блокировок вообще**, итератор ходит по снимку и не бросает `ConcurrentModificationException`.



Профиль: «много читают / мало пишут» — может быть примерно от 100:1 до 1000:1. Слушатели событий, конфиги, справочники.

```
CopyOnWriteArrayList<Integer> list = new CopyOnWriteArrayList<>
(intialMillionItems);

measure("20 readers × 1_000_000 reads", () -> /* все читают параллельно, без
единой блокировки */);
measure("20 writers × 10 writes", () -> /* каждый add = копия миллионного
массива */);
```

Справка: если писать будут часто, COWAL превратится в генератор мусора — каждая запись создаёт новый массив, а старые снапшоты держат итераторы, пока GC их не уберёт.

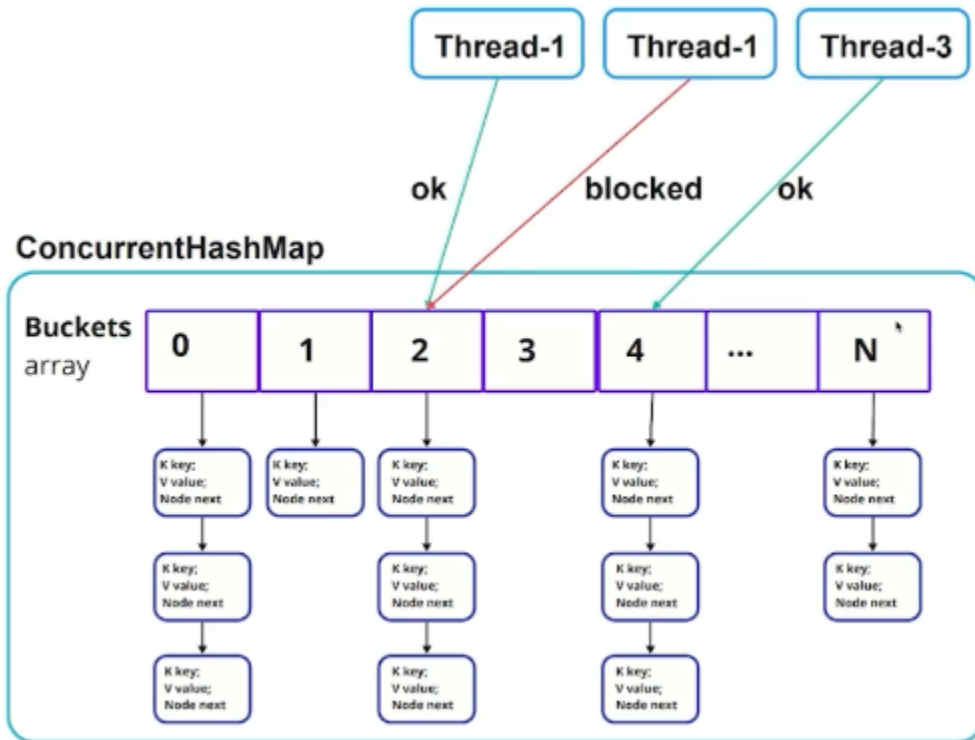
4. ConcurrentHashMap (CHM)

Идея: CHM оптимизирован под сценарий «много потоков одновременно пишут и читают» — в отличие от `synchronizedMap`, где блокируется вся мапа.

Как устроено внутри (концептуально):

- массив **бакетов**, в каждом бакете — список/дерево узлов
- запись в бакет **блокирует только этот бакет** (lock striping), и случается это довольно редко

- **чтение — вообще без блокировок:** читатели видят актуальные значения через volatile-ссылки
- итератор — **слабосогласованный (weakly consistent):** не бросает `ConcurrentModificationException`, идёт по состоянию на момент создания, изменения после — могут быть видны, а могут и нет



Бенчмарк: `synchronizedMap` vs CHM

50 писателей + 5000 читателей, каждый делает 10_000 операций:

Реализация	Время	Что происходит
<code>synchronizedMap(HashMap)</code>	~3 сек	Глобальный монитор: читатели ждут писателей и наоборот; с итераторами вообще <code>ConcurrentModificationException</code>
<code>ConcurrentHashMap</code>	~0.6 сек	Читатели не блокируются, писатели расходятся по бакетам

Числа условны (зависят от машины), но **разница на порядок при росте читателей** — константа этой главы.

```
Map<Integer, String> map = new ConcurrentHashMap<>();
// 50 writers: map.put(j, "val-" + j)    ×10_000
```

```
// 5000 readers: map.get(j)
```

```
×10_000
```

5. Блокирующие очереди

Producer–Consumer — безопасный транспорт данных между потоками:

Очередь	Внутри	Особенности
<code>ArrayBlockingQueue</code>	фиксированный массив (кольцевой буфер)	Предсказуемая память, компактность; одна общая блокировка на put+take
<code>LinkedBlockingQueue</code>	связный список, гибкая ёмкость	Аллокация ноды на каждый элемент; две блокировки : отдельно для producers, отдельно для consumers → выше пропускная способность под нагрузкой

- Операции `put / take` блокируют потоки для безопасной передачи данных
- **Выбор**: нужен компактный фикс. размер и минимум аллокаций → `ArrayBlockingQueue`; нужна высокая пропускная способность на больших объёмах → `LinkedBlockingQueue` (цена — накладные расходы на создание объектов-нод)

6. Другие concurrent-коллекции

Коллекция	Где уместна	Издержки
<code>ConcurrentSkipListMap/Set</code>	Упорядоченные ключи (sorted)	Логарифмические операции
<code>ConcurrentLinkedQueue</code>	Lock-free, очереди событий	Без back-pressure (неограниченный рост)
<code>LinkedBlockingDeque</code>	Двусторонние сценарии (work-stealing у воркеров)	Lock-free не гарантирован, FIFO/LIFO
<code>CopyOnWriteArraySet</code>	«Много читают / мало пишут»	Дорого на запись (копирует весь backing-массив)

7. Ключевые решения и почему

Решение	Почему
Копия массива вместо блокировки (COWAL)	Чтение становится lock-free, но запись дорожает
Блокировка по бакету, а не по мапе (CHM)	Писатели расходятся, читатели не блокируются вообще
Снапшот-итераторы	Никаких <code>ConcurrentModificationException</code> , цена — не всегда свежие данные
Две блокировки в <code>LinkedBlockingQueue</code>	<code>Producers</code> и <code>consumers</code> не мешают друг другу

8. Итоги

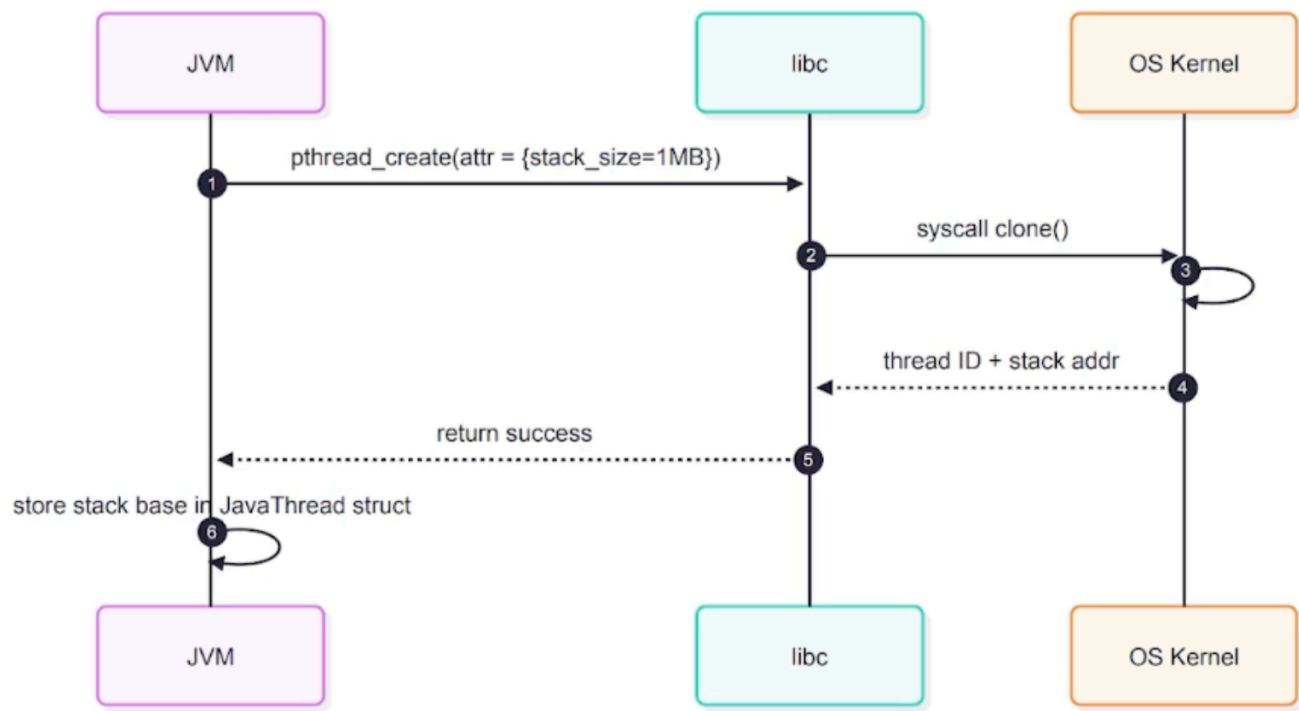
- Выбор коллекции = профиль чтения/записи + требования к порядку
- **CHM** — для параллельных обновлений; **COWAL** — для read-heavy (100:1 и выше)
- **BlockingQueue** — безопасный транспорт между потоками (back-pressure из коробки)
- `Collections.synchronizedXxx` — только как заплатка: работает, но глобальная блокировка убивает масштабируемость
- Быстрый критерий: пишешь часто → CHM / блокирующие очереди; читаешь почти всегда → COWAL; нужен порядок ключей → `SkipList`

Глава 3.4. Virtual Threads (Loom) введение, mapping и mount unmount

Темы: почему OS-потоки дорогие, что такое виртуальные потоки, mount/unmount (park/unpark), сравнение с платформенными потоками, executor'ы для VT

1. Проблема: OS-потоки дорогие

Создание платформенного потока — это системный вызов в ядро ОС:



- **Мало:** десятки-сотни потоков на процесс (лёгких мест нет)
- **Дорого:** каждый держит native stack ~1 МБ + syscall на создание и context-switch
- **Предел:** тысячи потоков → OOM unable to create native thread

Проблема: IO-bound нагрузка

Тип операции	CPU время	Время ожидания I/O
HTTP-запрос	<1%	>99%
JDBC-query	<1%	>99%
File read/write	<1%	>99%

Потоки в бэкенде 99% времени **ждут**. Классическое решение — «пул из N потоков, N >> ядер» (глава N) — бьётся о цену потока: держать 10_000 OS-потоков, которые просто ждут сокет, — расточительно.

2. Виртуальные потоки — это что

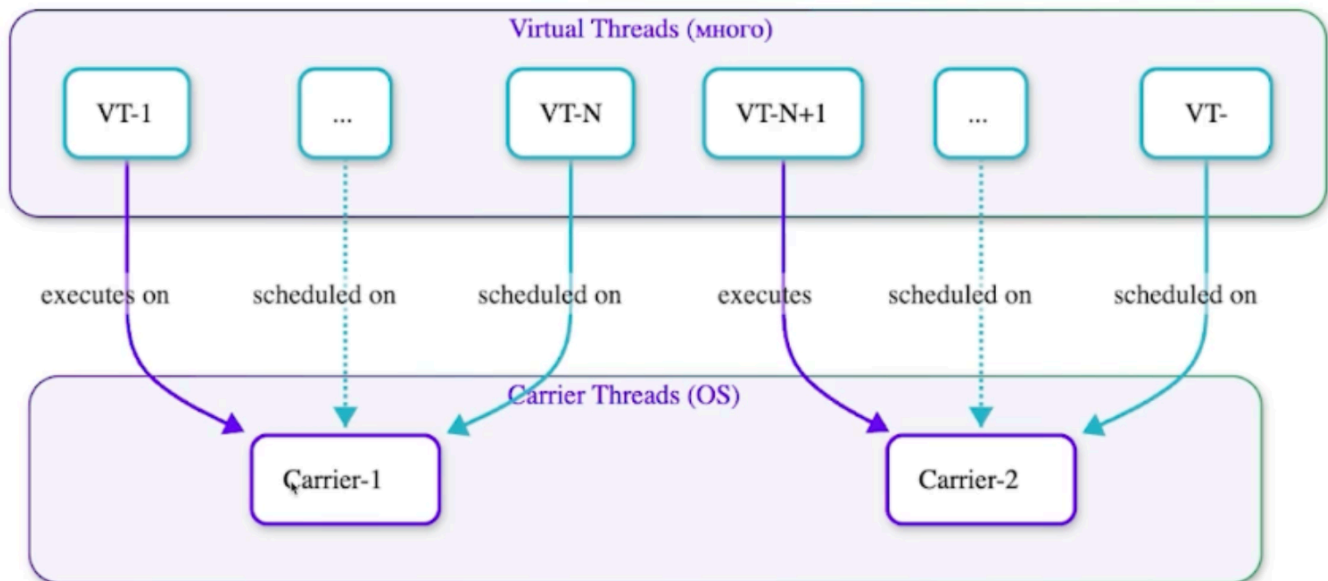
Виртуальные потоки — лёгкие потоки исполнения в JVM, **спланированные самой JVM** поверх ограниченного набора «носителей» (**carrier threads**) — обычных OS-потоков.

- **Цель:** дешёвая конкурентность без перегрузки OS-потоками
- **Модель:** много VT маются на малое число carrier-потоков (N:M)

- Блокирующие операции в VT приводят к **снятию с носителя (unmount)**, что позволяет носителю выполнять другие VT
- Это даёт возможность запускать **тысячи/десятки тысяч конкурентных задач** без затрат на создание и простой OS-потоков

Связь с предыдущими главами

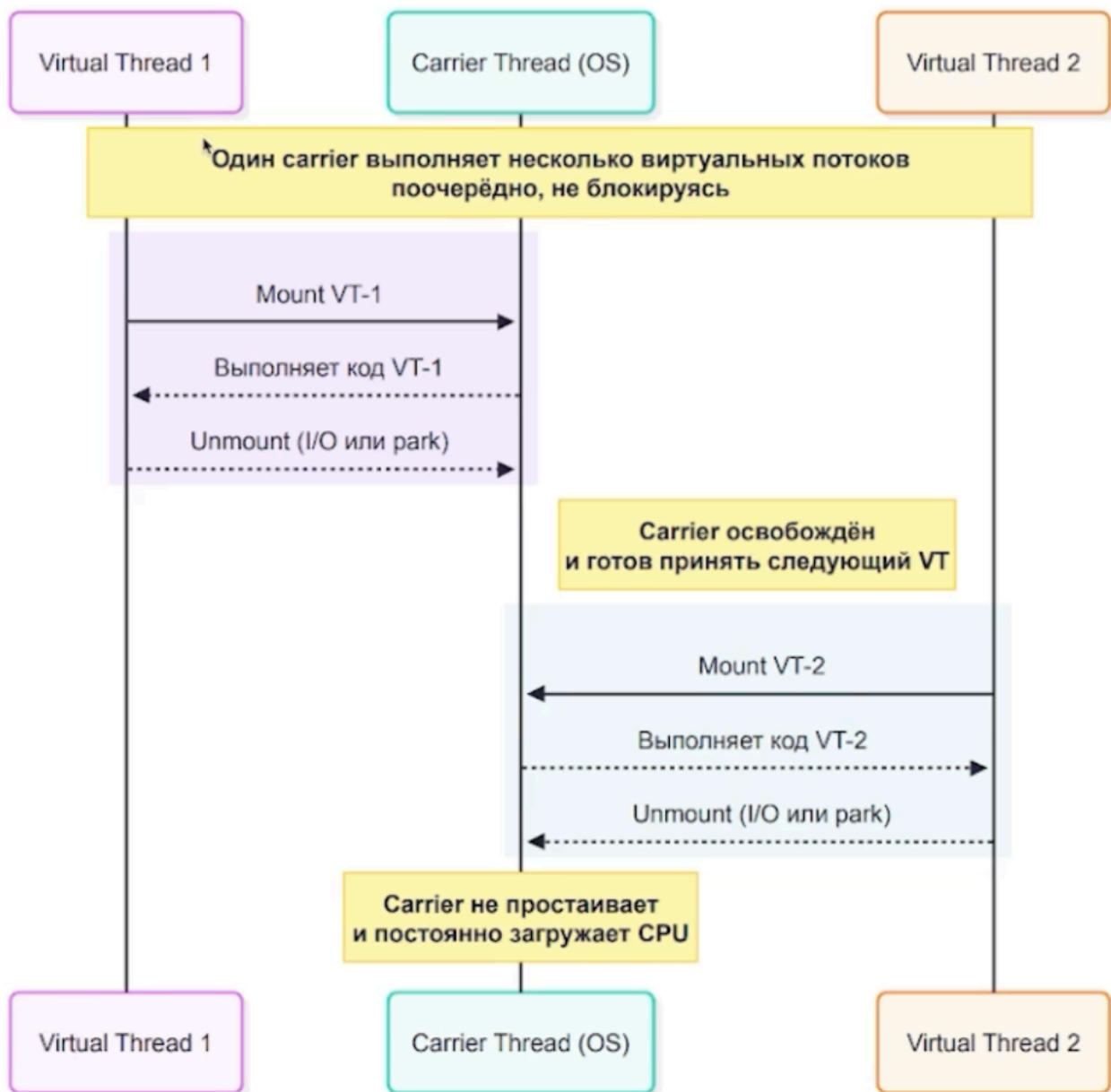
- **ForkJoinPool** — scheduler для **CPU-bound задач**: воркеры (carrier-подобные потоки) сами считают `RecursiveTask`, work-stealing балансирует нагрузку на ядра
- **Virtual Threads** — scheduler для **IO-bound задач**: JVM планирует лёгкие `Runnable`-задачи, а на блокирующем вызове VT паркуется и освобождает carrier



VT выполняется на carrier, затем может быть **снят и продолжен позже на другом carrier**.
 Выигрыш — при большом числе ожиданий (IO-bound).

3. Mount/Unmount стека (park/unpark)

- При блокировке VT «**снимает**» стек (**unmount**), освобождая carrier
- При возобновлении стек «**монтируется**» обратно (**mount**) — возможно, уже на другом carrier
- VT **паркуется на блокирующих вызовах** (`IO`, `LockSupport.park`, `sleep`), освобождая carrier для других VT
- ⚠ **Некоторые блокировки могут «прибить» carrier (pinning)** — например, `synchronized` + native вызов внутри. Carrier не освобождается, теряется смысл VT. В логах: `-Djdk.traceVirtualThreads=...`. Лечится заменой `synchronized` на `ReentrantLock`



Где стек VT живёт между mount'ами: continuation в heap — поэтому VT стоит десятки КБ, а не мегабайты.

4. VT vs OS Threads — сравнение

Критерий	Виртуальные	Платформенные
Соответствие ОС	N:M через carriers ($N \gg M$)	1:1 OS-thread
Где хранится стек	в heap (continuation)	в native stack
Занимает памяти	десятки КБ	~1–8 МБ
Количество	сотни-тысячи	десятки-сотни

Критерий	Виртуальные	Платформенные
Планирование	JVM	ОС
Блокировки	паркуются на IO	блокируют OS-поток
Контекст-свитч	дешёвый (копия Java-фреймов)	дорогой (syscall)

5. Демо

Создание Virtual Thread

```
Thread virtualThread = Thread.ofVirtual()
    .name("VirtualThread-" + id)
    .start(() -> {
        ExternalCallSimulationUtil.simulateCall(1, 100, 1); // IO
        log("external call done, id=%d", id);
    });

virtualThread.join();
```

API зеркалит обычный Thread: `ofVirtual()` вместо `new Thread()`, дальше те же `start/join/isAlive`. Разница — внутри.

Демо: массовый запуск — VT vs fixed pool

```
private static final int TASKS_COUNT = 100_000;

// Вариант 1: виртуальный executor – VT на задачу
try (ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor())
{
    for (int i = 0; i < TASKS_COUNT; i++) {
        int taskId = i;
        executor.submit(() -> makeExternalCall(taskId));
    }
}

// Вариант 2: fixed pool из 10_000 платформенных потоков
try (ExecutorService executor = Executors.newFixedThreadPool(10_000)) {
    for (int i = 0; i < TASKS_COUNT; i++) {
        int taskId = i;
        executor.submit(() -> makeExternalCall(taskId));
    }
}
```

```
}  
}
```

Результат: fixed-пул из 10_000 OS-потоков ($\approx 10+$ ГБ стеков + 10_000 syscall) — вплоть до `OutOfMemoryError: unable to create native thread`. Виртуальный executor с тем же числом задач работает на горстке carrier-потоков — стоимость задачи = объект в heap.

Справка: `newVirtualThreadPerTaskExecutor()` — каждая задача получает свою VT, пул сам масштабируется. Под капотом — тот же `Thread.ofVirtual().start(...)`, просто в обёртке `ExecutorService` для совместимости с существующим кодом (`submit`, `try-with-resources`).

6. Ключевые решения и почему

Решение	Почему
Стек VT в heap (continuation)	Поток дешёвый: КБ вместо МБ, создаём тысячи
Unmount при блокировке	Carrier не простаивает на IO — один carrier обслуживает множество VT
Планирование в JVM, не в ОС	Контекст-свитч без syscall — десятки КБ копии фреймов
N:M маппинг	Количество VT не ограничено ядрами и лимитами ОС

7. Итоги

- **VT — простой путь к масштабной конкурентности для IO-bound:** пиши блокирующий код как раньше, получай поведение «как async», но без callback-ада
 - **Mapping и unmount уменьшают давление на планировщик ОС:** тысячи задач на нескольких carrier-потоках
 - **Выигрыш меньше на чистом CPU-bound:** если задача не ждёт, VT не даёт преимущества перед platform-потоками — бери `ForkJoinPool` (глава H)
 - **Когда что:** IO-bound сервер → VT (новый дефолт в Spring Boot 3 / Tomcat); CPU-bound вычисления → `ForkJoinPool`; независимые фоновые задачи → `ThreadPoolExecutor`
-

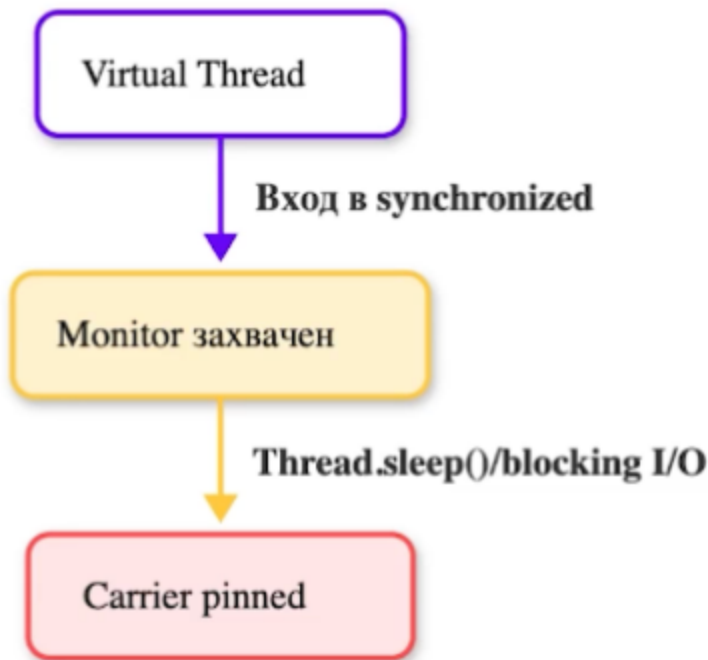
Глава 3.5. Проблемы Virtual Threads: pinning, нативные вызовы, ThreadLocal

Темы: что такое pinning, synchronized vs ReentrantLock, как обнаружить pinning, нативные библиотеки, ScopedValue вместо ThreadLocal

1. Что такое pinning

Thread Pinning — ситуация, когда виртуальный поток **удерживает carrier-поток**, и JVM не может размонтировать VT при блокирующей операции.

Обычно VT на IO делает так: `Unmount` → `carrier` свободен → другие VT. При pinning:



Carrier простаивает вместе с VT — масштабируемость резко падает до уровня «обычных потоков», а то и хуже.

Когда pinning происходит:

1. **Блокировка внутри synchronized** — монитор нельзя отдать другому VT, JVM вынуждена держать carrier (в JDK 21)
 2. **Нативные вызовы** — если нативная библиотека блокируется (socket, file IO в старых JNI-обёртках), JVM не может размонтировать VT: нативный код держит native stack carrier'a
-

2. Демо: pinning под synchronized

```
Object lock = new Object();
Runnable task = () -> {
    synchronized (lock) {
        System.out.println(Thread.currentThread() + " acquired lock");
        try {
            Thread.sleep(1000); // ← блокирует carrier-thread! (pinning на
JDK 21)
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
};

try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 10; i++) {
        executor.submit(task);
    }
}
```

Все 10 VT последовательно захватывают один монитор и засыпают внутри него — **каждый раз carrier застревает на секунду**. Параллелизма нет, несмотря на виртуальные потоки.

Как обнаружить: флаг `jdk.tracePinnedThreads=full` — JVM печатает стектрейсы pinned-потоков:

```
System.setProperty("jdk.tracePinnedThreads", "full");
```

3. Демо: synchronized vs ReentrantLock — массовый контекст

`ReentrantLock` совместим с VT: `lock.lock()` → если не захвачен, VT может быть размонтирован при ожидании; при захваченном локе **sleep/IO НЕ пиннит carrier** (ожидание лока реализовано через `LockSupport.park`, VT-паркинг). В отличие от монитора, лок — это не часть carrier'a.

Сценарий: 500 разных ресурсов, 8 locking-воркеров случайно лочат ресурс и **спят внутри критической секции 10–100 секунд**, плюс 2000 IO-воркеров просто делают внешние вызовы.

```

private static final int SHARED_RESOURCES_COUNT = 500;
private static final Object[] MONITORS = new Object[SHARED_RESOURCES_COUNT];
private static final ReentrantLock[] REENTRANT_LOCKS = new
ReentrantLock[SHARED_RESOURCES_COUNT];

// воркер на synchronized: IO ПОД захваченным монитором → carrier pinning!!
static void makeWorkWithSynchronized(CountDownLatch latch) {
    int idx = ThreadLocalRandom.current().nextInt(SHARED_RESOURCES_COUNT);
    synchronized (MONITORS[idx]) {
        safeSleepWithoutThrow(10_000); // 10 сек IO внутри монитора
    }
    latch.countDown();
}

// воркер на ReentrantLock: то же самое, но БЕЗ pinning
static void makeWorkWithReentrantLock(CountDownLatch latch) {
    int idx = ThreadLocalRandom.current().nextInt(SHARED_RESOURCES_COUNT);
    REENTRANT_LOCKS[idx].lock();
    try {
        safeSleepWithoutThrow(100_000); // даже 100 сек IO – carrier
свободен
    } finally {
        REENTRANT_LOCKS[idx].unlock();
    }
    latch.countDown();
}

```

```

// main: 2000 IO-worker'ов + 8 locking-worker'ов, ждём IO-группу отдельно
inputOutputWorkersLatch.await(); // ← с ReentrantLock IO завершаются
быстро

// ← с synchronized они ВИСЯТ за pinned
carriers

```

Что видно: с `synchronized` IO-воркеры ждут, пока освободятся carriers, застрявшие в `sleep`'ах под мониторами. С `ReentrantLock` — 2000 IO-вызовов завершаются на горстке carrier'ов, не замечая locking-воркеров.

4. Исторический контекст: почему в JDK 21 это проблема

- В **JDK 21** (первый LTS с VT) pinning на мониторах — известное ограничение: JVM не умеет «отдавать» монитор без carrier'a

- **JEP 491: Synchronize Virtual Threads without Pinning** — пофикшено в новых JDK (начиная с JDK 24; в линейке LTS — начиная с **JDK 25**). `synchronized` больше не пиннит при обычном ожидании монитора
 - **Вывод для кода:** на JDK 21/23 — используй `ReentrantLock` там, где внутри критической секции бывает IO/ожидание. На JDK 24/25+ — `synchronized` снова безопасен, но рекомендация «не делать долгие IO внутри локов» остаётся в силе независимо от VT
-

5. Нативные библиотеки

Второй источник pinning — **JNI и нативные вызовы:**

- Нативная библиотека может заснуть внутри `native` метода (старый IO, некоторые драйверы БД, netty-эпох до обновлений)
- JVM **не может размонтировать VT посреди нативного кода:** `native stack` принадлежит конкретному carrier-потoku
- Итог: VT с native-вызовом = pinned carrier на всё время вызова

Что делать: обновлять библиотеки до VT-совместимых версий (драйверы БД, Netty, gRPC — все выпускали апдейты под Loom), держать нативные вызовы вне критических секций и пиннинг-трейсить в нагрузочных тестах.

6. ThreadLocal: риски и замена

`ThreadLocal` на VT юридически работает, но есть ловушка:

- Платформенный поток живёт долго — `ThreadLocal` «окупается»
- VT живут **миллионами и коротко** — копия `ThreadLocal` на каждый VT. Тяжёлые значения (кеши, буферы на мегабайты) × миллионы VT = **утечка памяти**, если не чистить вручную

Практические рекомендации VT:

- Предпочитай неблокирующие конструкции и VT-совместимые примитивы
- Избегай долгих критических секций (и pinning'a)
- Разделяй IO-bound и CPU-bound задачи
- Проверь библиотеки на использование `synchronized` и нативных блокировок

- Избегай тяжёлых ThreadLocal для миллионов VT — риск утечки памяти, если не ЧИСТИТЬ

Замена — ScopedValue (JDK 25):

```
// было: ThreadLocal — своя копия на КАЖДЫЙ поток, живёт пока не remove()
static final ThreadLocal<Context> CTX = new ThreadLocal<>();

// стало: ScopedValue — одно значение на всю цепочку задач, иммутабельно,
// освобождается автоматически по выходу из scope, нет риска утечки
static final ScopedValue<Context> CTX = ScopedValue.newInstance();

ScopedValue.where(CTX, context).run(() -> {
    // внутри — и VT, и код, который они вызывают, видят context
    // никакого remove(), никаких миллионов копий
});
```

ScopedValue — не «локальная переменная потока», а **неизменяемый контекст, пробрасываемый через вызовы** (аналог structured concurrency-подхода): без копирования, без утечек, дешевле на массовых VT. В JDK 25 ScopedValue — финализирован (JEP 492), замена ThreadLocal для нового кода.

7. Итоги: проблемы с VT

- **Pinning ломает преимущества VT** — carrier застревает, масштабируемость падает до уровня платформенных потоков
- **Фикс — VT-совместимые локи и паттерны ожидания:** ReentrantLock, Semaphore, CountdownLatch, LockSupport.park — всё это умеет отдавать carrier; synchronized — только начиная с JDK 24/25 (JEP 491)
- **Нативные блокировки невидимы для JVM** — обновляй библиотеки, трассируй пиннинг (jdk.tracePinnedThreads) в стейджинге
- **ThreadLocal — источник утечек на миллионах VT;** на JDK 25+ используй ScopedValue (JEP 492)
- **VT просты в применении, но могут выстрелить** (особенно на JDK 21 LTS) — профилируй pin-предупреждения перед продам

4. Многопоточность на проде

4.1. Модели обработки запросов в бэкенде (Thread-per-Request vs Event Loop vs Virtual Threads)

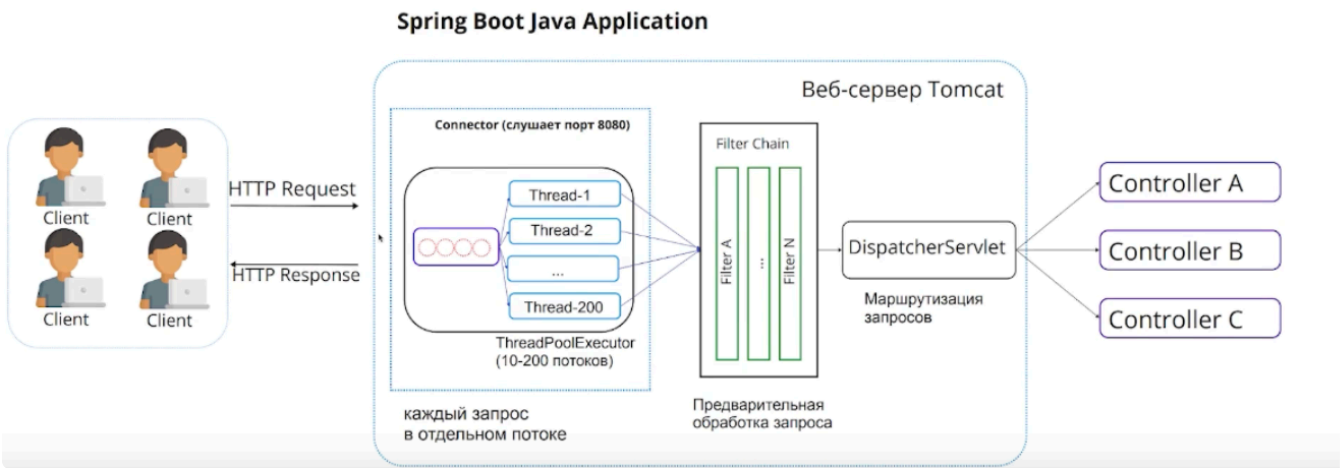
Темы: thread-per-request (Tomcat), очереди и узкие места, rate limiter, event loop / WebFlux, виртуальные потоки

Что такое бэкенд

- Приложение на Java, запущенное на **сервере** (просто мощный компьютер; часто арендуется в облаке — «облачный сервер»)
- Основная работа — **I/O нагрузка**: ходить в другие бэкенды, БД, читать/писать по сети
- Современные бэкенды держат **тысячи–сотни тысяч RPS** (маркетплейсы, такси, платёжные системы)

Модель 1: Thread-per-Request (Tomcat, дефолт Spring Boot)

Thread-per-request: Tomcat



Как работает

- Каждый запрос обрабатывается в **отдельном потоке** из пула
- Spring скрывает всё это за аннотацией `@RestController`

Лимиты

Параметр	Значение
Потоков по умолчанию	200 (можно переопределить, но...)

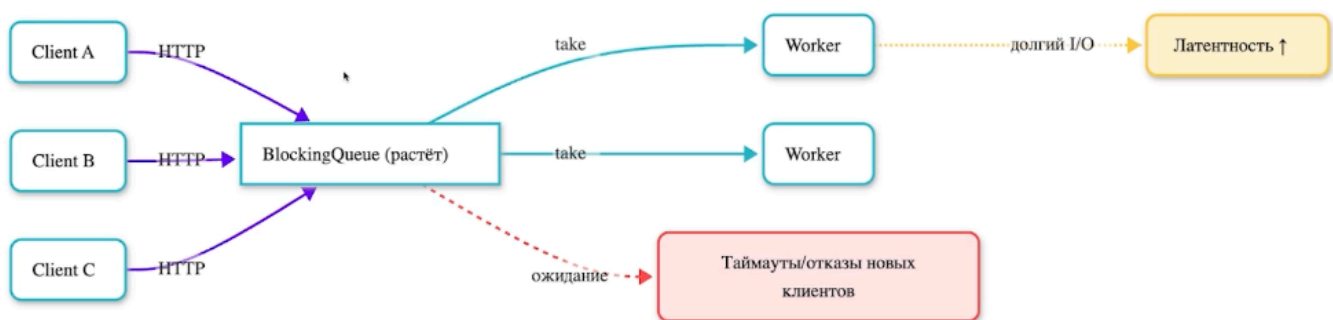
Параметр	Значение
Проблема	Обычные платформенные потоки = стек ~1 МБ на каждый. Тысячи потоков не создать — не хватит памяти

Где возникает бутылочное горлышко

1. Поток в основном **блокируется** (ждёт БД/сеть): ~1% времени CPU, 99% — ожидание
2. Заблокированный поток **не может взять новый запрос** из очереди
3. Если запросов больше, чем пропускная способность → очередь растёт

Пропускная способность = 50 запр/сек, приходят 60 запр/сек
→ +10 запросов в очереди КАЖДУЮ секунду
→ очередь бесконечно копится → память забита → сервис падает

Последствия переполненной очереди



- Самые старые запросы **отваливаются по таймауту** (клиенты ждут 10–30 сек и отключаются)
- Клиенты думают, что сервис **упал** (хотя он жив и работает)
- **Защита:** лимит очереди в Tomcat (напр., max 10 000 задач) — новые запросы сразу откидываются

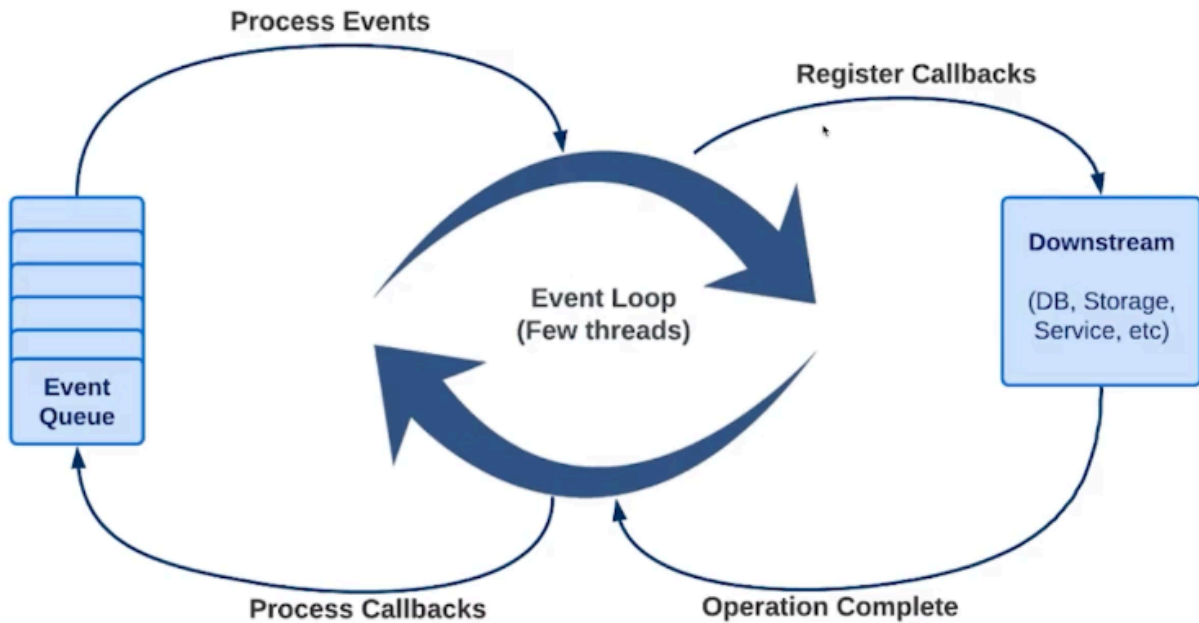
Rate Limiter — паттерн защиты

Клиенты → [Rate Limiter] → Backend
(max 50 RPS) (очередь не растёт)

- Если запросов больше лимита → **сразу отклоняем**, не пропуская в систему
- Реализации: встроенный в сервис, внешний (nginx, Traefik), на стороне клиента (недоверенный — клиент может слать в обход)

Модель 2: Event Loop (Spring WebFlux)

Event Loop - неблокирующая модель, где небольшое фиксированное число потоков (обычно = числу CPU). Обработывает множество соединений и задач, переключаясь при блокирующих операциях.



Стек

- **Netty** - неблокирующий сетевой фреймворк на Java. На нем написан сервер Reactor Netty.
- **Spring WebFlux** - реактивный веб-стек Spring на **Project Reactor**, по умолчанию работает поверх сервера Reactor Netty.

Принцип

- Воркеров столько, сколько **ядер CPU** (напр., 4)
- Потоки **никогда не блокируются**: постоянно крутятся в цикле, беря задачи из очереди

1. Воркер берёт задачу, исполняет
2. Задача упирается в I/O (БД, сеть)
3. НЕ ждём! Регистрируем callback в ОС:
«когда данные появятся — кинь сигнал»
4. Воркер берёт СЛЕДУЮЩУЮ задачу из очереди
5. ОС сигналиит → задача-дописывается в очередь → воркер её доберёт

Плюсы и минусы

Плюс	Минус
Нет блокировок → идеальная утилизация CPU при I/O-нагрузке	Совсем другая парадигма (всё на callback'ах)
Нужно мало потоков	Высокий порог входа, легко «выстрелить себе в ногу»
Отлично для I/O-heavy нагрузки	Не подходит для CPU-heavy задач

Блокирующая операция в Event Loop **не останавливает всю обработку, но пропорционально съедает пропускную способность** (1 из N воркеров те 25 процентов потенциально против 0.5% от дефолт томкат) и, при систематическом повторении, деградирует latency всего сервиса. Модель держится на инварианте «воркер никогда не ждёт» — его нарушение делает event loop бесполезным: получаем худшее из двух миров (мало потоков + блокировки).

Модель 3: Виртуальные потоки (Java 21+, Spring Boot 3.2+)

Как включить

```
spring:
  threads:
    virtual:
      enabled: true # Томкат переключается на виртуальные потоки
```

Что меняется

- Архитектура та же (thread-per-request), но потоки **виртуальные**
- Миллионы лёгких потоков вместо сотен тяжёлых
- Получаем преимущества Event Loop, **не меняя парадигму** — код остаётся обычным синхронным

Важные оговорки

Оговорка	Суть
Не серебряная пуля	Узкое место во внешней системе (20 RPS в БД) не решить никакими потоками
Pinning	Если виртуальный поток «прибьёт» носителя (synchronized , native-вызовы) — выигрыш сводится на нет

Оговорка	Суть
Java 21	Pinning реален → на 25 Java безопаснее; на 21 — включать флаг — <code>Djdk.tracePinnedThreads=full</code> и следить

5. Сравнение трёх моделей

Модель	Потоки	Парадигма	Когда брать
Tomcat (обычные потоки)	~200, блокируются	Обычный синхронный код	Дефолт для большинства сервисов — работает норм
WebFlux (Event Loop)	= ядра CPU, не блокируются	Реактивная (Mono/Flux)	Экстремальный I/O-heavy RPS, готовность к сложности
Виртуальные потоки	миллионы, дешёвые	Обычный синхронный код	I/O-heavy + хотим оставить привычный код (Java 21+/Boot 3.2+)

💡 Event Loop и виртуальные потоки решают одну задачу — **не держать дорогой ресурс (поток) во время I/O-ожидания** — но разными ценами: первая требует другой парадигмы, вторая — осторожности с pinning'ом.

Итоги

- Большинство бэкендов = I/O-нагрузка → потоки большую часть времени **ждут**
- Tomcat: 200 потоков → при превышении RPS очередь копится → таймауты → «сервис упал»
- Защита: **лимит очереди + rate limiter** перед сервисом
- Event Loop (WebFlux) = максимальная эффективность, но другая парадигма и высокий порог входа
- Виртуальные потоки = преимущества Event Loop в привычном коде, но следим за pinning'ом и внешними bottleneck'ами
- Виртуальные потоки не лечат узкие места во внешних системах — сначала найдите свой bottleneck

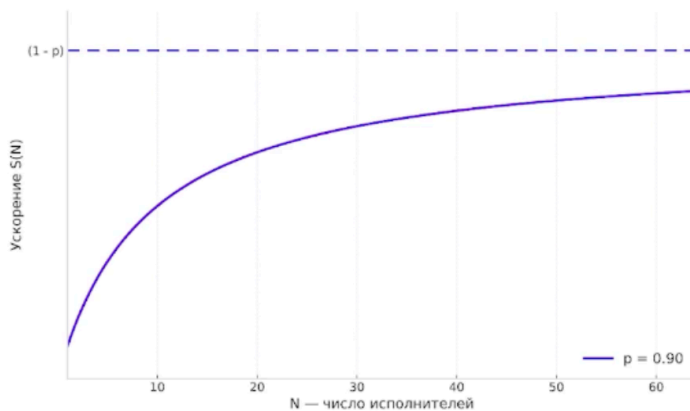
4.2. Закон Амдала и узкие места в продакшене

Темы: закон Амдаля, почему рост потоков $\neq$ рост TPS, типичные bottleneck'и (пул коннектов БД, rate limiter), практические советы

Закон Амдаля

Программа = параллелизуемая часть + последовательная (блокировки, ожидание ответов, общие ресурсы). Ускорение от N процессоров:

$$S = \frac{1}{(1 - P) + \frac{P}{N}} \quad \text{где} \quad \begin{cases} S & \text{— ускорение} \\ P & \text{— доля параллелизуемой части} \\ N & \text{— число процессоров} \end{cases}$$



Даже при ∞ количестве процессоров, но с $p = 90\%$ -> ускорение не больше чем в 10 раз

Ключевой вывод: **предел ускорения — $1/(1-P)$** . Даже при ∞ процессоров и $P = 90\%$ ускорение не больше чем в 10 раз.

Почему $P \neq 1$ на практике: мы всё равно ждём ответов (сеть, БД), есть общие ресурсы (локи, пулы), есть последовательные участки кода.

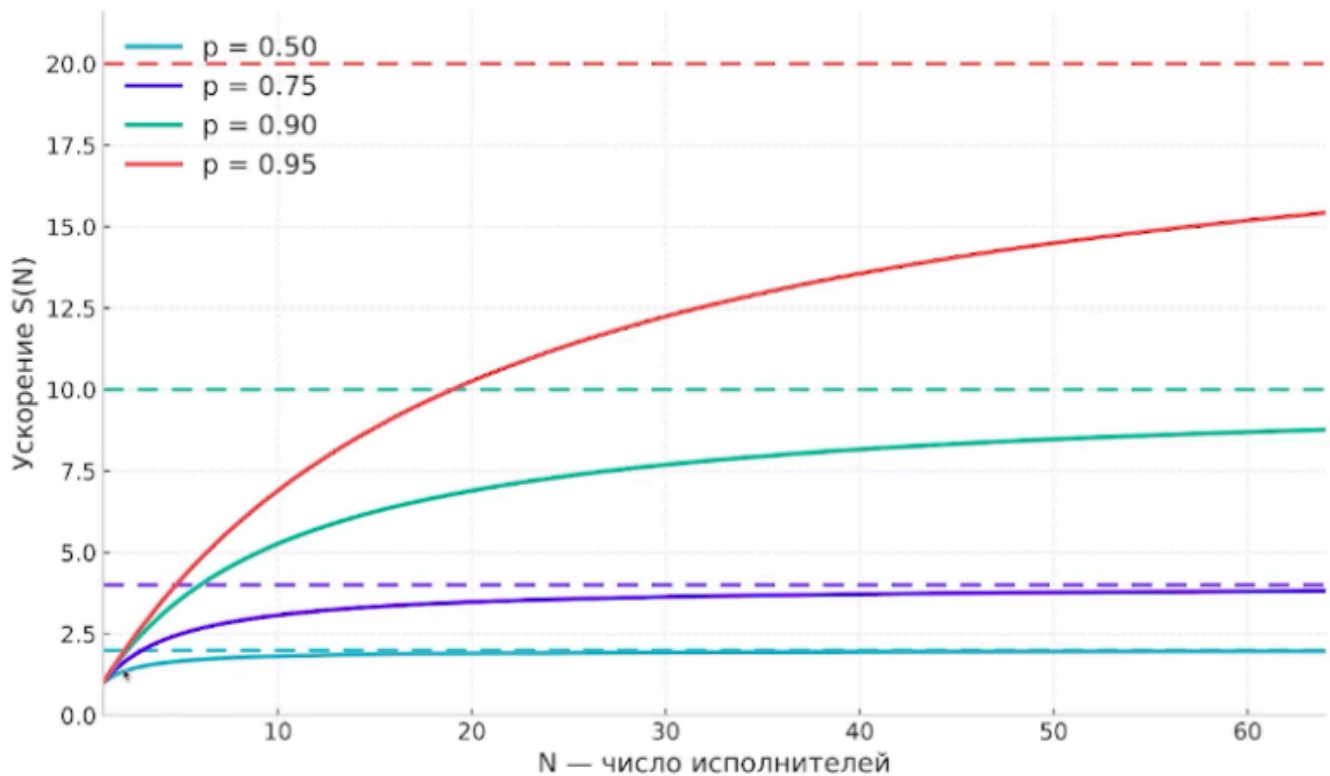


график: чем выше P , тем выше потолок; при $P=0.5$ потолок = $2x$, при $P=0.95 \rightarrow \sim 20x$

Выводы из закона Амдаля

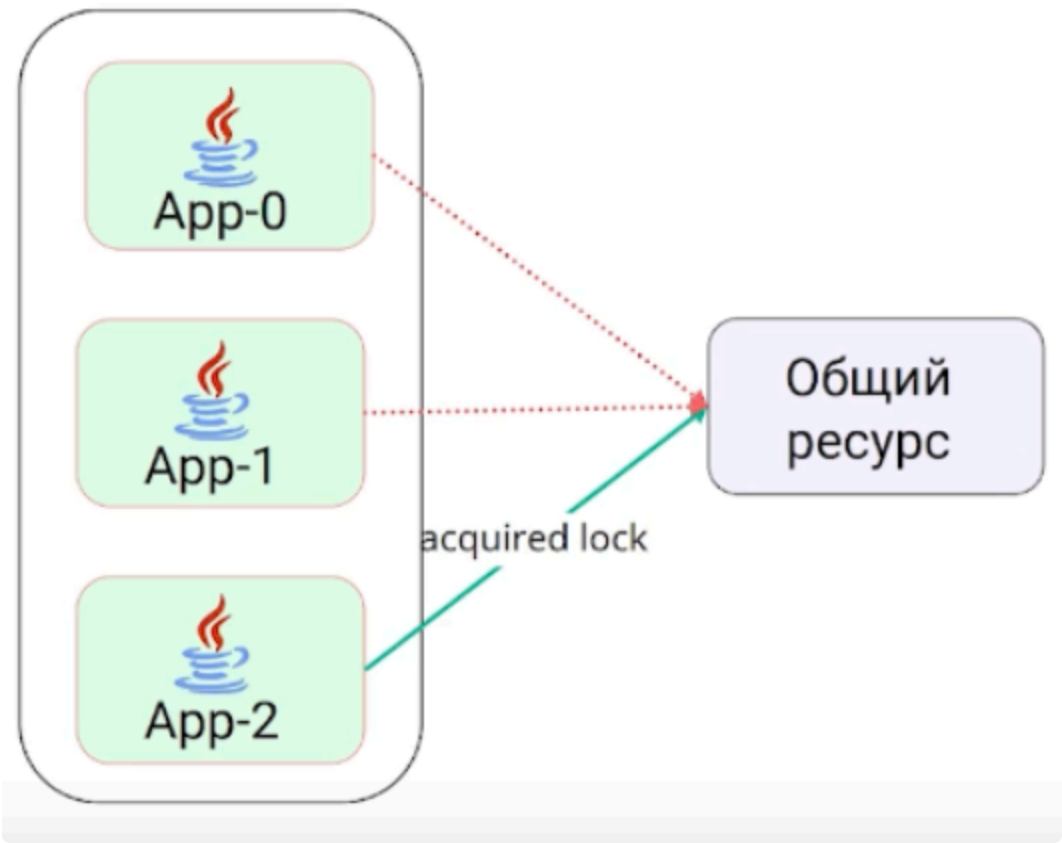
- Масштабировать ядра/инстансы/реплики даёт хороший рост, при условии, что они не упрутся в **bottleneck**
- С определённого момента становится **дешевле убрать непараллелимую часть, чем добавлять ядра**
- Любые блокировки/секции ухудшают ускорение всей системы

Это объясняет главы: почему глобальная блокировка `synchronizedMap` убивает масштабируемость, почему `pinning VT` обнуляет выигрыш, почему «просто добавить потоков» не работает.

Где чаще всего узкие места

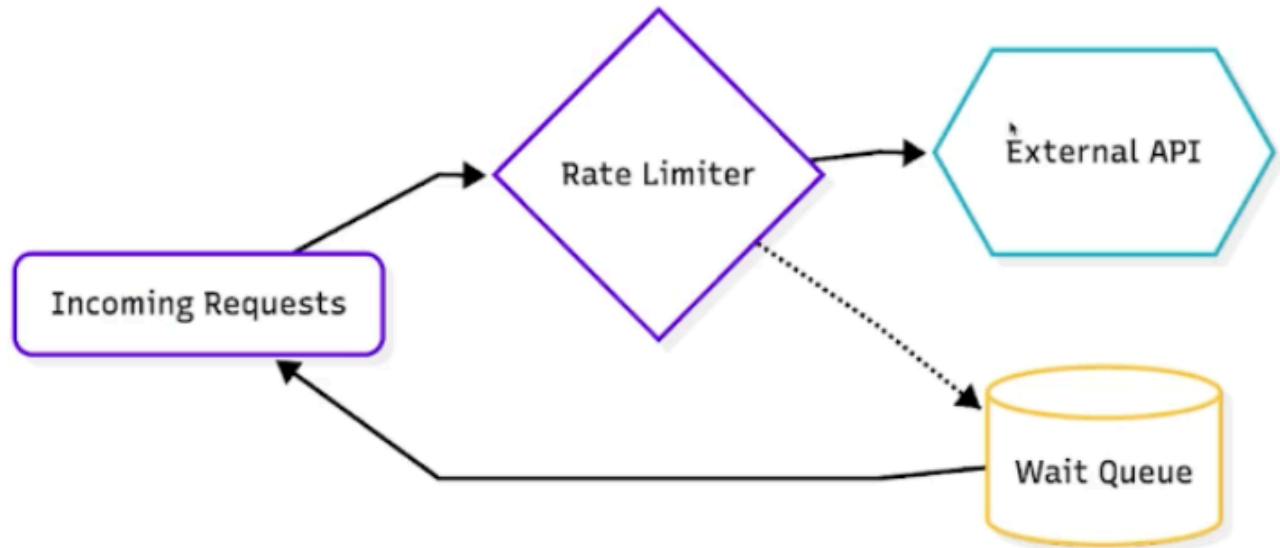
Общее правило: любой разделяемый ресурс = потенциальный **bottleneck**. Уменьшай количество блокировок.

Типичные места:

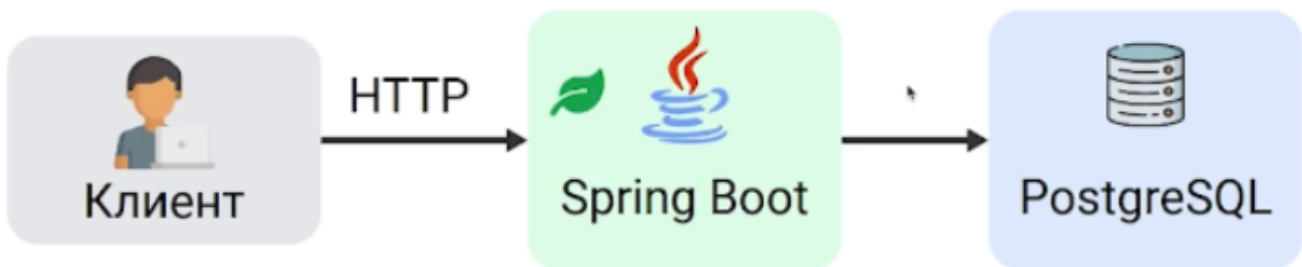


Bottleneck	Что происходит
Пул коннектов к БД, файловая система	Коннекты тяжело создавать → переиспользуются (pool). Пул ограничивает параллельные транзакции
Rate limiter внешнего API	Если внешний сервис режет запросы, никакие потоки/ядра не помогут. Стратегии: очередь/ретраи/деградация/кэш. Back-pressure обязателен
Блокировки в приложении	App-0..2 держат acquired lock на общем ресурсе → всё остальное ждёт

Rate-Limiter как bottleneck

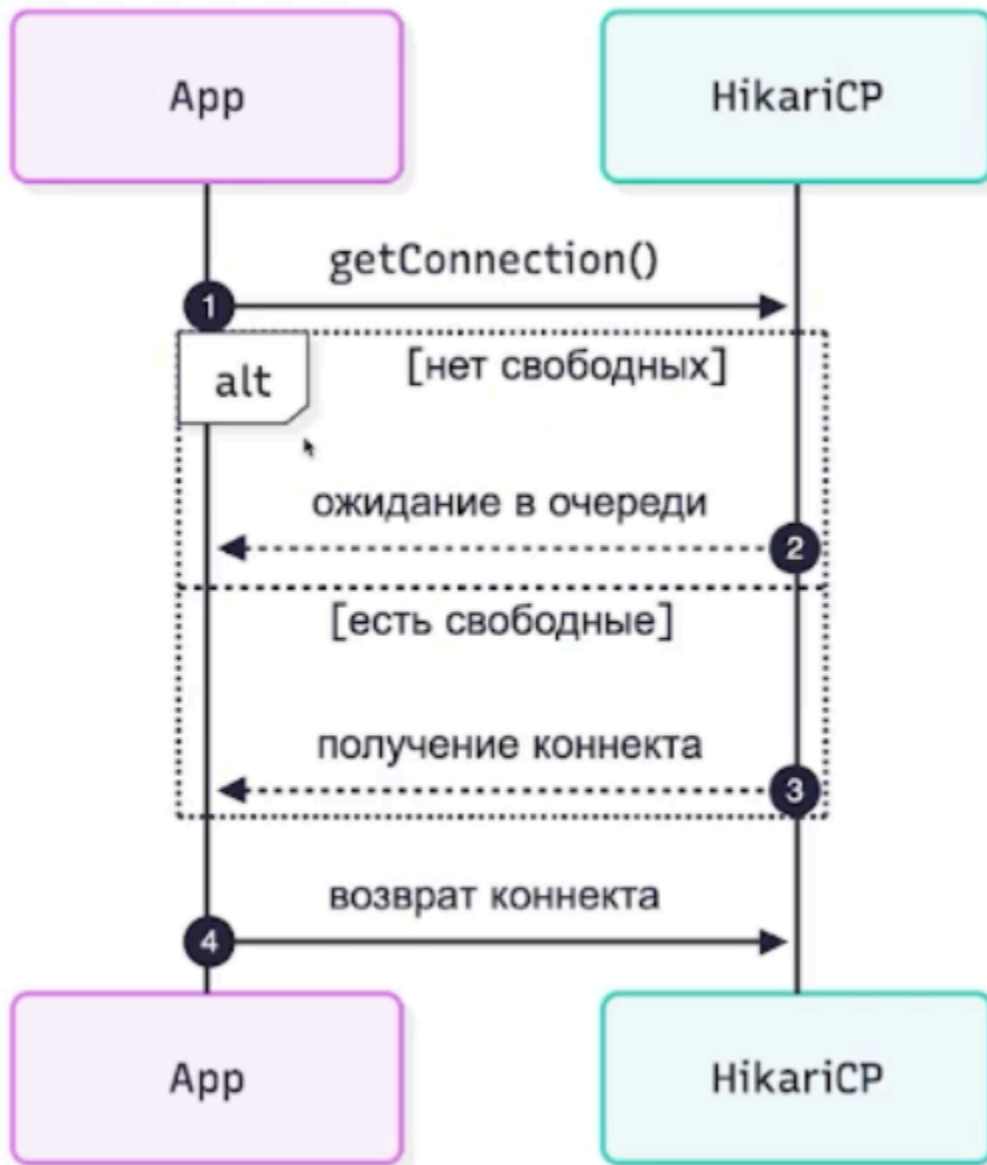


Pool коннектов БД как bottleneck



- Коннекты к БД тяжело создавать, они могут быть дорогими, поэтому переиспользуются (pool)
- **HikariCP ограничивает параллельные транзакции** — размер пула = потолок конкурентных запросов к БД
- **Рост потоков в приложении ≠ рост TPS при малом пуле:** добавишь 10_000 VT/потоков, но пул из 10 коннектов пропустит всё равно 10 запросов. Остальные — в очередь
- PostgreSQL вообще **на каждый коннект создаёт отдельный процесс** — поэтому коннекты дороги и пул маленький

HikariCP как устроен



- **Блокирует потоки, если все коннекты заняты** — `getConnection()` ждёт
- Долгие или зависающие запросы могут **положить весь сервис**, т.к. забивают общий пул коннектов

Практические советы

- **Сокращай блокирующие секции/синхронизацию** — прямая заявка закона Амдаля: меньше последовательной части → выше потолок
- **Изолируй медленные ресурсы** — выноси вызовы внешних API в отдельные пулы или очереди, чтобы не блокировать общий executor
- **Каждое I/O должно иметь время ожидания и план Б** для деградации (заглушку, кэш, старые данные) — timeout'ы

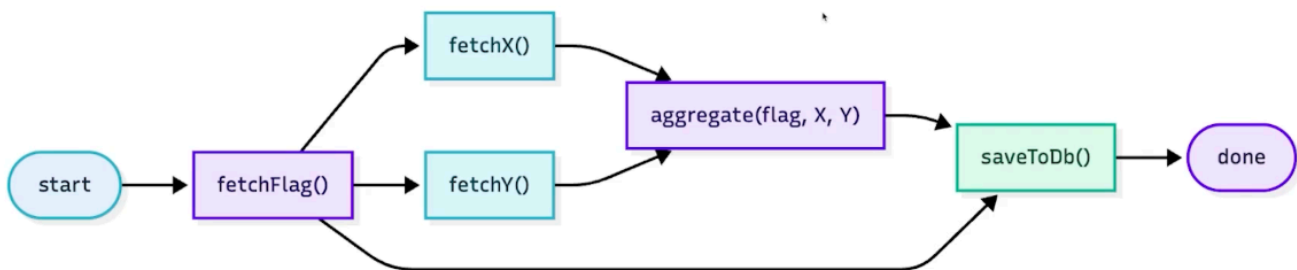
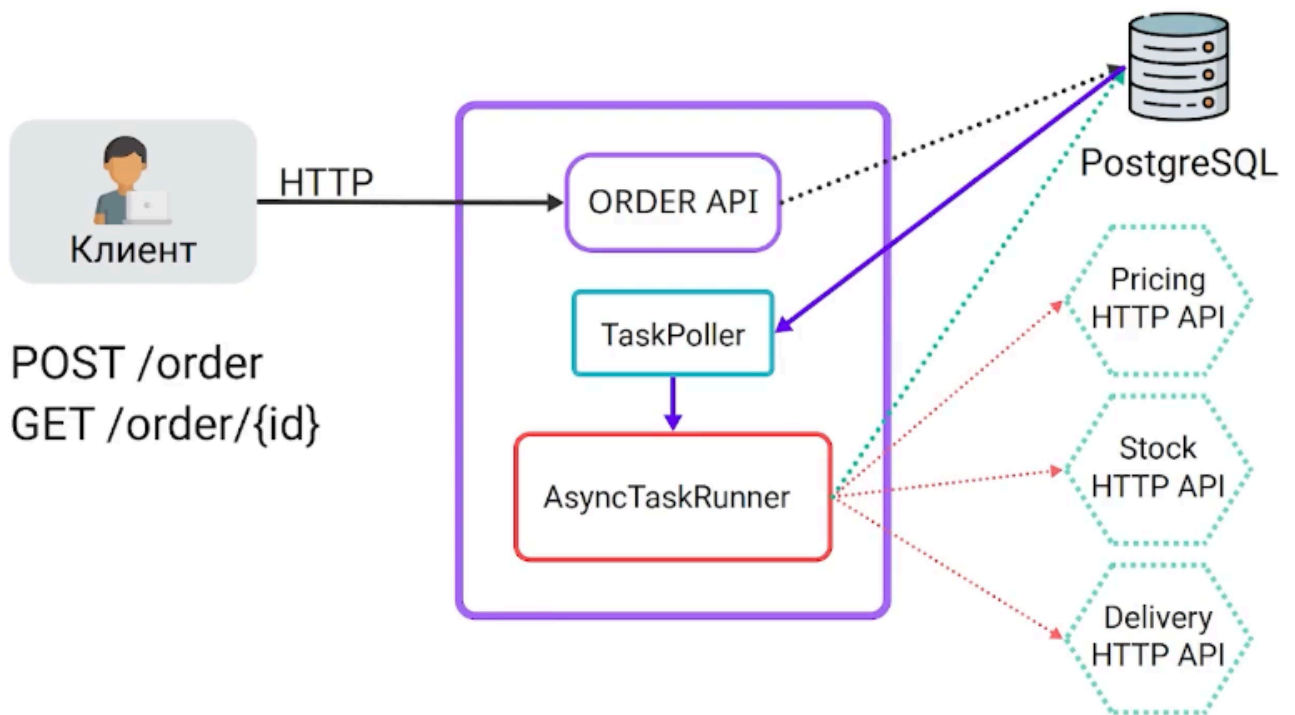
- **Не жди ответа от сервисов, которые упали** — circuit breaker при массовых ошибках во внешних сервисах: быстрый фейл вместо зависания потоков на таймаутах
-

Итоги

- **Закон Амдаля: ускорение ограничено последовательной частью**, а не числом потоков
 - Пул коннектов БД и внешние API — самые частые bottleneck'и; рост потоков их не лечит
 - Масштабирование работает, только если убраны точки сериализации (блокировки, общие пулы, ожидания)
 - Формула практика: **$TPS \approx \min(\text{потоки, пул коннектов, пропускная способность БД, лимиты внешних API})$** — масштабировать надо самое узкое звено
-

4.3 Финальный прод-кейс AsyncTaskRunner

Получается такая цепочка: быстрый синхронный ответ клиенту → фоновый планировщик по расписанию → диспетчер раздаёт задачи по виртуальным потокам → внутри задачи один запрос идёт последовательно, два — параллельно → результат пишется обратно в базу. Клиент при этом может в любой момент спросить `GET /orders/{id}`, и это просто чтение текущего статуса из базы — к самому конвейеру оно никак не привязано.



Репозиторий	Роль
async-order-processor-service	сам прод-кейс: Spring Boot сервис с AsyncTaskRunner (poller → dispatcher → processor)
order-external-http-stub	управляемая заглушка внешних зависимостей (inventory/pricing/shipping) — источник задержек и отказов для нагрузочных тестов
order-payment-system	отдельное ДЗ «AsyncRunner 2049» с тем же паттерном, но бизнес-логика оркестрации платежа не реализована — каркас для самостоятельного дописывания, к этому прод-кейсу прямого отношения не имеет

Разбор ниже — про первые два репозитория: это готовая, рабочая реализация AsyncTaskRunner, которую можно гонять под нагрузкой.

Архитектура: синхронный ответ + асинхронный конвейер

Ключевая идея кейса — `POST /orders` отвечает быстро и не ждёт внешних вызовов; вся работа с инвентарём/ценой/доставкой уезжает в фон через таблицу `tasks` как очередь.

DB как очередь задач

`TaskPoller` выбирает пачку задач на обработку одним запросом с `FOR UPDATE SKIP LOCKED` :

```
select * from tasks
where (status = :newStatus)
      or (status = :retryStatus and next_attempt_at <= :now)
      or (status = :processingStatus and next_attempt_at <= :now)
order by id
limit :batchSize
for update skip locked
```

- **SKIP LOCKED** — если несколько экземпляров сервиса поллят одну таблицу параллельно, каждый берёт свою порцию строк, не блокируясь на чужих — без этого параллельные поллеры либо сериализовались бы, либо задваивали обработку.
- **next_attempt_at как лиза (visibility timeout)** — тот же `UPDATE` сразу выставляет `status=IN_PROGRESS` и `next_attempt_at = now + retryDelay`. Запрос забирает не только `NEW / FAILED_RETRYABLE`, но и просроченный `IN_PROGRESS` — если сервис упал посреди обработки, задача не зависает навсегда, а переподхватывается следующим поллером.
- Статус и `attempts` обновляются **до** `dispatch()`, в той же транзакции — это и есть «бронирование» задачи конкретным воркером.

Диспетчеризация: fire-and-forget + виртуальные потоки

```
public void dispatch(AsyncTaskEntity task) {
    CompletableFuture
        .supplyAsync(() -> taskProcessor.processTask(task),
taskDispatcherThreadPool)
        .thenAccept(result -> handleTaskExecuted(task, result))
        .exceptionally(ex -> handleExceptionInTaskHappened(task, ex));
}
// poll() не ждёт dispatch() — fire-and-forget,
// поэтому следующий тик планировщика может перекрыться с этим
```

- `poll()` не ждёт `dispatch()` — это **fire-and-forget**: `@Scheduled(fixedDelay=...)` тикает заново, пока предыдущие задачи ещё выполняются в фоне. Именно поэтому `SKIP LOCKED` обязателен, а не просто «на всякий случай» — без него два последовательных тика поллера реально могут пересечься.
- Ошибки до/во время `processTask()` ловятся в `.exceptionally(...)` и уходят в `scheduleTaskRetry()` — ни одно исключение не тонет молча (в отличие от `JoinDemo` из главы 1, где необработанное исключение в голем `Thread` терялось).

Executor'ы — виртуальные потоки вместо пула фиксированного размера:

```
@Bean(destroyMethod = "shutdown")
public ExecutorService taskDispatcherThreadPool(TaskDispatcherProperties
properties) {
    // return Executors.newFixedThreadPool(properties.getThreadPoolSize());
    return Executors.newVirtualThreadPerTaskExecutor();
}

@Bean(destroyMethod = "shutdown")
public ExecutorService externalHttpThreadPool(
    @Value("${task-execution.external-http.thread-pool-size}") int
threadPoolSize
) {
    // return Executors.newFixedThreadPool(threadPoolSize);
    return Executors.newVirtualThreadPerTaskExecutor();
}
```

- **Почему virtual threads:** работа `TaskDispatcher / TaskProcessor` — это почти целиком ожидание блокирующих HTTP-вызовов (JDK `HttpClient` под `RestClient` блокирует поток на `.send()`). Для IO-bound нагрузки дешёвые виртуальные потоки снимают числовой лимит «сколько задач можно обрабатывать одновременно» — раньше это упиралось бы в размер `newFixedThreadPool`.
- **Footgun:** `TASK_EXEC_DISPATCHER_THREAD_POOL_SIZE` и `TASK_EXEC_EXTERNAL_HTTP_THREAD_POOL_SIZE` всё ещё читаются из конфига (`properties.getThreadPoolSize()`, `@Value(...)`), но никак не влияют на `Executors.newVirtualThreadPerTaskExecutor()` — реальные ограничители этих пулов закомментированы строкой выше. Крутить эти env-переменные при нагрузочном тесте — бесполезно, пока бины не вернут на `newFixedThreadPool`.
- **Где реально может прилипнуть виртуальный поток:** если в коде на пути блокирующего вызова появится `synchronized` — виртуальный поток **пиннится** к своему carrier-потoku на всё время блокировки (глава 5 — `synchronized / monitor lock`). В `build.gradle.kts` сервиса уже включён `-Djdk.tracePinnedThreads=full` — это диагностический флаг JVM, который печатает стектрейс каждый раз, когда

виртуальный поток пинится; в этом коде `synchronized` не используется, но флаг стоит на будущее/для упражнения.

Параллельные внешние вызовы

```
var reservation = stubHttpClient.reserveInventory(orderId.toString());
if (!reservation.stockReserved()) {
    return handleStockReservationRejected(order.get(), reservation);
}

var pricingFuture = CompletableFuture.supplyAsync(
    () -> stubHttpClient.calculatePricing(orderId.toString()),
    externalHttpThreadPool);
var shippingFuture = CompletableFuture.supplyAsync(
    () -> stubHttpClient.scheduleShipping(orderId.toString()),
    externalHttpThreadPool);

pricingFuture.thenAcceptBoth(shippingFuture,
    (pricing, shipping) -> handlePriceAndShippingSucceed(order.get(),
    pricing, shipping)
).get(); // ждём обе ветки здесь, внутри virtual-поток задачи
```

`reserveInventory` — последовательно и в начале (нет смысла резервировать цену/доставку для заказа, которому откажут в резерве). `calculatePricing` / `scheduleShipping` — параллельно через `thenAcceptBoth`, после чего `.get()` блокирует текущий виртуальный поток до завершения обеих веток — это нормально именно потому, что поток виртуальный и блокировка дешёвая.

Ретраи и статусы

TaskStatus	Когда	Что дальше
NEW	задача только создана	подхватывается поллером
IN_PROGRESS	поллер забрал в работу	либо завершится, либо «протухнет» по <code>next_attempt_at</code> и будет подхвачена снова
FAILED_RETRYABLE	<code>RETRYABLE_ERROR</code> / неожиданное исключение	ждёт <code>nextAttemptAt</code> , снова подхватывается поллером, пока <code>attempts < maxAttempts</code>

TaskStatus	Когда	Что дальше
FAILED_NON_RETRYABLE	NON_RETRYABLE_ERROR либо исчерпан maxAttempts	терминальный, без дальнейших попыток
DONE	TaskExecutionStatus.SUCCESS	терминальный

Заказ (OrderStatus) при этом может стать CANCELLED «успешно» с точки зрения задачи (TaskExecutionStatus.SUCCESS) — отказ в резервировании склада это бизнес-результат, а не ошибка исполнения.

Конфиг = точки настройки и потенциальные буттлнеки

Параметр	Дефолт / нагрузочный профиль (dev-compose-all.yaml)	Чем ограничивает
server.tomcat.threads.max	10	сколько POST/GET /orders обрабатывается одновременно на входе — при 60 VU k6 это первый кандидат в буттлнек
server.tomcat.threads.max-queue-capacity	1000	сколько запросов ждут очереди, прежде чем Tomcat начнёт их отбрасывать
spring.datasource.hikari.maximum-pool-size	100	общий лимит соединений с Postgres — делят контроллер, поллер (SELECT...FOR UPDATE) и сохранение результата диспетчером
task-execution.poller.poll-interval-ms	1000 → 10	как часто перечитывается очередь; слишком редко при высоком притоке создания заказов → растущий бэклог NEW - задач
task-execution.poller.batch-size	10 → 100	сколько задач забирается за один тик

Параметр	Дефолт / нагрузочный профиль (dev- compose- all.yaml)	Чем ограничивает
		— должно быть согласовано с интервалом и ёмкостью даунстрима
task-execution.dispatcher.max-attempts / retry-delay	10 / 1s	предел ретраев и скорость их повторения
*_THREAD_POOL_SIZE (dispatcher/external-http)	не действует	см. footgun выше — виртуальные потоки без пула
STUB_*_LATENCY_MIN/MAX_MILLIS , *_REJECT/EXCEPTION_PROBABILITY	0..30000ms / 0..1	искусственная задержка и хаос на стороне даунстрима — основной управляемый источник нагрузки на сам конвейер

Внешний стаб как генератор задержек и отказов

```

STUB_INVENTORY_REJECT_ENABLED: true
STUB_INVENTORY_REJECT_PROBABILITY: 0.5
STUB_PRICING_EXCEPTION_ENABLED: true
STUB_PRICING_EXCEPTION_PROBABILITY: 0.3
STUB_SHIPPING_EXCEPTION_ENABLED: true
STUB_SHIPPING_EXCEPTION_PROBABILITY: 0.3
# + *_LATENCY_MIN_MILLIS / *_LATENCY_MAX_MILLIS на каждый эндпоинт

```

order-external-http-stub не трогает БД — вся логика в памяти, задержка через Thread.sleep внутри контроллера (StubUtils.randomSafeSleepMs). В dev-compose.yaml включены случайные отказы (reject/exception), в dev-compose-all.yaml — фиксированная латентность 100мс на каждый эндпоинт без отказов (чистый профиль для измерения пропускной способности, а не устойчивости к ошибкам).

Нагрузочное тестирование: k6, а не Grafana

В обоих репозиториях Grafana/Prometheus не настроены — есть только k6 -скрипт (test-scripts/create_and_poll.js), который печатает сводку метрик в консоль по завершении

прогона. Grafana сюда можно прикрутить отдельно (k6 умеет писать в InfluxDB/Prometheus remote-write или через `xk6-dashboard`), но в этом коде такой связки нет.

```
export const options = {
  scenarios: {
    create_and_poll: {
      executor: 'constant-vus', // пул из N VU крутит default() без
остановки
      exec: 'default',
      vus: 60, // число виртуальных пользователей
      duration: '10s', // период нагрузки
    },
  },
};

const createLatency = new Trend('create_latency_ms'); // POST /orders
const terminalLatency = new Trend('terminal_latency_ms'); // create →
терминальный статус
```

- **constant-vus** — фиксированное число виртуальных пользователей (60), каждый крутит `default()` в цикле всю `duration` (10s); в отличие от `constant-arrival-rate`, тут нагрузка не фиксирует RPS — при замедлении сервиса VU просто проводит больше времени в одной итерации, и RPS сам падает (это важно: `constant-vus` нагружает «давлением», а не «частотой запросов»).
- **createLatency vs terminalLatency** — первая метрика мерит только `POST /orders` (синхронный путь), вторая — время от создания до `CREATED_SUCCESSFULLY / CANCELLED` (весь асинхронный конвейер + клиентский поллинг). Разрыв между ними — это и есть «стоимость» очереди/конвейера отдельно от стоимости самого HTTP-запроса.
- Поллинг статуса в самом k6-скрипте — отдельный экспоненциальный backoff (1→2→4...мс, до 15 итераций), не путать с backoffом ретраев на сервере (`task-execution.dispatcher.retry-delay`) — это два разных механизма на разных концах системы.

5. Вопросы с собеседований

Процесс vs поток

	Процесс	Поток
Память	своя изолированная, выделена ОС	делит heap с другими потоками того же процесса

	Процесс	Поток
Взаимодействие	не может напрямую влезть в память другого процесса	потоки одного процесса видят одни и те же объекты в heap
Что приватное	всё	стек вызовов (фреймы методов) и program counter

Стек потока хранит не сами объекты, а **ссылки** на них — сами объекты живут в общей куче (heap). Отсюда и все проблемы многопоточности: два потока держат ссылки на один и тот же объект в heap и могут менять его одновременно, ничего не зная друг о друге.

Race condition: определение

Несколько потоков работают с общими данными без координации → результат недетерминирован, зависит от того, в каком порядке потоки успели сделать свои операции. Классика — некорректный инкремент: `x=10`, два потока читают `10`, оба прибавляют `1`, оба пишут `11` — одно изменение теряется.

Причина — `counter++` не атомарен, это три отдельных шага (прочитать → увеличить → записать), между которыми может вклиниться другой поток. На малом числе итераций (10, 80, 800) race condition может вообще не проявиться — это не значит, что его нет, на 8000+ итераций он стабильно виден.

Live-coding задача №1: многопоточный счётчик — пять версий подряд

Задача, которую дают на лайв-кодинге: есть `Counter` с полями-методами `increment()` / `get()`, 8 потоков по 100 000 инкрементов каждый, ожидаемый результат `800000`. Нужно объяснить, почему он не получается, и исправить.

Какие версии правильные: 0 и 4 — сломаны (демонстрация проблемы), 1, 2 и 3 — все три рабочие и равноценно правильные (разница между ними — не корректность, а цена/подход: лок с очередью потоков у 1 и 2 против CAS без блокировок у 3).

Версия 0 — без защиты (базовый race condition) ❌ сломана

```
private static class Counter {
    private int counter;

    public void increment() {
        counter++;
    }

    public int get() {
        return counter;
    }
}
```

```
    }  
}
```

Запуск даёт что-то вроде `expected: 800000, got: 773421` — конкретное число каждый раз разное и непредсказуемое, именно это и есть `race condition` вживую.

Версия 1 — `synchronized` -блок снаружи, в месте вызова  правильная

```
Thread t = new Thread(() -> {  
    for (int k = 0; k < incrementsPerThread; k++) {  
        synchronized (counter) {  
            counter.increment();  
        }  
    }  
});
```

Синхронизация по объекту `counter` — монитор один на всех, критическая секция (вызов `increment()`) выполняется строго одним потоком за раз. Результат уже стабильно `800000`.

Версия 2 — `synchronized` -метод (эквивалент, но синхронизация внутри класса, а не снаружи)  правильная

```
public synchronized void increment() {  
    counter++;  
}
```

Это ровно то же самое, что написать вручную:

```
public void increment() {  
    synchronized (this) {  
        counter++;  
    }  
}
```

Разница с версией 1 — чисто организационная (где стоит `synchronized`, снаружи или внутри класса), эффект на корректность одинаковый. **Проблема у обеих:** потоки выстраиваются в очередь на монитор — вся параллельность на этом участке теряется, остальные потоки просто простаивают, ничего не делая, пока один работает в критической секции.

Версия 3 — `AtomicInteger` , без блокировок вообще правильная (и предпочтительная)

```
private static class Counter {
    private final AtomicInteger counter = new AtomicInteger(0);

    public void increment() {
        counter.incrementAndGet();
    }

    public int get() {
        return counter.get();
    }
}
```

`java.util.concurrent.atomic` — классы, оптимизированные под многопоточность через аппаратную инструкцию **compare-and-swap (CAS)**: атомарно сравнить значение в памяти с ожидаемым и, если совпало, заменить на новое; если не совпало (кто-то успел изменить) — повторить попытку в цикле, без блокировки других потоков. Результат корректный и заметно быстрее `synchronized` — нет очереди, только короткий `retry` при коллизии.

Важно: именно `incrementAndGet()` одним вызовом, а не `increment()` + `get()` отдельно — между двумя отдельными вызовами другой поток может вклиниться, и `get()` вернёт не то значение, которое «только что» увеличили.

Версия 4 — `volatile` вместо блокировки (финальная версия в репозитории — и она всё ещё сломана) сломана

```
private static class Counter {
    private volatile int counter;

    public Counter(int initValue) {
        this.counter = initValue;
    }

    public void increment() {
        counter++;    // volatile НЕ спасает: это всё ещё 3 отдельных шага
    }

    public int get() {
        return counter;
    }
}
```

```
}  
}
```

Это специально оставленный «капкан»: `volatile` гарантирует только **видимость** — что при чтении поля поток увидит последнее записанное значение, а не устаревшее из своего кэша ядра. Он **не делает составную операцию атомарной**. `counter++` как было тремя шагами (прочитать/увеличить/записать), так и осталось — IDE прямо подсвечивает это как «non-atomic operation on volatile field». На 800 000 инкрементов результат снова не совпадёт с ожидаемым, несмотря на `volatile`.

Это тот же механизм, что в главе 5 конспекта («volatile — гарантия видимости, не атомарности») и в главе 6 (формальная happens-before семантика `volatile write → volatile read`) — просто показан на интервью-формате задачи.

Live-coding задача №2: многопоточный Singleton — пять этапов

Классика собеседов: `getInstance()` должен возвращать единственный экземпляр, создавать его лениво, и корректно работать при параллельных вызовах.

Этап 1 — eager-инициализация через `static final`

```
private static final MultithreadedSingleton INSTANCE = new  
MultithreadedSingleton();  
  
public static MultithreadedSingleton getInstance() {  
    return INSTANCE;  
}
```

Самый простой вариант — потокобезопасен «бесплатно», потому что статическая инициализация поля гарантированно выполняется один раз, до того как класс станет доступен другим потокам (гарантии JVM на уровне загрузки класса). Но это **не лениво** — объект создаётся сразу при загрузке класса, даже если он может никогда не понадобиться, что дорого, если конструктор тяжёлый. Отдельный риск: если в статической инициализации вылетит исключение, class loader пометит класс как ошибочный, и он больше никогда не сможет загрузиться в этой JVM.

Этап 2 — наивная ленивая инициализация (работает только в один поток)

```
private static MultithreadedSingleton instance;  
  
public static MultithreadedSingleton getInstance() {  
    if (instance == null) {  
        instance = new MultithreadedSingleton();  
    }  
}
```

```

    }
    return instance;
}

```

В однопоточной среде — ок. В многопоточной — race condition: поток 1 проверяет `instance == null`, получает `true`, но до того как успевает присвоить — его прерывают; поток 2 тоже видит `null` и тоже создаёт инстанс. В итоге создаются два разных объекта вместо одного — именно то, чего синглтон обязан не допускать.

Этап 3 — `synchronized` на весь метод (корректно, но дорого)

```

public static synchronized MultithreadedSingleton getInstance() {
    if (instance == null) {
        instance = new MultithreadedSingleton();
    }
    return instance;
}

```

Race condition исправлен — два потока физически не могут оказаться внутри одновременно. Но плата высокая: **каждый** вызов `getInstance()` берёт лок, даже когда `instance` давно создан и по-хорошему блокировка вообще не нужна. Если метод дёргается часто — все вызовы без необходимости выстраиваются в очередь.

Этап 4 — `double-checked locking` без `volatile` (быстрее, но со скрытым багом)

```

public static MultithreadedSingleton getInstance() {
    if (instance == null) { // быстрая проверка без
        лока
        synchronized (MultithreadedSingleton.class) {
            if (instance == null) { // повторная проверка
                внутри лока
                    instance = new MultithreadedSingleton();
            }
        }
    }
    return instance;
}

```

Идея: проверить `null` снаружи блокировки (когда объект уже создан, большинство вызовов вообще не трогают `synchronized`), и только если похоже, что объекта ещё нет — взять лок и проверить **ещё раз изнутри** (потому что пока первый поток ждал захвата монитора, второй мог успеть создать инстанс первым).

Скрытая проблема, из-за которой IntelliJ подсвечивает именно эту конструкцию:

`instance = new MultithreadedSingleton()` — не атомарная операция. Сначала выделяется память под объект, вызывается конструктор, и только потом ссылка присваивается полю — это минимум два отдельных шага. Без `volatile` компилятор/CPU формально вправе переставить их местами (instruction reordering), и другой поток может увидеть уже не- `null` ссылку в `instance`, когда объект за ней ещё не до конца сконструирован. Тот же сценарий **broken publication**, что разбирался в главе 6 на примере `Holder`.

Этап 5 — double-checked locking + `volatile` (финальная рабочая версия, из репозитория)

```
public class MultithreadedSingleton {

    private static volatile MultithreadedSingleton INSTANCE;

    private MultithreadedSingleton() {
    }

    public static MultithreadedSingleton getInstance() {
        if (INSTANCE == null) {
            synchronized (MultithreadedSingleton.class) {
                if (INSTANCE == null) {
                    INSTANCE = new MultithreadedSingleton();
                }
            }
        }
        return INSTANCE;
    }
}
```

Единственное отличие от этапа 4 — `volatile` на поле `INSTANCE`. Через happens-before (`volatile write` → `volatile read`) это гарантирует: если какой-то поток увидел не- `null` ссылку в `INSTANCE`, он гарантированно видит и полностью отработавший конструктор — reordering, который ломал этап 4, запрещён. Быстрый путь (без лока) остаётся быстрым для всех вызовов после первой инициализации, а корректность гарантирована на все 100%.

Типы `synchronized` — на чём именно висит монитор

`synchronized` всегда работает по объекту-монитору, вопрос только в том, какой именно объект выбран — это и есть ответ на вопрос «какие виды `synchronized` бывают»:

Вид	Пример	Монитор
<code>synchronized</code> -метод экземпляра	<code>public synchronized void increment()</code> (счётчик, версия 2)	<code>this</code> — конкретный объект, на котором вызван метод
<code>synchronized</code> -блок по <code>this</code>	<code>synchronized (this) { ... }</code>	<code>this</code> , но область поменьше, чем весь метод
<code>synchronized</code> -блок по явному объекту	<code>synchronized (counter) { ... }</code> (счётчик, версия 1); <code>synchronized (MakeStepsInOrder.LOCK)</code>	произвольный объект, который сами выбрали как «ручку» лока
<code>synchronized</code> -статический метод / блок по <code>Class</code>	<code>synchronized (MultithreadedSingleton.class)</code> (singleton, этапы 3–5)	объект <code>Class</code> — один на весь класс, общий для всех экземпляров

Разница между вторым и третьим вариантом принципиальна: `synchronized(this)` — это лок на конкретном объекте, два разных экземпляра друг другу не мешают; `synchronized` по отдельному выделенному объекту (`private final Object lock`) — тот же эффект, но вы явно контролируете, что именно этот объект никто больше не использует как монитор (так сделано в `Account` ниже — у каждого счёта свой приватный `lock`, а не `this`, чтобы снаружи никто случайно не засинхронизировался на самом объекте `Account`).

Deadlock: как его получить и как избежать

Классическая демонстрация — перевод денег между двумя счётами одновременно в обе стороны (`DeadlockAccountTransferService`). У каждого `Account` свой приватный объект-лок.

Наивная (ломающаяся) версия — захватываем лок счёта-отправителя, потом лок счёта-получателя, в порядке параметров:

```
public static void makeTransfer(Account accountFrom, Account accountTo, int amount) {
    synchronized (accountFrom.lock) {
        synchronized (accountTo.lock) {
            accountFrom.balance -= amount;
            accountTo.balance += amount;
        }
    }
}
```

```

    }
}
}

```

Если один поток одновременно вызывает `makeTransfer(account1, account2, 50)`, а другой — `makeTransfer(account2, account1, 50)`, возможен classic **circular wait**: поток 1 захватил лок `account1` и ждёт лок `account2`; поток 2 в этот же момент захватил лок `account2` и ждёт лок `account1`. Оба ждут друг друга вечно — приложение зависает.

Deadlock в общем случае требует одновременно четырёх условий (условия Коффмана) — если нарушить хотя бы одно, deadlock невозможен:

Условие	Смысл	В примере выше
Mutual exclusion (взаимное исключение)	ресурс нельзя шарить, им владеет не больше одного потока за раз	каждый lock — обычный монитор, его держит только один поток
Hold and wait (удержание и ожидание)	поток держит один ресурс и при этом ждёт второй, не отпуская первый	поток держит <code>firstLock</code> и блокируется на <code>secondLock</code>
No preemption (отсутствие вытеснения)	захваченный лок нельзя отобрать силой у потока извне	захвативший монитор поток сам решает, когда выйти из <code>synchronized</code>
Circular wait (циклическое ожидание)	есть цикл из потоков, где каждый ждёт ресурс, занятый следующим в цепочке	поток 1 ждёт лок, который держит поток 2, а поток 2 ждёт лок, который держит поток 1

Проще всего на практике убрать именно `circular wait` — это и делает фикс ниже (единый порядок захвата).

Исправленная версия (то, что реально в репозитории) — всегда захватывать локи в одном и том же порядке, независимо от направления перевода, определяя порядок по `hashCode()` объектов-локов:

```

public static void makeTransfer(Account accountFrom, Account accountTo, int
amount) {
    var firstLock = accountFrom.lock.hashCode() <= accountTo.lock.hashCode()
        ? accountFrom : accountTo;
    var secondLock = accountFrom == firstLock ? accountTo : accountFrom;

    synchronized (firstLock.lock) {
        synchronized (secondLock.lock) {

```

```

        accountFrom.balance -= amount;
        accountTo.balance += amount;
    }
}

```

Теперь оба потока — что при переводе 1→2, что при 2→1 — сначала пытаются взять лок счёта с меньшим `hashCode()`, и только потом второй. Циклического ожидания не может возникнуть в принципе: один и тот же поток всегда «выигрывает» первый лок первым, если он меньше по порядку, а второй поток на этом же локе просто ждёт своей очереди — без встречного захвата.

Другие стандартные способы не допустить deadlock (не показаны в коде, но стоит знать для собеса): `tryLock(timeout)` у `ReentrantLock` — не ждать лок бесконечно, а отступить и повторить; сократить область, которую вообще нужно держать под локом; свести к одному общему локу вместо нескольких, если это приемлемо по производительности.

Пулы потоков — какие бывают и когда какой

Частый вопрос на собесе — не просто «что такое пул потоков», а «какие виды есть и чем они отличаются». Создаются через фабричные методы `Executors` (сами классы — `ThreadPoolExecutor` под капотом):

Пул	Создание	Размер	И
Fixed	<code>Executors.newFixedThreadPool(n)</code>	ровно <code>n</code> потоков, остальные задачи ждут в неограниченной очереди	1 1 1 1 (
Cached	<code>Executors.newCachedThreadPool()</code>	без верхней границы — создаёт новый поток под каждую задачу, если все заняты, неиспользуемые потоки убивает через 60 секунд простоя	1 3 у 1 с 1 } 1

Пул	Создание	Размер	
Scheduled	<code>Executors.newScheduledThreadPool(n)</code>	фиксированный, как Fixed	
ForkJoinPool (в т.ч. <code>commonPool()</code>)	<code>new ForkJoinPool()</code> / дефолтный для параллельных стримов	по числу ядер CPU, work-stealing между потоками	
Virtual-thread-per-task (Java 21+)	<code>Executors.newVirtualThreadPerTaskExecutor()</code>	фактически безлимитный, виртуальный поток на задачу	

Общий принцип выбора: CPU-bound и предсказуемая нагрузка → `Fixed` / `ForkJoinPool`; много I/O-ожидания и непредсказуемые всплески → `Cached` (с осторожностью) или виртуальные потоки; что-то по расписанию → `Scheduled`. Самодельную `BlockingQueue` под задачи городить не нужно — у `ExecutorService` очередь уже внутри (см. разбор ДЗ №1).

`wait()` / `notifyAll()` — согласование очереди между потоками

Два живых примера одного и того же паттерна: заставить потоки выполнять действия строго по очереди.

`MakeStepsInOrder` — «ноги робота» должны шагать по порядку

```
public void makeStep() throws InterruptedException {
    synchronized (MakeStepsInOrder.LOCK) {
        while (!isCurrentLegTurn()) {
            MakeStepsInOrder.LOCK.wait();
        }
    }
}
```

```

    }
    System.out.println("Make step by leg: " + number);
    currentLegTurn = (currentLegTurn + 1) % legsCount;
    MakeStepsInOrder.LOCK.notifyAll();
}
}

```

5 потоков («ног»), общий счётчик `currentLegTurn`, общий лок `LOCK`. Если сейчас не твоя очередь — уходишь в `wait()` (отпуская лок на время ожидания); как только отработал — сдвигаешь счётчик на следующую ногу и будишь всех (`notifyAll()`), а не одного (`notify()`), потому что проснуться должна конкретно та нога, чья очередь настала следующей, а не случайная — `notify()` разбудил бы один случайный поток, и если это не «та нога», она снова уснёт, а нужная так и не проснётся.

Почему `while`, а не `if` вокруг `wait()`: после пробуждения поток не гарантированно получает то состояние, которое ждал (spurious wakeup, либо проснулся не по адресу из-за `notifyAll()`) — нужно перепроверить условие заново, а не слепо продолжать.

Остановка потоков — через `interrupt()` + проверку

`Thread.currentThread().isInterrupted()` в цикле `run()`, с перевыставлением флага прерывания в `catch (InterruptedException)` (ровно то, что разбиралось в дополнении к модулю 1 про `interrupt()`).

PrintFooInOrder — тот же паттерн, но лок — это `this`

```

public synchronized void second() {
    while (counter != 1) {
        try {
            this.wait();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return;
        }
    }
    counter++;
    System.out.println("second");
    this.notifyAll();
}

```

Отличие от `MakeStepsInOrder` чисто организационное: вместо отдельного объекта `LOCK` используется сам `this` (монитор `Foo`) через `synchronized`-методы — `first()` / `second()` / `third()` обязаны выполняться строго в этом порядке для трёх

независимых потоков. Условие ожидания — не «чья очередь», а конкретное значение `counter`, остальная механика (`while + wait() + notifyAll()`) идентична.

Важная деталь по теме главы 5: `wait()` специально устроен так, что **отпускает** лок на время ожидания — в отличие от `Thread.sleep()` внутри `synchronized`, который лок держит всё время сна (см. «Дополнение к модулю 1»). Именно поэтому `wait()`, а не `sleep()`, используют для ожидания условия внутри `synchronized`-блока — иначе все потоки, ожидающие своей очереди, попросту заблокировали бы друг друга намертво.